



Puma optimizer (PO): a novel metaheuristic optimization algorithm and its application in machine learning

Benyamin Abdollahzadeh¹ · Nima Khodadadi² · Saeid Barshandeh³ · Pavel Trojovský¹ · Farhad Soleimanian Gharehchopogh⁴ · El-Sayed M. El-kenawy⁵ · Laith Abualigah^{6,7,8} · Seyedali Mirjalili^{9,10}

Received: 1 September 2023 / Revised: 10 November 2023 / Accepted: 24 November 2023 / Published online: 19 January 2024
 © The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

Optimization techniques, particularly meta-heuristic algorithms, are highly effective in optimizing and enhancing efficiency across diverse models and systems, renowned for their ability to attain optimal or near-optimal solutions within a reasonable timeframe. In this work, the Puma Optimizer (PO) is proposed as a new optimization algorithm inspired from the intelligence and life of Pumas in. In this algorithm, unique and powerful mechanisms have been proposed in each phase of exploration and exploitation, which has increased the algorithm's performance against all kinds of optimization problems. In addition, a new type of intelligent mechanism, which is a type of hyper-heuristic for phase change, is presented. Using this mechanism, the PO algorithm can perform a phase change operation during the optimization operation and balance both phases. Each phase is automatically adjusted to the nature of the problem. To evaluate the proposed algorithm, 23 standard functions and CEC2019 functions were used and compared with different types of optimization algorithms. Moreover, using the statistical test T-test and the execution time to solve the problem have been discussed. Finally, it has been tested using four machine learning and data mining problems, and the results obtained from all the analysis signifies the excellent performance of this algorithm against all kinds of problems compared to other optimizers. This algorithm has performed better than the compared algorithms in 27 benchmarks out of 33 benchmarks and has obtained better results in solving the clustering problem in 7 data sets out of 10 data sets. Furthermore, the results obtained in the problems of community detection and feature selection and MLP were superior. The source codes of the PO algorithm are publicly available at <https://www.mathworks.com/matlabcentral/fileexchange/157231-puma-optimizer-po>.

Keywords Optimization · Metaheuristic algorithm · Puma optimization algorithm · Machine learning · Global optimization · Automatic phase change

Abbreviations

PO	Puma optimizer	X_i	Current search agent
<i>rand</i>	A random number between [0,1]	Ub	The Upper bound of search spaces
UM	Unimodal	<i>Cost</i>	The cost of the solution
<i>randn</i>	Random numbers in the normal interval	Lb	The lower bound of search spaces
MM	Multimodal	<i>exp</i>	Exponential function
<i>mean</i>	Mean function	EB	Evolutionary based
CM	Composition	SB	Swarm-based
<i>Iter</i>	Current number of iterations	HP	Human-based
SM	Scalable Multimodal function	iterations	Max number of iterations
FM	Fixed-dimension multimodal function	PB	Physic-based
$Puma_{male}$	Best search agent	Best	The best result
<i>MaxIter</i>	Max number of iterations	population	Number of pumas
<i>Npop</i>	Total number of search agent	Worst	The worst result
<i>cos</i>	Function of cosine	Mean	Average results
		MLP	Multi-layer perceptron
		STD	Standard deviation
		FS	Feature selection

Extended author information available on the last page of the article

CD	Community detection
FCCD	The face-centered central composite design
DC	Data clustering
SCA	Sine cosine algorithm
FHO	Fire hawk optimizer
SAO	Smell agent optimization
TSA	Tunicate swarm algorithm
GWO	Gray wolf algorithm
MFO	Moth-flame optimization algorithm
WOA	Whale optimization algorithm
DMOA	Dwarf mongoose optimization algorithm
PSO	Particle swarm optimization
BAT	Bat algorithm
GA	Genetic algorithm
ABC	Artificial bee colony
FFA	Farmland fertility algorithm
BBO	Biogeography-Based optimizer
SMA	Slime mould algorithm
CSO	Cuckoo search optimization

1 Introduction

Optimization refers to the process of selecting the most appropriate values for specific parameters within a system to meet all design criteria while keeping expenses to a minimum. Optimization is a ubiquitous challenge in all fields of science. It demands the development of new and advanced algorithms to tackle increasingly complex problems. Traditional optimization algorithms have constraints including single-based solutions, converging to local optima, and difficulties in unknown search spaces [1]. Over the past few decades, many researchers have built meta-heuristics, which are algorithms designed to overcome these shortcomings to address unresolved optimization issues.

Optimization involves identifying the decision variables that result in the best possible solution for one or more fitness functions [2]. Before the emergence of heuristic optimization, analytical approaches were the primary method of addressing optimization problems. Within the context of first and linear combinations, analytical approaches concentrate on gathering information pertaining to individual or constraint-penalized fitness function variables, as well as the primary data regarding the fitness function and violation of constraints. This approach made locating the exact optimal solution for linear or convex non-linear optimization issues simpler [3]. However, this method was weak in more complicated optimization

problems, which could result in local optima trapping [4, 5] or various premature convergence inability for stochastic or unexplained search space problems.

The characteristics of real-world optimization problems often include stochastic behavior and uncertain search space. This has resulted in the development of meta-heuristic algorithms that are free of derivatives and do not require restrictive assumptions. Metaheuristic algorithms are highly flexible and can be utilized to solve various optimization problems [6, 7]. Metaheuristic algorithms are highly flexible and can be used to solve various optimization problems, providing valuable solutions in complex and dynamic environments. Mathematical optimization methods can be classified as stochastic and deterministic. Deterministic approaches, such as linear and non-linear programming, use the gradient knowledge of the problem for explore the problem space and find a solution. While these techniques are helpful for linear search space problems, they are impressionable to local optima trapping when applied to non-linear search space problems, such as real-world non-convex problems [8]. Therefore, modifying or hybridizing these algorithms is necessary to address such issues.

Metaheuristics are a well-known category of algorithms specifically designed to address intricate optimization challenges. They are based on evolutionary algorithms (EA) [9], swarm intelligence (SI) [10, 11], physics-based methods (PH) [12, 13], and human-based (HU) [14, 15] methods. The simplicity of mathematical models in the SI is one of the main reasons for higher popularity among all classes. Metaheuristics have gained significant popularity in the field of optimization and various other disciplines. Stochastic optimization algorithms, which fall under this category, find utility in numerous scientific domains [16] as well as industries [17, 18]. However, since the focus of this paper primarily lies in the theoretical aspect, the applications of these algorithms are not extensively elaborated on. Researchers who are interested in exploring the practical applications are encouraged to refer to other relevant sources.

The primary objective of this research paper is to introduce a novel metaheuristic algorithm named Puma Optimizer (PO), which is specifically designed to address optimization problems characterized by extensive search spaces. It is based on animals' behavior and mimics Pumas' behavior. The proposed algorithm is evaluated against five popular and recent metaheuristic algorithms using 23 standard functions, CEC2019, and machine learning problems. However, it is essential to note that the performance of an algorithm on one optimization problem does not guarantee its success on another problem with different characteristics, according to the No Free Lunch theorem for optimization [19]. Hence, it is crucial to

thoroughly analyze and tailor metaheuristic algorithms to suit particular problem domains in order to optimize their efficiency and efficacy to the fullest extent.

The main contributions of this paper are presented as follows.

- Introducing a new and intelligent way to change phase and balance automatically, which is presented for the first time.
- An innovative optimization algorithm named Puma Optimizer is introduced for global optimization Machine learning and data mining problems.
- The performance of the Puma Optimizer is calculated utilizing three challenging problems: 23 standard test benchmark, CEC2019, and machine learning problem.
- The performance of the Puma Optimizer is analyzed with the state-of-the-art swarm intelligence (SI) algorithms, physical inspiration algorithms, and biological inspiration-based algorithms.
- Providing a new method in the exploration phase using two operators and a new approach.
- Presenting a new method in the exploitation phase using three new mechanisms and methods.

The subsequent sections of the paper are structured as follows: Sect. 2 presents an elaborate literature review, while Sect. 3 provides a comprehensive explanation of the inspiration behind and the mathematical model of the newly proposed Puma Optimizer. The outcomes derived from the experimentation are discussed in Sect. 4. The paper concludes by summarizing the findings and outlining potential future direction for research in Sect. 5.

2 Related works

Due to the increasing popularity of meta-heuristic algorithms for solving different types of problems, different types of meta-heuristic algorithms have been presented. Each of these algorithms has its characteristics and methods for solving optimization problems. The sources of inspiration for optimizing algorithms can include different types of natural phenomena, including animals and humans, physics, and evolution. According to the source of inspiration, various algorithms have been presented. In the case of mathematical modeling of natural behaviors, these algorithms have led to the creation of new methods and techniques in optimization methods. Several optimization methods have been proposed in the literature, each with unique inspirations. The sources of inspiration for metaheuristic algorithms can be broadly categorized into several natural sources.

The primary sources of inspiration for metaheuristic algorithms are animal life and behavior. These algorithms, which were influenced by the group behavior of social

insects and animals, have also made a major contribution to the growth of metaheuristic algorithms. The Golden Eagle Optimizer (GEO) [20], a metaheuristic algorithm inspired by nature, is one such approach. It simulates the intelligence of golden eagles in hunting by adjusting its speed at different spiral trajectory phases. The Whale Optimization Algorithm (WOA) [21], a new meta-heuristic optimization algorithm, is a different method. It was developed in response to the social behavior of humpback whales engaged in bubble-net hunting. Another algorithm that has been discussed in the literature [22] and is inspired by a group of sailfish's hunting activity is called Sail Fish Optimizer (SFO). Two populations create SFO: the sailfish population improves the search for the best answer, and the sardine population diversifies the search area. The behavior of Coaties in nature served as the model for an effective and efficient optimization technique, which was reported in [23]. Exploration and exploitation are the two stages of the algorithm's mathematical modeling. The authors have evaluated the algorithm's performance by testing it on 51 standard objective functions of varying complexity and dimensions, comparing it to 11 other optimization algorithms. The results show that the Coaties-inspired algorithm outperforms the others. The Ant Lion Optimizer (ALO) [20] is another optimization algorithm inspired by nature. It simulates the hunting process of antlions, including five key hunting prey steps. Another algorithm, the Pathfinder Algorithm (PFA) [21], has been proposed to address optimization problems with a different structure. It takes inspiration from the collective change of animal groups and stimulates the hierarchy of swarm leadership to find the most suitable place or prey for food.

Mathematical models or physical processes serve as the second source of inspiration. For instance, Abualigah et al. [6] presented the Arithmetic Optimization Algorithm (AOA), a novel meta-heuristic method. This method makes use of how popular arithmetic operators, such multiplication, division, addition, and subtraction, behave in various search spaces. The AOA is represented mathematically and used to carry out optimization procedures. Using a mathematical model based on sine and cosine functions, Mirjalili [25] developed the Sine Cosine technique (SCA), a population-based optimization technique. SCA fluctuates outward or towards the best solution and implements multiple random and adaptive variables to discover and utilize the search space in various optimization milestones. Additionally, Faramarzi et al. [22] introduced a new Equilibrium Optimizer (EO) inspired by control volume mass balance models and uses each solution as a search agent with its position to achieve the equilibrium state.

The motivation of the human population and its activities serves as another source of inspiration for metaheuristic algorithms. The Driving Training-Based Optimization (DTBO)

algorithm [27] is inspired by how people behave when learning to drive. The presentation is broken down into three stages: practice, pattern creation by the teacher, and teaching by the driving instructor. The authors employed 53 industry-standard functions, including CEC 2017 functions, to assess and test the DTBO algorithm. The Group Teaching Optimization Algorithm (GTOA) [28], which is inspired by a community teaching mechanism, is a new metaheuristic algorithm that the study suggests be used to tackle various optimization issues. Four basic rules are described to adjust group teaching for optimization techniques, and a group teaching model is constructed. A new optimization algorithm has been introduced based on human behavior in elections and voting processes to choose a leader [23]. The search agents are guided based on the leader, and the algorithm includes two phases: exploration and exploitation. The authors used 33 objective functions of different dimensions and complexity to test this algorithm. The outcomes demonstrate how effectively the algorithm handles a range of optimization issues. In order to solve issues with numerical and structural design optimization, Jahangiri et al. [30] presented Interactive Autodidactic School (IAS), a novel efficient and reliable metaheuristic method. The IAS is a population-based algorithm inspired by experiences in an autodidactic school where students improve their knowledge through self-teaching, collaborative discourse, feedback, and rivalry.

Art serves as the last motivation for metaheuristic algorithms. The Harmony Search Algorithm (HSA) has drawn interest from all around the world due to its well-balanced mix of exploration and exploitation and ease of use [31]. The HSA consists of three operators: random search, harmony memory considering rule, and pitch adjusting rule. These three operators were inspired by the musical performance process. An other optimization method that takes influence from art is the Stochastic Paint Optimizer (SPO) [32]. In order to find the ideal color, a population-based optimizer uses the search region like a canvas and applies many color schemes to it. There are four straightforward color combination principles that the SPO algorithm may explore and exploit without the need for internal parameters. Apart from the sources mentioned above, metaheuristic algorithms have been influenced by various other fields.

Finally, the technical studies conducted in the literature can be divided into three parts. In the first section, researchers attempt to improve optimizer with other mathematical or random mechanism to enhance performance. Popular approaches are evolutionary operators, chaotic maps, opposite based-learning, and local searches. An overview of the popular optimization algorithms is stated in Table 1.

According to all the explanations in this section, each optimization algorithm has advantages and disadvantages.

One of the significant drawbacks of the optimizer algorithms is the poor performance in the components of intensification and diversity, the lack of balance in the phases of exploration and exploitation, stuck at the local optimum, the lack of adaptation to different mechanisms to solve discrete problems, and high execution time.

The ability of the optimizer algorithms can be increased by adapting to powerful mechanisms that increase the variety of output solutions or moving quickly towards the best solution to explore the entire optimization space easily. However, in order to minimize the execution time to a reasonable level, these techniques should be as computationally simple as possible. Moreover, intelligent mechanisms and procedures can be used to balance the exploration and exploitation phases so that the algorithm's performance increases significantly. Regarding the explanations given, these cases motivated us to provide a robust algorithm with powerful mechanisms in both exploration and exploitation phases and minimal execution time; then, a new type of intelligent mechanism for phase change is introduced to get maximum performance from the proposed algorithm. Finally, due to its unique features, the proposed algorithm can be used to solve a wide range of optimization problems in different optimization spaces. We have studied global optimization problems and machine learning problems case by case.

3 Puma optimizer (PO)

In this part, the main inspiration of the Puma algorithm is reviewed and then the proposed algorithm is fully described along with the mathematical model.

3.1 Inspiration

Puma is a big cat species from the subfamily of small cats native to the American continent and is also known by other names such as cougar and mountain lion, as shown in Fig. 1. The habitat of this animal ranges from the Andes Mountains in South America to the Yukon in Canada. The mountain lion is the largest feline in the Americas after the jaguar. Puma is a highly adaptable animal that lives in different habitats and feeds on other prey. It is primarily nocturnal but can be seen during the day (Fig. 2).

Puma is an ambush predator whose main prey is ungulates, including deer [50]. However, it also feeds on small prey such as rodents and insects and sometimes attacks domestic livestock [51]. It prefers dense scrub and rocky areas where it is possible to ambush [52]. It lives in open plains too. Puma, like other cats, is territorial and has a very large territory. It has a low population density and, in some areas, it competes with other predators such as the

Table 1 An Overview of popular optimization algorithms

References	Type	Algorithm	Year	Inspiration	Pros	Cons
[10]	SB	Particle Swarm Optimization (PSO)	1995	The natural behaviors of swarm particles	Good ability of the intensification component	Sensitivity to parameters
[24]	SB	Firefly Algorithm (FA)	2009	Flashing light of fireflies in oceans	Good ability of the diversification component	Poor ability of the intensification component
[25]	SB	Ant Colony Optimization (ACO)	2006	Ants deposit pheromones on the ground	Powerful exploitation phase	Falling into the trap of local optimality in some problems
[26]	SB	Whale Optimization Algorithm (WOA)	2016	The behavior of humpback whales	Simplicity, efficiency, and static parameter-free	Easy to fall into the local optimum
[27]	SB	Artificial Bee Colony (ABC)	2007	The behaviors of the honey bees colony	Good ability to solve unimodal problems	The difficult setting of parameters
[28]	SB	Salp Swarm Algorithm (SSA)	2017	The behavior of salps navigating in oceans	Simplicity, efficiency, and static parameter-free	Slow convergence speed
[29]	SB	Mountain Gazelle Optimizer (MGO)	2022	The social life and hierarchy of wild mountain gazelles	Good performance in both exploitation and exploration phases	If the dimensions of the problem increase, the execution time also increases
[30]	SB	Lion Optimization Algorithm (LOA)	2016	Lifestyle and cooperation of lions	A great balance between exploration and exploitation	Poor performance against CEC issues
[31]	SB	Artificial Gorilla Troops Optimizer (GTO)	2021	The behavior of gorilla troops' social intelligence in nature	Powerful diversity component	Average execution time
[33]	SB	Artificial lizard search optimization (ALSO)	2021	The behavior of southern flying squirrels	Ability to escape the local optimality trap in some problems	High execution time in large problem
[34]	SB	African Vultures Optimization Algorithm (AVOA)	2021	The behavior of African vultures' foraging and navigation behaviors	Very low running time	Falling into the trap of local optimality in some problems
[35]	SB	Marine Predators Algorithm (MPA)	2020	Predators foraging strategy in oceans	Good ability of the intensification component	Easily fall into a local optimum
[9]	EB	Differential evolution (DE)	1997	The natural phenomenon of evolution	Production solutions have a very good variety	Poor intensification Component
[36]	EB	Genetic Algorithm (GA)	1992	Darwinian theory of evolution	Powerful exploration phase	Poor Exploitation phase
[37]	SB	Tree Growth Algorithm (TGA)	2018	Competition of trees for acquiring food and light	Good variety of output solutions	Imbalance in exploration and exploitation phases
[38]	SB	Invasive Tumor Growth	2015	Kidney process	Locally optimal trap escape using levy	Weakness in the exploitation phase
[39]	SB	Coati Optimization Algorithm (COA)	2023	The behavior of Coati in nature	High ability to balance exploration and exploitation No need for control parameter setting process	No guarantee of achieving the global optimum time consuming

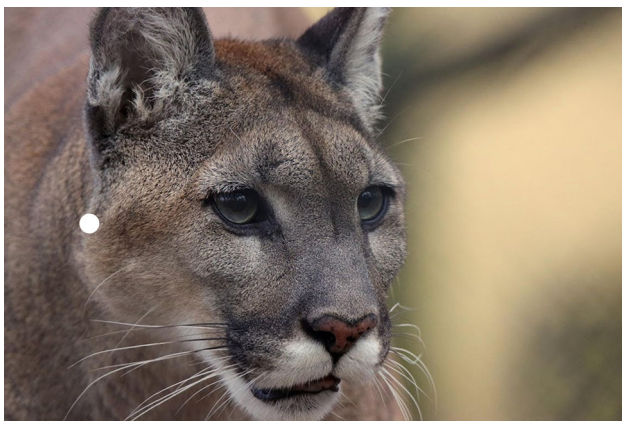
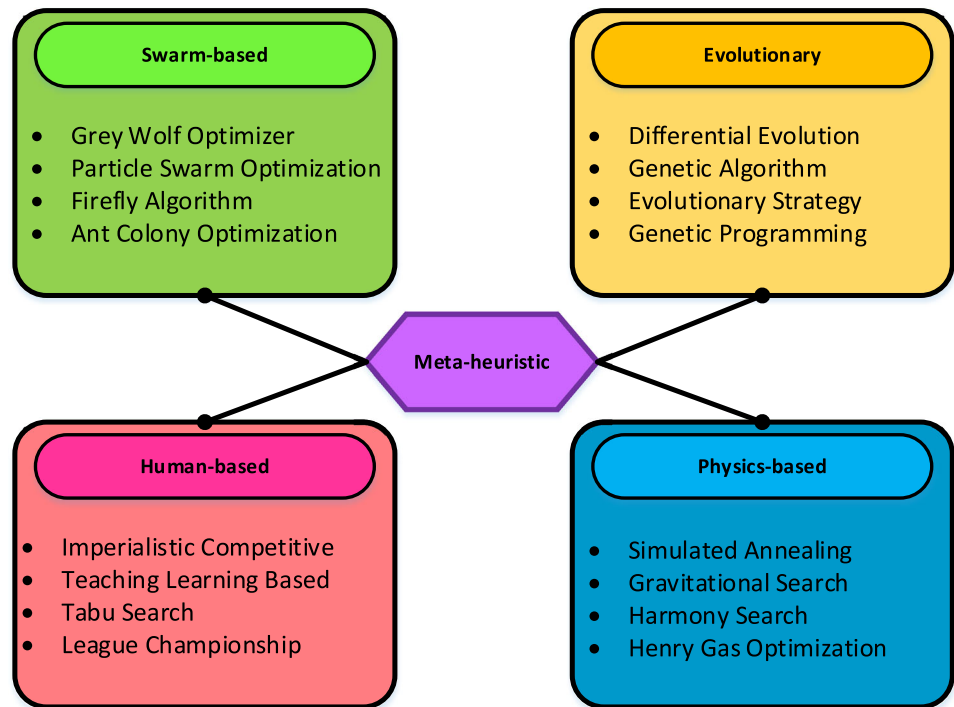
Table 1 (continued)

References	Type	Algorithm	Year	Inspiration	Pros	Cons
[40]	EB	Biogeography-Based Optimizer (BBO)	2008	Biogeography related to species migration	Simplicity and efficiency	Sensitive to the curse of dimensionality
[41]	PB	Simulated Annealing (SA)	1983	Annealing in metallurgy	Easy to code and use	Very slow and sensitive to parameter
[42]	PB	Big Bang–Big Crunch (BB-BC)	2006	The evolution of the universe	Good ability in the exploitation phase	Easily trapped in local optima
[23]	HB	Election-Based Optimization Algorithm (EBOA)	2022	Human behavior in the election of the leader	High ability to balance exploration and exploitation No dependence of the update process on specific members	Need to adjust R and N_c parameters. No guarantee of achieving the global optimum time consuming
[43]	PB	Central Force Optimization (CFO)	2007	The metaphor of gravitational kinematics	Poor exploration phase	sensitivity to parameters
[44]	PB	Electromagnetic Field Optimization	2016	The behavior of electromagnets	Avoid the local optimal point	sensitivity to parameters
[45]	HB	Driving Training-Based Optimization (DTBO)	2022	The learning process of humans in driving	High ability to balance exploration and exploitation No dependence of the update process on specific members	Need to adjust P parameter No guarantee of achieving the global optimum
[12]	PB	Gravitational Search Algorithm (GSA)	2009	The law of gravity and mass interactions	Good at finding the global optimum	slow convergence speed and getting stuck in local minima in the last iterations
[46]	PB	Henry Gas Solubility Optimization (HGSO)	2019	The behavior of Henry's law	A wide variety of production solutions	having slow convergence and being stuck in a local best solution
[47]	PB	Multi-verse Optimizer (MVO)	2016	Multi-verses theory	Powerful diversity component	less simplicity and the need for a high number of function evaluations at each iteration
[14]	HB	Teaching learning-based algorithm (TLBO)	2011	The influence of a teacher on the output of learners	Very strong convergence ability	Easy to fall into a local search
[48]	EB	Socio Evolution	2018	Learning Optimization Algorithm (SELOA) & Social learning behavior of humans	Good ability in the exploitation phase	Easily trapped in local optima
[49]	HB	Collective Decision Optimization	2017	Human decision-making characteristics	Very strong convergence ability	Easily getting stuck in local minima

jaguar, wolves, American black bears, and American alligators [53–56].

Extensive hunting and the development of human settlements after the arrival of Europeans in America endangered mountain lions in many areas [57]. For example, the

eastern puma subspecies became extinct in the east of North America, except for a small population of pumas in Florida [58]. In recent years, the puma's range has expanded from west to east and even reappeared in Connecticut on the Atlantic coast.

Fig. 1 Meta-heuristic classification**Fig. 2** The Puma

Puma is one of the slender and agile cats. This animal is the fourth largest feline among all felines. An adult puma is 60 to 90 cm tall at the shoulder. The length of the male puma is about 2.4 m from the nose to the end of the tail [59]. A female puma is about 1.5 m long and generally, the length of this animal is considered to be between 1.50 and 2 m. The length of the tail alone is between 63 and 95 cm. A male puma weighs between 55 and 100 kg, with an average of 64 kg. The female weighs between 29 and 64 kg, and the average weight of the female is about 55 kg [60]. The puma size is smaller in tropical regions and more significant in Polar Regions. The largest puma with a weight of 105.2 kg was hunted in 1901 [55]. In different areas, pumas may be smaller or larger than jaguars, but

they are usually smaller, weaker, and have less muscle mass compared to jaguars [55]. Cougars, due to the lack of a specialized larynx and laminar bone, are unable to roar like leopards; Therefore, despite their large size, pumas are not classified as big cats [56, 57].

Puma can sprint, though. They are looking for prey to hunt. But they usually ambush for hunting. To hunt, they pass through bushes and trees or other covered places and with a powerful leap, fall behind the prey and suffocate the prey with their fangs. Then, using its powerful fangs, the puma grabs the neck of some petite prey and breaks it on the ground.

Pumas can generally catch a large-sized game once every 2 weeks [58]. The puma drags the prey to a preferred place, covers it with bushes, and returns to nourish within a few days [9]. Like almost all felines, cougars are solitary animals. Only cubs and mothers live in groups, and matures rarely meet each other. Although usually solitary, they share prey and seem to organize themselves into petite communities defined by the dominant male's territory. Cougars in these fields socialize more with each other than with other foreign cougars. Investigation has shown that the smallest range for a puma is 25 square kilometers (9.7 square miles), and the largest range is 1,300 square kilometers (500 square miles) for males [59]. Male territories include territories of female cougars that overlap with each other, and males are permanently marking their territories. The territory of pumas depends on the abundance of pumas in the area or the vegetation and the abundance of prey

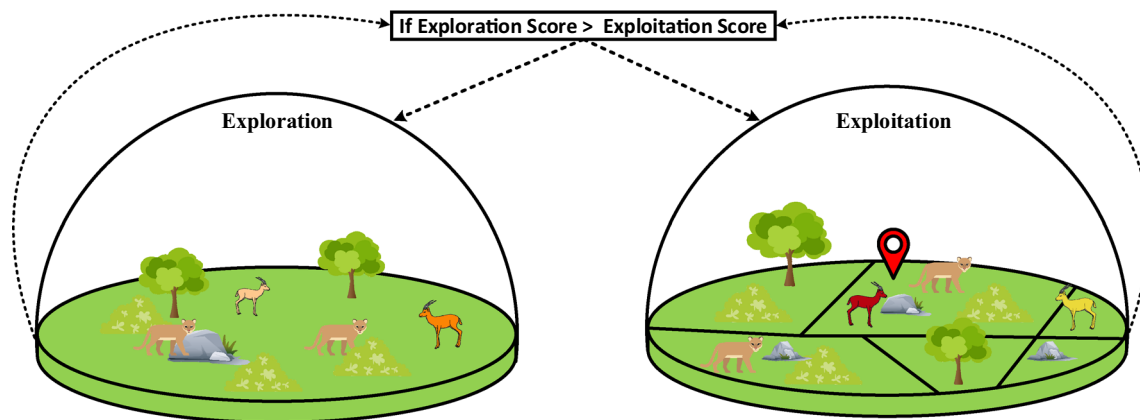


Fig. 3 PO optimization procedure

[60, 61]. The area of the territory of large males is between 150 to 1000 square kilometers (58 to 386 square miles), and the area of the territory of females is almost half that size. One study has shown that puma abundance ranges from 0.5 to 7 animals per 100 km² (39 sq mi) in some locations [62].

3.2 Mathematical model

In this subdivision, the PO based on natural behaviors for hunting is modeled and presented, which utilizes the main conceptions of puma in nature to create a mathematical model for the PO algorithm. Majority of the meta-heuristic optimizers are inspired by natural phenomena and perform the optimization operation by generating a random solutions and then making changes to the exploration factors using the mechanisms available in each optimization algorithm. The PO optimizer algorithm uses a new mechanism to change the phases of exploration and exploitation, which is innovative and purposeful in this phase change mechanism and is presented for the first time. On the other hand, in both the exploitation and exploration phases, two different mechanisms have been used to perform optimization operations. In the PO algorithm, the best solution is considered as a male puma, and the entire optimization space is also considered a puma's territory. Also, other solutions (X_i) have been considered as the female puma. In this algorithm, all the solutions enter one of the exploitation or exploration phases in each iteration using the phase change mechanism, and the selection of these phases is also purposeful and intelligent. In each phase of exploration, different approaches have been used for optimization operations, and in each phase, two different mechanisms have been applied, inspired by the way pumas live in nature. Subsequently, the mathematical formulations and expressions are presented to elucidate the optimization operations performed by PO. Additionally, Fig. 3

provides an overview of the optimization process, illustrating how it operates based on the mechanisms employed in PO.

3.2.1 Puma intelligence (phase change mechanism)

Pumas are very intelligent animals and have a very good memory. For hunting, they often go to places where hunting is more likely, which is based on their previous experiences. These targeted hunting trips can be to places where he has previously hunted and hidden his prey. Or travel to a newer place he has not hunted in previous stages. In this paper, we have considered the exploitation phase for puma trips to previous promising places and the exploration phase for traveling to new areas. To change the phases, we are inspired by the intelligence and memory of pumas and a new and intelligent mechanism that has been presented for the first time and it can be considered as an advanced hyper-heuristic algorithm, which is fully explained below.

The phase change mechanism in the proposed algorithm is a type of heuristic selection algorithm that uses two components of diversity and intensification to perform reward and penalty operations for scoring. The phase change section is inspired by the intelligence of cougars. It has two approaches, the first approach is that cougars do not have enough experience and energy, and they try to explore unfamiliar territory to find prey simultaneously. They ambush in promising areas, which is discussed in the section on the first generation who do not have enough experience.

3.2.1.1 Unexperienced phase Puma does not have much experience in its early life and often performs exploration operations at the same time due to its unfamiliarity with its living space and unawareness of where the hunts are located in its territory. And on the other hand, it looks for

hunting in propitious areas. In the Puma algorithm, in the first three iterations, both the exploration operation and the exploitation operation are performed simultaneously until the initialization is done in the phase change phase. In this section, because the pair of exploitation and exploration phases are selected in each iteration, only two functions (f_1 and f_2) have been used, which are calculated with Eqs. (1–4).

$$f1_{Explor} = PF_1 \cdot \left(\frac{Seq_{CostExplore}^1}{Seq_{Time}} \right) \quad (1)$$

$$f1_{Exploit} = PF_1 \cdot \left(\frac{Seq_{CostExploit}^1}{Seq_{Time}} \right) \quad (2)$$

$$f2_{Explor} = PF_2 \cdot \left(\frac{Seq_{CostExplore}^1 + Seq_{CostExplore}^2 + Seq_{CostExplore}^3}{Seq_{Time}^1 + Seq_{Time}^2 + Seq_{Time}^3} \right) \quad (3)$$

$$f2_{Exploit} = PF_2 \cdot \left(\frac{Seq_{CostExploit}^1 + Seq_{CostExploit}^2 + Seq_{CostExploit}^3}{Seq_{Time}^1 + Seq_{Time}^2 + Seq_{Time}^3} \right) \quad (4)$$

Variable values of Seq_{Cost} , which are related to each of the phases of exploitation and exploration, are calculated by Eqs. (1–4). And Seq_{Time} is also a variable with a constant value, which we have considered as 1. PF_1 and PF_2 are both parameters with fixed values that must be set previously the optimization procedure, which are used to prioritize each of the f_1 and f_2 functions.

$$Seq_{CostExplore}^1 = |Cost_{Best}^{Initial} - Cost_{Explore}^1| \quad (5)$$

$$Seq_{CostExplore}^2 = |Cost_{Explore}^2 - Cost_{Explore}^1| \quad (6)$$

$$Seq_{CostExplore}^3 = |Cost_{Explore}^3 - Cost_{Explore}^2| \quad (7)$$

$$Seq_{CostExploit}^1 = |Cost_{Best}^{Initial} - Cost_{Exploit}^1| \quad (8)$$

$$Seq_{CostExploit}^2 = |Cost_{Exploit}^2 - Cost_{Exploit}^1| \quad (9)$$

$$Seq_{CostExploit}^3 = |Cost_{Exploit}^3 - Cost_{Exploit}^2| \quad (10)$$

In Eqs. (5 and 8), $Cost_{Best}^{Initial}$ initial is the cost of the first-rate solution produced in the initialization phase and six variables $Cost_{Explore}^1$, $Cost_{Explore}^2$, $Cost_{Explore}^3$, $Cost_{Exploit}^1$, $Cost_{Exploit}^2$, $Cost_{Exploit}^3$ each cost of the best solution obtained from each of the phases exploitation and exploration are in repetitions 1, 2, and 3. After calculating the functions f_1 and f_2 at the end of the 3rd iteration, from

now on only one of the exploration and exploitation phases is selected. Because other Pumas have a pleasing experience, to choose one of the two phases, the two points of the exploitation and exploration phases are calculated using Eqs. (11 and 12).

$$Score_{Explore} = (PF_1 \cdot f1_{Explor}) + (PF_2 \cdot f2_{Explor}) \quad (11)$$

$$Score_{Exploit} = (PF_1 \cdot f1_{Exploit}) + (PF_2 \cdot f2_{Exploit}) \quad (12)$$

After calculating $Score_{Explore}$ and $Score_{Exploit}$, which were done using Eqs. (11 and 12) to enter one of the phases of exploration and exploitation, if $Score_{Exploit} \geq Score_{Explore}$ enters the exploitation stage and otherwise enters the exploration step, but the critical point is that at the end of the 3rd iteration, each of the steps will find solutions independently produced, which is more than the total population. To tackle this issue, the total cost of the solutions produced in both phases is calculated at the final of the 3rd iteration, and only the best solutions in the entire set of solutions produced are equal to the whole population that replaces the current solutions.

3.2.1.2 Experienced phase After passing three generations, Pumas has an acceptable experience to decide to change the phase, and in the continuation of the iterations, they choose only one phase for the optimization operation. In this phase, three different functions f_1 , f_2 , and f_3 are used for scoring. The first function emphasizes the escalation component and leads to the priority of one of the two phases of exploration and exploitation that have been selected and performed better than each other. The first function has more emphasis on the exploration phase. The first function is calculated using Eq. (1).

$$f_{1t}^{exploit} = PF_1 \cdot \left| \frac{Cost_{old}^{exploit} - Cost_{new}^{exploit}}{T_t^{exploit}} \right| \quad (13)$$

$$f_{1t}^{explore} = PF_1 \cdot \left| \frac{Cost_{old}^{explore} - Cost_{new}^{explore}}{T_t^{explore}} \right| \quad (14)$$

In Eqs. (13 and 14), $f_{1t}^{exploit}$ and $f_{1t}^{explore}$ represent the amount of the first function based on the exploration phase or the exploitation phase, where t depict the present iteration number. $Cost_{old}^{exploit}$ and $Cost_{new}^{exploit}$ are the costs of the best solution before improvement in the current choice. On the other hand, $Cost_{new}^{exploit}$ and $Cost_{new}^{explore}$ are the costs of the best solution obtained after improving the present selection. $T_t^{explore}$ and $T_t^{exploit}$ are the numbers of unselected iterations from the previous selection to the current selection. PF_1 is an user adjustable parameter that must be set with a value between 0 and 1 before the optimization operation. This parameter determines how important the

first function is, and it is prioritized as the value of this function increases and decreases as its priority decreases.

The second function also emphasizes the resonance component and leads to the phase that performs better than the other prioritized phase. Good performances are checked and measured sequentially. So this function can help the exploitation phase in its selection. The second function is calculated using Eqs. (15 and 16).

$$f_{2t}^{exploit} = PF_2 \cdot \frac{(Cost_{Old,1}^{exploit} - Cost_{New,1}^{exploit}) + (Cost_{Old,2}^{exploit} - Cost_{New,2}^{exploit}) + (Cost_{Old,3}^{exploit} - Cost_{New,3}^{exploit})}{T_{t,1}^{exploit} + T_{t,2}^{exploit} + T_{t,3}^{exploit}} \quad (15)$$

$$f_{2t}^{explore} = PF_2 \cdot \frac{(Cost_{Old,1}^{explore} - Cost_{New,1}^{explore}) + (Cost_{Old,2}^{explore} - Cost_{New,2}^{explore}) + (Cost_{Old,3}^{explore} - Cost_{New,3}^{explore})}{T_{t,1}^{explore} + T_{t,2}^{explore} + T_{t,3}^{explore}} \quad (16)$$

In Eqs. (15 and 16), $f_{2t}^{exploit}$ and $f_{2t}^{explore}$ represent the second function related to the exploration stage or the exploitation step, where t represents the present iteration number. $Cost_{Old,1}^{explore}$ and $Cost_{Old,1}^{exploit}$ are the cost of the best solution before improvement in the current selection for exploration and exploitation phases, and $Cost_{Old,2}^{explore}$ and $Cost_{Old,2}^{exploit}$ are the cost of the best before improvement in a previous choice for exploitation and exploration phases, and $Cost_{Old,3}^{exploit}$ and $Cost_{New,3}^{explore}$ are costs of the best Puma before improvement in the previous two choices for the exploration and exploitation phases. $Cost_{New,1}^{exploit}$ and $Cost_{New,1}^{explore}$ are costs of the best solution obtained after improvement in the current selection for both exploration and exploitation phases, and $Cost_{New,2}^{exploit}$ and $Cost_{New,2}^{explore}$ are costs of the best search agent obtained after improvement in a previous choice for exploitation and exploration phases, and $Cost_{New,3}^{exploit}$ and $Cost_{New,3}^{explore}$ are costs of the best solution obtained after improvement in the previous two choices in the exploration and exploitation phases. $T_{t,1}^{explore}$ and $T_{t,1}^{exploit}$ are numbers of unselected iterations from the previous selection to the current selection in each of the exploration and exploitation phases, and $T_{t,2}^{exploit}$ and $T_{t,2}^{explore}$ are numbers of unselected iteration from two previous selections to one previous choice for exploration and exploitation phases, and $T_{t,3}^{exploit}$ and $T_{t,3}^{explore}$ are numbers of unselected iteration from three previous choices to two previous choices for exploitation and exploration phases. PF_2 is a parameter that must be set with a value between 0 and 1 before the optimization operation. This parameter determines to what extent the second function is adequate.

It is prioritized as the value of this function increases and decreases as its priority decreases.

The third function in the selection mechanism emphasizes the diversity component and leads to the fact that the phase that is not selected in many repetitions also has a chance to be selected because the selection of one phase leads to collapsing into the local optimal trap. The function is shown in Eqs. (17 and 18).

$$f_{3t}^{exploit} = \begin{cases} \text{if selected, } f_{3t}^{exploit} = 0 \\ \text{otherwise, } f_{3t}^{exploit} + PF_3 \end{cases} \quad (17)$$

$$f_{3t}^{explore} = \begin{cases} \text{if selected, } f_{3t}^{explore} = 0 \\ \text{otherwise, } f_{3t}^{explore} + PF_3 \end{cases} \quad (18)$$

In Eqs. (17 and 18), $f_{3t}^{exploit}$ and $f_{3t}^{explore}$ represent the third function related to the exploitation stage or the exploration stage, where t represents the current iteration number. According to Eq. (3), the value of the third function of each of the phases of exploitation and exploration, if one of the stages is not selected, will be increased by the parameter PF_3 in each iteration; otherwise, it will be set to zero. PF_3 parameter is a user adjustable parameter and must be set with a value between 0 and 1 before the optimization operation. The closer the value of the PF_3 parameter is to 1, the phase that has a low score has a higher chance of being selected, and the chance of selection decreases with the decrease of the value. Using Eq. (19 and 20), the cost of the phase change function is calculated.

$$F_t^{exploit} = (\alpha_t^{exploit} \cdot (f_{1t}^{exploit})) + (\alpha_t^{exploit} \cdot (f_{2t}^{exploit})) + (\delta_t^{exploit} \cdot (lc \cdot f_{3t}^{exploit})) \quad (19)$$

$$F_t^{explore} = (\alpha_t^{explore} \cdot (f_{1t}^{explore})) + (\alpha_t^{explore} \cdot (f_{2t}^{explore})) + (\delta_t^{explore} \cdot (lc \cdot f_{3t}^{explore})) \quad (20)$$

$$lc = \{ \{ |Cost_{old} - Cost_{New}| \}^{exploitation}, \{ |Cost_{old} - Cost_{New}| \}^{exploration} \}, 0 \notin lc \quad (21)$$

$$\alpha_t^{explore, exploit} = \begin{cases} \text{if } F_t^{exploit} > F_t^{explore}, \alpha_t^{exploit} = 0.99, \alpha_t^{explore} = [\alpha_t^{explore} - 0.01, 0.01] \\ \text{otherwise, } \alpha_t^{explore} = 0.99, \alpha_t^{exploit} = [\alpha_t^{exploit} - 0.01, 0.01] \end{cases} \quad (22)$$

$$\delta_t^{exploit} = 1 - \alpha_t^{exploit} \quad (23)$$

$$\delta_t^{explore} = 1 - \alpha_t^{explore} \quad (24)$$

Using Eqs. (19 and 20), the final cost of each exploitation and exploration phase is calculated. Parameters α and δ for both the exploration and exploitation phases are

variables during the search operation according to the results obtained from each phase. Diversity is prioritized; otherwise, the priority diversity component is reduced, and if the value of parameter α is close to 1, the resonance component is prioritized. According to Eq. (22), if the cost of the exploration phase function is larger than the exploitation function, the value of the parameter α of the exploitation phase will be penalized linearly with a value of 0.01, and on the other hand, the value of the parameter α of the exploitation phase will have a maximum value close to one. But if the exploitation phase function costs more than the exploration function, the above procedure will be the opposite. This approach was utilized in [61]. lc is a set of calculation cost differences from the improvements obtained from exploitation and exploration phases, which includes a set of values other than zero numbers ($0 \notin lc$).

3.2.2 Exploration

Pumas often roam long distances in their territory to find food and hunt. This search can be a trip to the points of the territory where he had a chance to find hunting in previous experiences, or he can conduct a search operation to places in his territory that are new and have not obtained hunting and food in that area. In the exploration department, we have been inspired by these behaviors of pumas to search for food. At this stage, pumas do a random search in their territory to find food or randomly go near other pumas and use their prey. Therefore, the puma randomly jumps into the search space or searches for food in the space between the puma. At first, the entire population is sorted in ascending order, then Puma improves its solutions in the exploration phase using Eq. (24).

$$\text{If } rand_1 > 0.5, Z_{i,G} = R_{Dim} * (Ub - Lb) + Lb \quad (25)$$

$$\begin{aligned} \text{Otherwise, } Z_{i,G} &= X_{a,G} + G \cdot (X_{a,G} - X_{b,G}) + G \cdot (((X_{a,G} - X_{b,G}) - (X_{c,G} - X_{d,G})) + ((X_{c,G} - X_{d,G}) - (X_{e,G} - X_{f,G}))) \\ G &= 2 \cdot rand_2 - 1 \end{aligned} \quad (26)$$

In Eq. (25), Ub and Lb are the lower and upper bounds of the problem and R_{Dim} are randomly generated numbers in the range of 0 and 1 and are in the dimensions of the problem. $rand_1$ is also a randomly generated number between 0 and 1. $X_{a,G}$, $X_{b,G}$, $X_{c,G}$, $X_{d,G}$, $X_{e,G}$, and $X_{f,G}$ are solutions in the whole population that are Faces are randomly selected. G is also calculated using Eq. (26) where $rand_2$ is a randomly generated number in uniform distribution between 0 and 1. According to Eq. (25), one of the two equations is selected to produce a different solution according to the existing condition, and the new solution generated is then applied to improve the current solution.

$$X_{new} = \begin{cases} Z_{i,G}, & \text{if } j_{rand} \text{ or } rand_3 \leq U \\ X_{a,G}, & \text{otherwise} \end{cases} \quad (27)$$

$$NC = 1 - U \quad (28)$$

$$p = \frac{NC}{Npop} \quad (29)$$

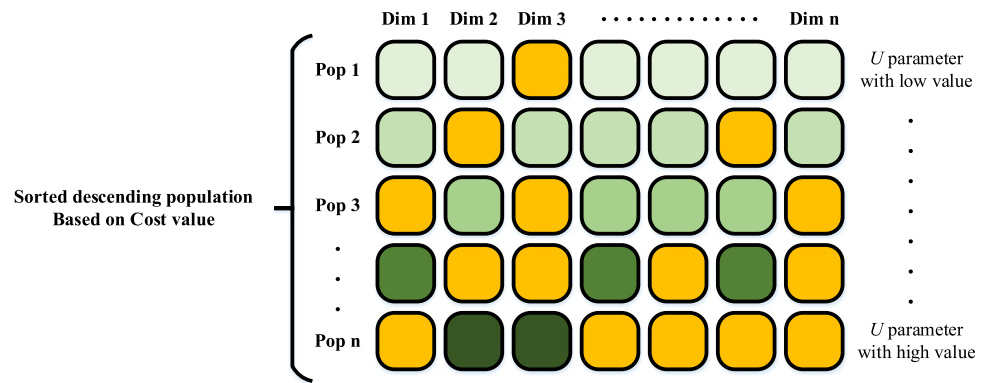
$$\text{if } CostX_{new} < CostX_i, U = U + p \quad (30)$$

In Eq. 27, $Z_{i,G}$ is a solution generated using Eq. (25). j_{rand} is a randomly generated integer in the scope of dimensions of the problem. $rand_3$ is also a randomly produced number in uniform distribution generated between 0 and 1. NC is also calculated using Eq. (28). U is a parameter set previous the optimization procedure; its value is a number between 0 and 1. In each iteration, according to the condition in Eq. (30), the number of dimensions that are replaced by new solutions increases, and this increase is done using Eqs. (28–30). In Eq. (29), $Npop$ is the total number of Pumas. Improving the solution is according to the condition in Eq. (30) in which only the solutions' dimensions are updated if this condition is met. This action leads to avoid the local optimum, and the product solutions have a good diversity. On the other hand, the mechanism explained in the exploration stage, considering that at the beginning of each iteration in the exploration phase, the search agents are sorted in ascending order based on their costs, high-quality solutions are placed first, then according to the Eqs. (28–30) at the beginning, quality solutions do not undergo many changes because the value of the U parameter is small. Still, later, with the increase of this parameter, the solutions that have a higher value in terms of cost undergo many changes, which this approach causes to explore worse solutions of the problem space with the aim of finding significant optimal points in the problem space. The critical point is that if the production Pumas is not better than the current Pumas, Eq. (30) will not be done because if it is improved, there is no need to increase redundant discoveries. But the solutions that are of good quality have undergone few changes and only try to avoid getting stuck in the trap of local optimality. Finally, the newly generated solutions are replaced with the current solution using Eq. (31).

$$X_{a,G} = X_{new}, \text{ if } X_{i,new} < X_{a,G} \quad (31)$$

The above procedure is shown in Fig. 4.

According to Eq. (31), the new production solution replaces the current solution if it has a better cost than the current solution. In Algorithm 1, the pseudo-code of the discovery phase is shown.

Fig. 4 PO exploration procedure**Algorithm 1** Pseudo-code of the exploration phase of PO

```

Sort ascending population
Calculate p
for (each Puma ( $X_i$ )) do
    Select four solutions randomly
    Calculate the new vector by Eq. (24).
    Check the boundary of the new vector position
    Update the current solution by Eq. (26).
    Calculate the cost of the updated solution
    if  $NewX_i \text{ Cost} < X_i \text{ Cost}$ 
         $X_i = NewX_i$ 
    else
         $U = U + p$ ;
    end
end
end

```

3.2.3 Exploitation

In the exploitation Stage, the PO algorithm utilize two different operators to improve the solutions, and these two mechanisms are based on the two behaviors of pumas, which are hunting using ambush and sprinting. In nature, pumas try to ambush their prey in bushes or on trees and rocks. In some cases, it runs after its prey, which is simulated using Eq. (32) of this behavior.

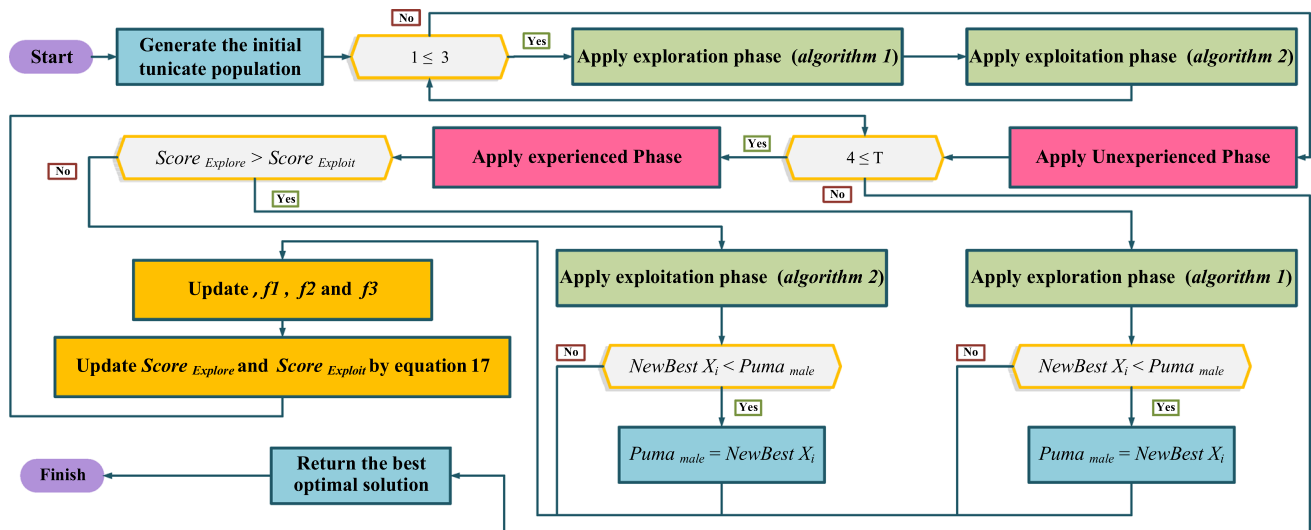
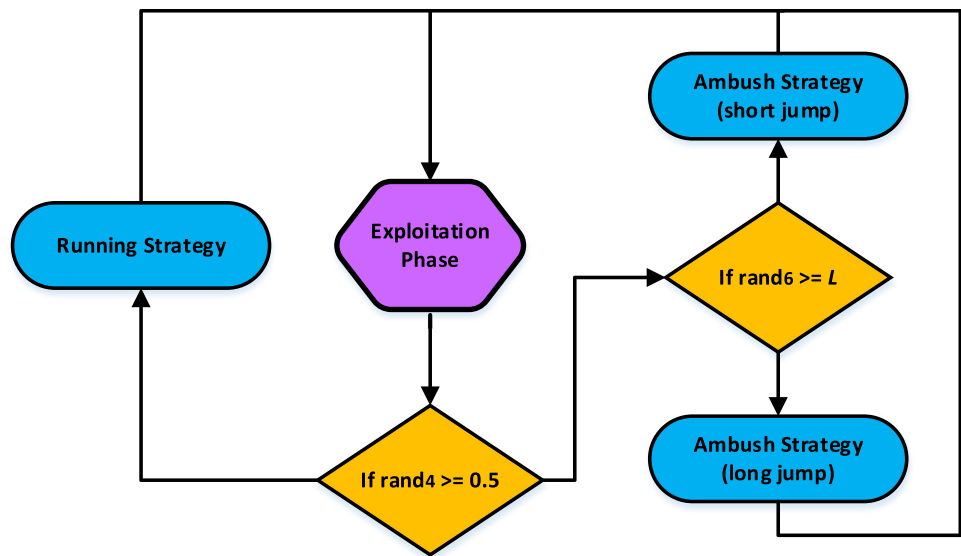
$$X_{new} = \begin{cases} \text{if } rand_4 \geq 0.5, X_{new} = \frac{\left(\frac{mean(Sol_{total})}{Npop}\right) \cdot X_1^r - (-1)^\beta \times X_i}{1 + (\alpha \cdot rand_5)} \\ \text{otherwise, if } rand_6 \geq L, X_{new} = Puma_{male} + (2 \cdot rand_7) \cdot \exp(rand_{n_1}) \cdot X_2^r - X_i \\ \text{otherwise, } X_{new} = (2 \times rand_8) \times \frac{(F_1 \cdot R \cdot X(i) + F_2 \cdot (1 - R) \cdot Puma_{male})}{(2 \cdot rand_9 - 1 + rand_{n_2})} - Puma_{male} \end{cases} \quad (32)$$

Equation (32) shows the two strategies used in the PO. Considering that in Pumas, for hunting in the first mode, in the exploitation phase, for running, and ambush strategies, case 1 in Eq. (32) is used, and a division operation does this operation to simulate pumas' fast run toward preys. According to Eq. (32), if $rand_5$, a randomly produced number in uniform distribution between 0 and 1, is greater than 0.5, the fast-running strategy is performed, and otherwise, the ambush strategy is selected, which consists

of two different operations that the first one used for simulation Pumas short jumps towards others Pumas' hunts, the and second one used to long jumps toward the best puma's hunt. According to Eq. (32), the *mean* represents the mean function and Sol_{total} represents the sum of all solutions and $Npop$ is the entire number of populations to execute the optimization procedure. X_1^r is a randomly selected solution in the whole population, and β is a zero or one that s randomly produced. Moreover, X_i is the current solution in the current iteration, and α and L are static parameters that must be tuned before the optimization procedure. In addition, $Puma_{male}$ is the best solution for the entire population, and $rand_4$, $rand_5$, $rand_6$, $rand_7$, $rand_8$, and $rand_9$ are random numbers that are produced between zero and 1. Also, exp represents the exponential function. $rand_{n_1}$ and $rand_{n_2}$ are randomly generated numbers in the normal distribution and the dimensions of the problem, and X_2^r is a randomly chosen solution that is selected based on Eq. (33).

$$round(1 + (Npop - 1) \cdot rand_{10}) \quad (33)$$

In Eq. (33), rounds each element of X to the nearest integer and $rand_{10}$ is a randomly produced number between 0 and 1 and $Npop$ is the total number of Pumas.

Fig. 5 PO exploitation procedure**Fig. 6** PO optimization procedure

Finally, R , $F1$, and $F2$ are calculated by Eqs. (34–36), respectively.

$$R = 2 \cdot rand_{11} - 1 \quad (34)$$

$$F1 = rand_{13} \cdot \exp\left(2 - Iter \cdot \left(\frac{2}{MaxIter}\right)\right) \quad (35)$$

In Eq. (35), $rand_{12}$ is a random number in the dimensions of the problem and the normal distribution. $Iter$ depicts the present iteration number and $MaxIter$ signifies the total number of iterations to perform the optimization operation. $\exp$ indicates the exponential function.

$$F2 = w \times (v)^2 \cdot \cos((2 \times rand_{12}) \cdot w) \quad (36)$$

$$w = rand_{14} \quad (37)$$

$$v = rand_{15} \quad (38)$$

In Eqs. (36–38), $rand_{14}$ and $rand_{15}$ are both randomly created numbers in the normal distribution and the dimensions of the problem. $\cos$ represents the cosine function and $rand_{12}$ is a randomly generated number between 0 and 1. The above procedure in the exploitation phase is shown in Figs. 5 and 6.

Finally, at the end of this phase, newly produced solutions are replaced if they have a lower cost than the current solution. The pseudocode of the proposed algorithm is shown in Algorithm 2.

Algorithm 2 Pseudo-code of exploitation phase of PO

```

for (each Puma ( $X_i$ )) do
    Calculate  $R$ ,  $F_1$  and  $F_2$  by Eqs.(34), (35) and (36) respectively
    Produce  $NewX_i$  by Eq. (32)
    Calculate the cost of  $NewX_i$ 
    if  $NewX_i$  Cost <  $X_i$  Cost
         $X_i = NewX_i$ 
    end
end
end

```

The Puma optimizer has relatively more implementation complexity than other optimizer because it has many phases and the creation of intelligent systems requires more operations. In each iteration the cost function is applied only once for each search agent, it has an acceptable computational complexity which is explained in the relevant section. Algorithm 3 shows the pseudo-code of the puma optimizer and Flowchart 2 is related to the overall process of the puma optimizer.

PO has three essential processes: initialization, fitness evaluation, and producing new solutions. As regards that there are N_{pop} , the computational complexity in the initialization phase is an $O(N)$. In addition, the computational complexity in the exploitation phase for producing a new solution, tracking for the best location, and updating the location vector of all Pumas equals $O(N \times (T + TD))$. Moreover, the computational complexity in the exploration phase for generating and alternation mechanism is equal to

Algorithm 3 Pseudo-code of PO

```

% PO setting
Inputs: The population size N and the maximum number of iterations and parameter settings
Outputs: Puma's location and fitness potential
% initialization
Create a random population using  $X_i$  ( $i = 1, 2, \dots, N$ ). Calculate Puma's fitness levels.
for iter = 1 : 3
    Apply exploration phase (Algorithm 1)
    Apply exploitation phase (Algorithm 2)
End
Apply Unexperienced Phase
for iter = 4: Max iteration
    Apply experienced Phase
    if  $Score_{Explore} > Score_{Exploit}$ 
        Apply exploration phase (algorithm 1)
        if Exploration  $NewBestX_i$  Cost <  $Puma\_male$  Cost
             $Puma\_male = NewBestX_i$ 
        end
    else
        Apply exploitation phase (algorithm 2)
        if Exploitation  $NewBestX_i$  Cost <  $Puma\_male$  Cost
             $Puma\_male = NewBestX_i$ 
        end
    end
    Update  $T$ ,  $f1$ ,  $f2$  and  $f3$ 
    Update  $Score_{Explore}$  and  $Score_{Exploit}$  by Eq. (17)
end

```

3.3 Computational complexity

Even though PO is challenging to implement, PO has regularly low computational complexity and depends on which phase is run as each stage has its computational complexity. Overall, the computational complexity of the

$O((2T + 1) \times N \times D)$, while the exploration phase uses sort, which computational complexity is $O(n^2)$ the worst case, therefore, computational complexity is equal to $O((2T + 1)N^2 \times N \times D)$. Where T is the maximum number of iterations, D is the dimension of the problems, and N is the total number of the population.

4 Results and discussion

4.1 Benchmark set and compared algorithms

In this article, we have used different standard composition model (CM), multimodal (MM), and unimodal (UM) [62, 63] benchmarks to accurately evaluate the Puma algorithm, each of the benchmarks has a different difficulty. Also, these benchmarks reveal the capabilities of the Puma algorithm in the exploitation and exploration phases or the capacity of creating balance. The optimization algorithms' abilities to exploit are demonstrated by the unimodal benchmarks (F1–F7) that have a single global optimal point. On the other hand, the multimodal (MM) benchmarks (F8–F23) help clarify the ability of optimization algorithms to produce diverse solutions (exploration phase). However, these benchmarks fail to show all the features of an optimization algorithm, therefore, in this article, CEC2019 benchmarks (C1–C10) are used, which have high complexity. Obtaining acceptable results in CEC benchmarks requires high performance in both the exploitation and exploration phases, as well as creating a balance in these two phases. Mathematical equations related to benchmarks and their variables are shown in Appendix 1 in Tables.

4.2 Parameter settings

All implementations of optimization algorithms used in this research are benefited from MATLAB (R2023a), and all runs were executed on a MacBook Pro with 16 GB of RAM, the macOS Ventura operating system, and a 2.3 GHz (8-Core) Intel Core i9 processor. Due to the MacBook Pro's CPU's resource constraints, larger values for the control parameters could cause the CPU to freeze and the RAM to overflow. In addition, to make a comparison fairly, the population parameters and the total number of iterations for all optimizer algorithms are set to 30 and 500, respectively. Furthermore, For deeper assessment, the results collected from the Puma optimizer have been compared with another robust optimizer algorithm, including GWO [64], WOA [26], SCA [65], DMOA [66], TSA [67], MFO [68], FHO [69], and SAO [70].

Due to the existence of several parameters in the PO, the face-centered central composite design (FCCD) [71] method was used to set the parameters. This method sets each parameter in 3 modes: low, high, and medium. The values are illustrated in Table 2. According to Table 2, 3⁶ modes of parameter combination have been created at each level. Then, each combination of parameters was tested using functions F1 to F13 with 30 dimensions. Finally, the

Table 2 FCCD levels of parameters

Parameter	Low	Medium	High
PF_1	0.2	0.5	0.8
PF_2	0.2	0.5	0.8
PF_3	0.1	0.3	0.5
U	0.2	0.4	0.6
L	0.5	0.7	0.9
α	1	2	3

Table 3 Parameter settings of optimization algorithms for comparison and evaluation of the PO

Algorithm	Parameter	Value
PO	PF_1	0.5
	PF_2	0.5
	PF_3	0.3
	U	0.2
	L	0.9 for the F function 0.7 for the C function
	α	2 for the Function 1 for C function
SCA[65]	b	2
SAO[70]	olf	0.9
	k	0.6
	T	0.95
	M	0.9
	Step	0.02
FHO[69]	SP	Mean(Pop)
TSA[67]	Parameter P_{min}	1
	Parameter P_{max}	4
DMOA[66]	bb	3
	K	7
	α	2
GWO[64]	Convergence constant a	[2 0]
MFO[68]	Convergence constant a	[-2 -1]
	Spiral factor b	1
WOA[26]	Convergence constant a	[2,0]
	Spiral factor b	1

best combination of parameter values that led to the best results is demonstrated in Table 3.

Finally, All the parameters related to the optimization algorithms used in this scientific research were selected according to the parameters presented in the base papers, which are shown in Table 2.

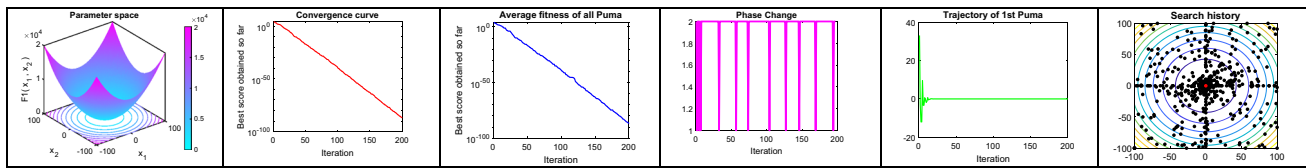


Fig. 7 Qualitative test results for the F1

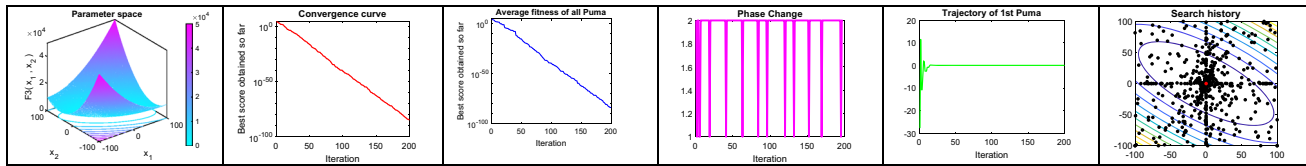


Fig. 8 Qualitative test results for the F3

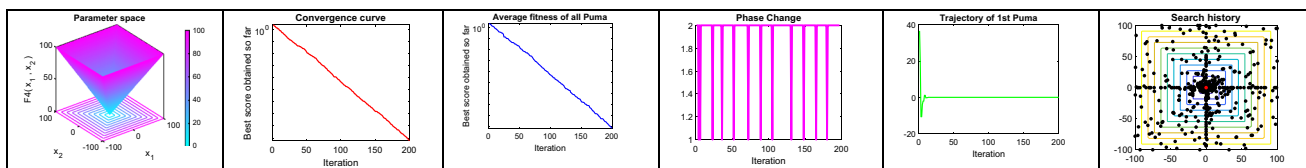


Fig. 9 Qualitative test results for the F4

4.3 Visual observation of PO's search behavior

In this subsection, the qualitative evaluations of the Puma algorithm are discussed. In the assessment in this section, different graphs have been used, such as convergence graphs, average cost graphs of all Pumas, phase change graphs, Trajectory of 1st Puma, and Search History graphs. Each of these graphs illuminates a series of features of the PO algorithm. Also, for the evaluation, the benchmarks F1, F3, F4, F7, F9, F10, and F13 have been used.

According to the convergence graphs shown in Figs. 7, 8, 9, 10, 11, 12, 13, it is easy to understand a continuous decreasing pattern, which is a uniform reducing pattern in unimodal functions, which can be concluded that the PO in the intensification component has an outstanding performance. It has and can go towards the optimal solution in all optimization stages, and it also has fast and good convergence. But according to the multimodal functions, the descending pattern can be observed again and in all optimization stages, the dynamic algorithm tries to escape from the local optimal points.

Regarding the graphs of the average cost of all PUMAs in Figs. 7, 8, 9, 10, 11, 12, 13, a decreasing pattern can be seen, which indicates the reduction of the cost of all solutions in all optimization stages, so it can be concluded that the PO is very capable. It also has the advantage of improving all existing solutions in the entire population. By looking more closely at these charts, it can be concluded

that if one solution is improved, other solutions will be enhanced as well.

The phase change graphs shown in Figs. 7, 8, 9, 10, 11, 12, 13 indicate which exploitation or exploration phase the Puma algorithm has applied in each iteration. Therefore, according to these graphs, the Puma algorithm in Unimodal functions focuses a lot on the exploitation phase, but in Multimodal functions, the focus is more on the exploration stage. The behavior of the Puma algorithm in changing the phase in different functions indicates that the algorithm can recognize which phase has performed better and can improve the solutions better and prioritizes the promising phase. But the noteworthy point is that even if the focus is on one phase, another phase has the chance to be selected to perform the optimization operation, which is balanced by the above procedure and the algorithm quickly finds the capacity of escape from the local optimal trap.

The Trajectory of the 1st Puma graphs illustrate in Figs. 7, 8, 9, 10, 11, 12, 13 corresponds to the movement pattern of the first Puma. By looking carefully at these diagrams, in the initial iterations of the algorithm, it produces varied solutions and tries to reach and examine all the optimization space in the initial phases of optimization. Then, the algorithm focuses more on promising areas and tries to perform optimization operations at points close to the global optimum.

Finally, regarding the Search History charts demonstrate in Figs. 7, 8, 9, 10, 11, 12, 13 related to all benchmarks, it can be concluded that the PO has a magnificent ability to detect promising areas and tries to Find optimal solutions.

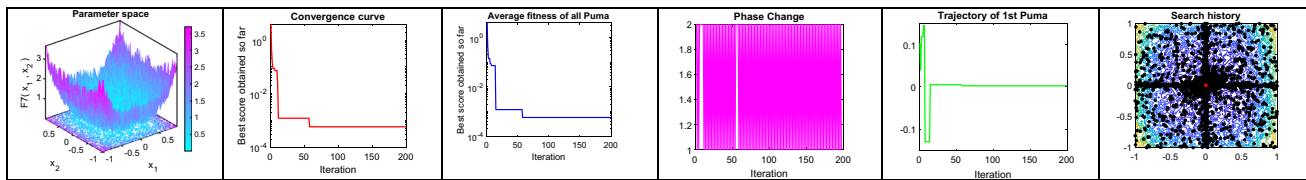


Fig. 10 Qualitative test results for the F7

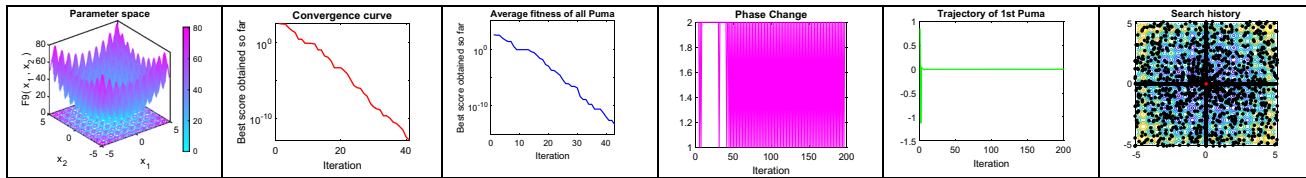


Fig. 11 Qualitative test results for the F9

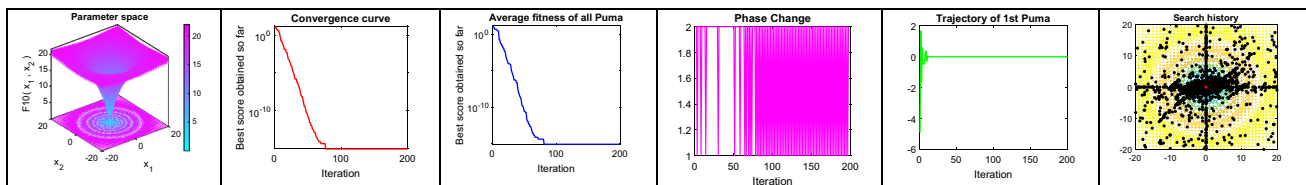


Fig. 12 Qualitative test results for the F10

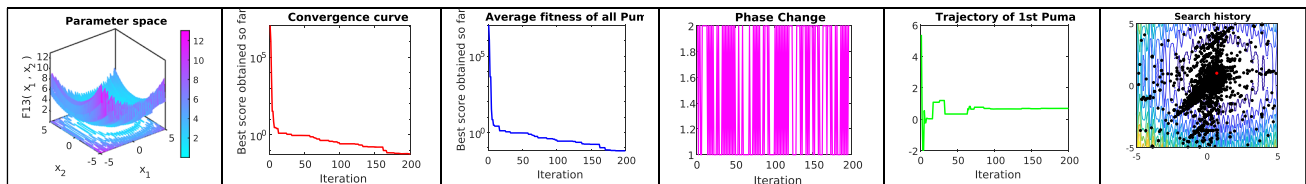


Fig. 13 Qualitative test results for the F13

But the PO explores almost the entire problem space, which indicates a large variety of production solutions. The noteworthy point in these graphs is that the Puma algorithm performs better if it detects the exploitation phase (Unimodal functions), the focus of optimization is near the global optimum, but if it detects that the exploration phase performs better (Multimodal functions) algorithm tries to explore the entire optimization space.

4.4 Benchmark test results

In this sub-section, PO and other compared algorithms have been evaluated. A total of 33 standard functions have been used for evaluation, each of which has been discussed in its respective sections. Firstly, the scalability analyses has been done using functions F1 to F13, which are scalable functions, and their dimensions can be changed. For this evaluation, functions F1 to F13 were tested with dimensions of 30, 100, 500, and 1000 respectively, in four separate implementations and the corresponding results are

depicted in four Table 4, 5, 6, 7. This experiment shows whether the proposed algorithm can maintain its search capacities of facing problems with large dimensions. Table 4 demonstrates the results of benchmarks F1 to F13 with dimensions of 30.

Focusing on the results illustrated in Table 4, the PO has a brilliant performance compared to functions F1 to F13 and has been able to achieve better results in 11 functions out of 13 benchmarks compared to other compared optimizers. In addition, the PUMA algorithm performed remarkably well in solving problems F1 to F7, which has a global optimal point, and only in function F5 of the SAO algorithm, it had better result than PO. But in comparison with benchmarks F8 to F13, PO has the second-best performance only in benchmark F8 and has shown much better efficiency in other benchmarks, and in benchmark 8 the best result is related to the FHO optimizer. Overall, PO has been able to perform very well against benchmarks F1 to F13 and is also significantly better in most of the obtained result functions. Next, Table 5 demonstrates the

results of the PO and other compared algorithms utilizing functions 1 to 13 with dimensions of 100.

In Table 5, the results achieved from the benchmarks F1 to F13 with dimensions of 100 are highlighted. Regarding the outcomes, it can be concluded that the Puma algorithm in functions F1 to F7, which are Unimodal benchmarks, 5 of the 7 cases have performed better, however in the other two functions, which are benchmarks F5 and F6, it has shown the second-best performance. However, in Multimodal functions 8 to 13, the Puma algorithm has again demonstrated exemplary performance and in 4 out of 6 functions, it has been able to obtain better solutions than other compared algorithms. In general, the Puma optimizer has been able to show the highest achievements against problems with dimensions of 100, while the SAO and FHO algorithms have also performed well.

Looking at the results shown in Table 6, by increasing the dimensions to 500, the Puma algorithm has once again obtained good results compared to the Unimodal functions. It has maintained its search capabilities in the intensification component. It is better than other algorithms and obtains high-quality solutions only in benchmark F5, which shows a relatively weaker performance than the compared algorithms. However, in contrast to the Multimodal functions, the PO has performed better in 3 benchmarks out of the total 6 functions, but in the other 3 functions, it has achieved almost the second best. In general, the FHO algorithm has also been able to have an exceptional ability to gain quality solutions when the dimensions increase.

It regarded the results shown in Table 7, which are related to the results gained from testing the PO and other optimization algorithms in benchmarks F1 to F13 with dimensions of 1000. By carefully examining the results, it can be quickly concluded that the PO, by increasing the dimensions to 1000, has once again obtained quality results against Unimodal problems and has depicted the second-best performance only in F5.

On the other hand, in comparison with the Multimodal benchmarks in large dimensions, the proposed model still shows an acceptable performance, which indicates that PO has the sufficient and necessary ability to explore large-scale problems and can create solutions with diversity. In general, by examining the tests performed in the scalability section, it can be concluded that PO has great power and capabilities in exploring the problem space by increasing the problem space and can maintain its search capabilities to an acceptable extent. Table 8 shows the results of the tests conducted from benchmarks F14 to F23, which are benchmarks with fixed dimensions.

Table 8 highlights the results obtained from the PO and other compared optimizer in solving problems F14 to F23. The results indicate the excellent power of the PO because it has the second-best performance in only 2 out of 10

functions and in other tested benchmarks it has been able to discover optimal or near-optimal solutions. Obtaining quality results in these benchmarks requires great performance in the exploration phase, and the proposed algorithm has been able to achieve many good results. Therefore, it can be concluded that the PO has an acceptable execution in the exploration stage. But the 23 tested standard functions fail to reveal all the features of an algorithm, so we have used the CEC2019 functions for further discussion and review since the CEC2019 benchmarks are tremendously complex and it can be difficult to solve. Table 9 shows the outcomes of the evaluations performed using the CEC2019 benchmarks.

Concerning the results obtained and depicted in Table 9, which are related to the functions of CEC2019, it indicates the excellent performance of the Puma algorithm compared to the other compared optimizers because the proposed model in 8 functions out of 10 functions tested in this section it has been able to obtain much better solutions than the compared optimizers. Also, the GWO has been able to perform best in benchmarks F3 and F4. But the critical detail is that obtaining good results in these functions requires performance in both the exploitation and exploration stages. Still, good performance in these two phases is not enough. The optimization algorithm should have the strength to create a balance in these two phases, which in the Puma algorithm is changed intelligently and automatically; therefore, PO works very well in creating balance. Convergence graphs obtained in CEC2019 functions are shown below.

Regarding the convergence graphs shown in Fig. 14, the proposed algorithm has a reduction pattern in most cases, and it can also be seen that the convergence, even in CEC2019 functions has an acceptable speed. This good performance in this series of functions indicates the production of quality and diverse solutions because the lack of diversity leads the optimizer algorithm to fall into the trap of local optimality, but by looking carefully at the graphs, it can be seen that the decreasing pattern to It has been created repeatedly, which indicates that the algorithm has tried to escape the trap of local optima successively and can discover better and more optimal solutions during the optimization operation. Overall, based on the results of various tests using standard benchmarks, it was concluded that the Puma algorithm is an intelligent and highly efficient algorithm that performs well in most situations. It has strengths in both the exploitation and exploration phases and is able to harmonize them effectively.

4.5 Statistics test

To discover major difference between the results of meta-heuristic algorithms, statistical tests can be used so that a thorough comparison can be made through these statistical tests. In this paper, we have used the Paired *T*-test to make

Table 4 Results of benchmark functions (F1–F13), with 30 dimensions

No	SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO	PO
F1	Best	2.2124E + 03	2.7107E + 00	1.3011E – 26	3.3088E – 01	6.9275E – 35	9.3216E – 96	3.0631E – 54	3.2310E – 04
	Worst	5.8978E + 01	1.4373E + 01	2.7632E – 23	1.0001E + 04	1.7114E – 32	6.4785E – 84	2.2521E – 40	3.6634E – 01
	Mean	6.2111E + 00	5.4284E + 00	2.7749E – 24	6.6993E + 02	2.7936E – 33	2.2937E – 85	7.5094E – 42	1.9290E – 02
	STD	1.4652E + 01	2.2672E + 00	5.1794E – 24	2.5363E + 03	4.5458E – 33	1.1807E – 84	4.1117E – 41	6.6051E – 02
F2	Best	1.0387E – 04	1.8108E + 00	1.7776E – 16	1.2860E – 01	9.4873E – 21	1.3316E – 60	1.0069E – 12	1.3024E – 02
	Worst	9.6036E – 02	8.8277E + 01	6.7489E – 14	8.0014E + 01	1.8285E – 19	3.3269E – 53	2.4059E – 10	9.9095E – 01
	Mean	1.5581E – 02	2.4146E + 01	9.6280E – 15	3.2889E + 01	5.1104E – 20	3.0174E – 54	5.2350E – 11	3.1135E – 01
	STD	2.1887E – 02	2.0120E + 01	1.4955E – 14	2.0887E + 01	3.5233E – 20	7.5937E – 54	6.5917E – 11	2.5875E – 01
F3	Best	3.2899E + 02	3.5012E + 04	3.3836E – 11	2.7763E + 03	9.2766E – 11	1.2149E + 04	2.9679E – 50	1.2486E – 02
	Worst	1.9030E + 04	8.7698E + 04	1.0153E – 04	4.2525E + 04	1.0235E – 07	6.0493E + 04	7.5095E – 39	3.6534E + 04
	Mean	6.3731E + 03	5.7334E + 04	6.5327E – 06	1.7466E + 04	1.6915E – 08	3.1646E + 04	2.6279E – 40	4.3455E + 03
	STD	4.4961E + 03	1.1875E + 04	2.1412E – 05	1.0341E + 04	2.8321E – 08	9.2494E + 03	1.3693E – 39	9.4676E + 03
F4	Best	8.4475E + 00	4.3825E + 01	3.2553E – 03	3.3241E + 01	1.7297E – 09	6.7785E – 03	8.0880E – 22	4.9784E – 03
	Worst	4.1075E + 01	6.4213E + 01	4.1271E – 01	8.1378E + 01	6.7664E – 08	8.2173E + 01	3.4784E – 18	1.5016E + 00
	Mean	2.6706E + 01	5.5922E + 01	6.2772E – 02	6.0500E + 01	1.9112E – 08	2.9740E + 01	4.2693E – 19	6.7655E – 02
	STD	9.2572E + 00	5.7066E + 00	8.0374E – 02	1.1995E + 01	1.7393E – 08	2.4940E + 01	8.4576E – 19	2.7117E – 01
F5	Best	4.0180E + 01	5.5954E + 04	2.6501E + 01	1.5700E + 02	2.5765E + 01	2.6816E + 01	7.0853E – 02	2.3555E – 03
	Worst	1.6898E + 05	3.9743E + 05	2.9607E + 01	9.0296E + 04	2.7932E + 01	2.7869E + 01	1.4070E + 00	3.1270E + 00
	Mean	1.1054E + 04	1.9388E + 05	2.8506E + 01	1.8778E + 04	2.6554E + 01	2.7328E + 01	5.1867E – 01	4.7353E – 01
	STD	3.2225E + 04	8.4958E + 04	6.5825E – 01	3.6330E + 04	6.2212E – 01	2.9264E – 01	3.6445E – 01	6.1131E – 01
F6	Best	4.3128E + 00	2.5960E + 00	2.5703E + 00	3.4755E – 01	4.2139E – 05	1.3024E – 02	8.5342E – 02	1.3979E – 04
	Worst	2.1696E + 01	1.2720E + 01	5.2877E + 00	1.0101E + 04	1.2621E + 00	2.8738E – 01	5.7554E + 00	4.3516E – 02
	Mean	7.2015E + 00	6.0084E + 00	3.6423E + 00	1.0045E + 03	4.3207E – 01	8.4289E – 02	6.7739E – 01	6.0237E – 03
	STD	3.8326E + 00	2.0870E + 00	6.4801E – 01	3.0615E + 03	3.3037E – 01	7.3733E – 02	1.4158E + 00	8.2107E – 03
F7	Best	5.0370E – 03	9.7885E – 02	1.8646E – 03	4.8461E – 02	3.3488E – 04	2.7369E – 05	2.5244E – 04	3.2647E – 04
	Worst	4.0621E – 01	4.1761E – 01	2.1269E – 02	1.3637E + 01	2.4842E – 03	8.6711E – 03	3.7198E – 03	1.8063E – 01
	Mean	6.4922E – 02	2.3983E – 01	6.0868E – 03	1.5589E + 00	1.3079E – 03	1.7844E – 03	1.5703E – 03	3.1016E – 02
	STD	8.1711E – 02	7.3349E – 02	4.1384E – 03	3.2894E + 00	6.3469E – 04	2.2937E – 03	9.0604E – 04	3.5882E – 02
F8	Best	– 4.4678E + 03	– 1.0736E + 62	– 7.0562E + 03	– 1.0908E + 04	– 7.7661E + 03	– 1.2569E + 04	– 1.2569E + 04	– 1.2569E + 04
	Worst	– 3.3080E + 03	– 4.0988E + 58	– 5.1052E + 03	– 7.2645E + 03	– 4.7621E + 03	– 7.7375E + 03	– 1.2456E + 04	– 2.0988E + 03
	Mean	– 3.9187E + 03	– 9.9211E + 60	– 6.0707E + 03	– 8.7881E + 03	– 6.1881E + 03	– 1.0627E + 04	– 1.2551E + 04	– 5.7829E + 03
	STD	2.8438E + 02	2.7049E + 61	4.2168E + 02	9.9340E + 02	6.7564E + 02	1.8363E + 03	3.1022E + 01	3.2159E + 03
F9	Best	1.3695E – 02	2.0670E + 02	1.0677E + 02	7.3650E + 01	0.0000E + 00	0.0000E + 00	0.0000E + 00	1.5006E – 02
	Worst	1.0670E + 02	2.6289E + 02	2.3860E + 02	2.0425E + 02	1.4889E + 01	0.0000E + 00	1.6262E – 04	1.1959E + 02
	Mean	2.2529E + 01	2.3357E + 02	1.8294E + 02	1.4881E + 02	1.7793E + 00	0.0000E + 00	5.4208E – 06	1.4233E + 01
	STD	2.7374E + 01	1.3567E + 01	3.6083E + 01	3.5535E + 01	3.8644E + 00	0.0000E + 00	2.9691E – 05	2.4154E + 01

Table 4 (continued)

No		SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO	PO
F10	Best	6.8630E - 02	2.4110E + 00	1.5321E - 13	3.4105E - 01	3.2419E - 14	4.4409E - 16	4.4409E - 16	5.5230E - 03	4.4409E - 16
	Worst	2.0365E + 01	3.6297E + 00	3.8105E + 00	1.9961E + 01	5.3735E - 14	7.5495E - 15	5.9015E - 12	3.6081E + 00	4.4409E - 16
	Mean	1.4309E + 01	3.0495E + 00	2.4712E + 00	1.2291E + 01	4.1656E - 14	3.4047E - 15	2.5008E - 13	2.5400E - 01	4.4409E - 16
	STD	9.1475E + 00	3.9409E - 01	1.4130E + 00	8.2847E + 00	4.6275E - 15	2.6526E - 15	1.0729E - 12	7.0466E - 01	0.0000E + 00
F11	Best	1.7781E - 01	1.0206E + 00	0.0000E + 00	4.4575E - 01	0.0000E + 00	0.0000E + 00	0.0000E + 00	2.6755E - 05	0.0000E + 00
	Worst	1.2163E + 00	1.1351E + 00	1.9842E - 02	9.0851E + 01	4.4024E - 02	8.6753E - 02	0.0000E + 00	2.4928E + 00	0.0000E + 00
	Mean	7.9172E - 01	1.0591E + 00	4.8497E - 03	1.5828E + 01	3.2803E - 03	2.8918E - 03	0.0000E + 00	4.1280E - 01	0.0000E + 00
	STD	2.4560E - 01	2.2099E - 02	6.3364E - 03	3.3987E + 01	9.5675E - 03	1.5839E - 02	0.0000E + 00	7.7099E - 01	0.0000E + 00
F12	Best	7.9456E - 01	3.7432E + 01	5.9740E - 01	9.7719E - 01	3.3712E - 06	1.5301E - 03	2.1560E - 03	1.3730E - 05	3.7264E - 07
	Worst	8.4378E + 03	3.8321E + 05	1.7473E + 01	2.5600E + 08	7.2307E - 02	2.2684E - 02	1.8478E - 02	3.8684E + 00	1.4741E - 05
	Mean	6.7235E + 02	6.2327E + 04	7.8395E + 00	8.5333E + 06	2.7619E - 02	6.9518E - 03	8.9559E - 03	7.3724E - 01	5.3640E - 06
	STD	2.0812E + 03	8.9944E + 04	3.9412E + 00	4.6739E + 07	1.6108E - 02	5.7680E - 03	3.6551E - 03	1.3682E + 00	3.8316E - 06
F13	Best	2.2905E + 00	2.1916E + 04	1.8925E + 00	1.1144E + 00	6.7487E - 05	4.1228E - 02	1.5623E - 02	1.8808E - 05	1.4826E - 05
	Worst	3.4001E + 05	1.5695E + 06	3.9319E + 00	1.7142E + 01	7.6352E - 01	4.7741E - 01	8.9205E - 02	3.1710E + 00	2.3648E - 04
	Mean	1.7813E + 04	3.2994E + 05	3.0632E + 00	7.2988E + 00	3.6956E - 01	2.2407E - 01	5.1799E - 02	1.2199E - 01	8.3312E - 05
	STD	6.3079E + 04	3.2431E + 05	5.5614E - 01	4.9839E + 00	2.1122E - 01	1.1585E - 01	1.8122E - 02	5.7955E - 01	5.3775E - 05

Bold Values indicate best solution obtained

Table 5 Results of benchmark functions (F1–F13), with 100 dimensions

No	SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO	PO
F1	Best	3.1540E + 03	2.2881E + 05	3.5801E – 13	2.9629E + 04	8.7023E – 16	6.9598E – 95	5.0799E – 53	1.0954E – 03
	Worst	1.9557E + 04	2.8242E + 05	9.1433E – 11	8.8089E + 04	1.1300E – 14	2.5607E – 81	3.8735E – 40	4.5043E – 01
	Mean	9.9859E + 03	2.6096E + 05	1.7830E – 11	5.5253E + 04	4.1670E – 15	9.5765E – 83	1.4067E – 41	4.7426E – 02
	STD	4.1843E + 03	1.4466E + 04	1.9101E – 11	1.4145E + 04	2.5754E – 15	4.6730E – 82	7.0571E – 41	8.5932E – 02
F2	Best	5.2976E – 01	1.4989E + 31	9.8024E – 09	1.7975E + 02	5.0480E – 10	2.8167E – 58	1.6572E – 12	2.3948E – 01
	Worst	2.5621E + 01	9.3671E + 38	1.3086E – 07	3.1931E + 02	2.7904E – 09	8.6166E – 52	3.5033E – 09	3.3935E + 00
	Mean	6.3675E + 00	1.2501E + 38	5.5473E – 08	2.4748E + 02	1.5421E – 09	4.9318E – 53	3.5559E – 10	1.0511E + 00
	STD	5.3174E + 00	2.5015E + 38	3.6343E – 08	4.0680E + 01	6.0014E – 10	1.6611E – 52	7.6094E – 10	7.2713E – 01
F3	Best	1.1809E + 05	5.9826E + 05	1.1784E + 03	1.4411E + 05	4.5520E + 00	5.2716E + 05	5.0913E – 45	3.4022E – 01
	Worst	4.0465E + 05	1.4543E + 06	1.5020E + 04	3.4097E + 05	6.4015E + 02	1.2099E + 06	7.7761E – 35	8.6867E + 05
	Mean	2.2940E + 05	9.5234E + 05	6.2841E + 03	2.0989E + 05	1.1233E + 02	8.6097E + 05	6.5640E – 36	1.6828E + 05
	STD	6.6950E + 04	1.9506E + 05	3.7867E + 03	4.9603E + 04	1.4792E + 02	1.8741E + 05	1.7926E – 35	2.4124E + 05
F4	Best	7.2710E + 01	9.1703E + 01	2.7356E + 01	8.6003E + 01	1.5260E – 02	1.0892E – 01	3.4807E – 22	5.6107E – 03
	Worst	9.2697E + 01	9.7073E + 01	6.5725E + 01	9.5928E + 01	8.3858E – 01	9.5301E + 01	1.1309E – 17	2.1949E + 00
	Mean	8.6853E + 01	9.5571E + 01	4.5580E + 01	9.2125E + 01	2.0524E – 01	6.5908E + 01	1.3775E – 18	9.6297E – 02
	STD	4.2329E + 00	1.2192E + 00	9.5311E + 00	2.2619E + 00	2.4236E – 01	2.8043E + 01	2.6326E – 18	3.9675E – 01
F5	Best	9.3728E + 06	9.5441E + 08	9.7007E + 01	4.2227E + 07	9.5995E + 01	9.7579E + 01	3.5342E – 01	4.6746E – 02
	Worst	1.9774E + 08	1.2631E + 09	9.8614E + 01	2.5747E + 08	9.8439E + 01	9.8206E + 01	9.8920E + 01	9.9202E + 01
	Mean	9.8852E + 07	1.1100E + 09	9.8228E + 01	1.0565E + 08	9.7646E + 01	9.7996E + 01	1.7348E + 01	4.6963E + 00
	STD	5.0694E + 07	7.9107E + 07	4.1877E – 01	5.3394E + 07	8.9435E – 01	1.9337E – 01	3.6870E + 01	1.7897E + 01
F6	Best	1.1189E + 03	2.1166E + 05	1.1557E + 01	2.5569E + 04	6.5278E + 00	9.0226E – 01	7.3367E – 01	1.2170E – 03
	Worst	2.5392E + 04	2.8113E + 05	1.5875E + 01	9.2954E + 04	9.9892E + 00	2.6375E + 00	2.2607E + 01	7.3407E – 02
	Mean	9.7961E + 03	2.5451E + 05	1.4047E + 01	5.0386E + 04	8.1005E + 00	1.7629E + 00	9.8228E + 00	1.9078E – 02
	STD	6.5091E + 03	2.0352E + 04	1.1944E + 00	1.7921E + 04	9.5425E – 01	4.7833E – 01	8.0728E + 00	1.9378E – 02
F7	Best	3.4005E + 01	1.4548E + 03	1.6215E – 02	5.4556E + 01	1.8079E – 03	3.5846E – 05	3.3301E – 04	3.3261E – 03
	Worst	3.0264E + 02	2.1181E + 03	4.8416E – 02	4.5686E + 02	7.9740E – 03	1.4112E – 02	4.8531E – 03	1.7823E – 01
	Mean	1.4544E + 02	1.7913E + 03	2.9449E – 02	2.2557E + 02	4.1933E – 03	2.4358E – 03	1.8589E – 03	4.1620E – 02
	STD	7.0069E + 01	1.3846E + 02	8.5410E – 03	1.0953E + 02	1.6233E – 03	3.4256E – 03	1.1276E – 03	4.6153E – 02
F8	Best	– 8.1887E + 03	– 1.7403E + 63	– 1.5095E + 04	– 2.6019E + 04	– 1.9585E + 04	– 4.1898E + 04	– 3.9494E + 04	– 4.1895E + 04
	Worst	– 6.4194E + 03	– 2.5072E + 57	– 1.2491E + 04	– 1.9739E + 04	– 6.5451E + 03	– 2.9564E + 04	– 3.8725E + 03	– 2.4518E + 04
	Mean	– 7.1522E + 03	– 6.2617E + 61	– 1.3645E + 04	– 2.3267E + 04	– 1.6847E + 04	– 3.8373E + 04	– 4.1863E + 04	– 1.7018E + 04
	STD	4.6556E + 02	3.1699E + 62	7.1560E + 02	1.8236E + 03	2.2112E + 03	4.0729E + 03	7.0610E + 01	1.0746E + 04
F9	Best	3.4562E + 00	1.5512E + 03	7.3463E + 02	6.8878E + 02	1.0346E – 11	0.0000E + 00	0.0000E + 00	3.5179E – 02
	Worst	4.4087E + 02	1.7126E + 03	1.2831E + 03	1.0855E + 03	1.9025E + 01	2.2737E – 13	0.0000E + 00	1.1693E + 02
	Mean	2.3201E + 02	1.6163E + 03	9.8350E + 02	8.2742E + 02	5.8398E + 00	7.5791E – 15	0.0000E + 00	4.2511E + 01
	STD	1.1341E + 02	4.0863E + 01	1.2670E + 02	8.9315E + 01	5.4815E + 00	4.1513E – 14	0.0000E + 00	5.0843E + 01

Table 5 (continued)

No	SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO	PO
F10	Best	5.1396E + 00	2.0781E + 01	3.3523E - 07	1.9387E + 01	2.6412E - 09	4.4409E - 16	8.9310E - 03	4.4409E - 16
	Worst	2.0668E + 01	2.0985E + 01	3.0665E + 00	1.9964E + 01	1.1837E - 08	3.5216E - 13	4.8717E - 01	4.4409E - 16
	Mean	1.7124E + 01	2.0896E + 01	1.5241E - 01	1.9896E + 01	6.7424E - 09	2.4573E - 15	6.9157E - 02	4.4409E - 16
	STD	5.5983E + 00	5.6673E - 02	6.1514E - 01	1.2471E - 01	2.4411E - 09	1.7906E - 15	8.4160E - 02	0.0000E + 00
F11	Best	1.6207E + 01	1.8276E + 03	1.0638E - 12	1.6028E + 02	3.1086E - 15	0.0000E + 00	3.3470E - 05	0.0000E + 00
	Worst	2.9705E + 02	2.5115E + 03	3.3981E - 02	8.3908E + 02	1.8461E - 02	0.0000E + 00	1.6338E + 01	0.0000E + 00
	Mean	1.1015E + 02	2.2752E + 03	7.5153E - 03	4.9576E + 02	1.1337E - 03	0.0000E + 00	1.1874E + 00	0.0000E + 00
	STD	7.6190E + 01	1.5232E + 02	1.1155E - 02	1.3008E + 02	4.3314E - 03	0.0000E + 00	3.5416E + 00	0.0000E + 00
F12	Best	6.6717E + 07	1.8096E + 09	9.8140E - 01	3.2459E + 07	1.0181E - 01	8.0757E - 03	2.6927E - 06	6.1531E - 04
	Worst	5.7697E + 08	3.2720E + 09	2.9312E + 01	5.8811E + 08	3.7537E - 01	1.3133E - 02	1.2633E + 01	1.3686E - 02
	Mean	2.7047E + 08	2.7016E + 09	1.1249E + 01	1.7557E + 08	2.0895E - 01	1.8376E - 02	6.3016E - 01	4.4763E - 03
	STD	1.3353E + 08	3.2018E + 08	5.2163E + 00	1.6036E + 08	6.2484E - 02	2.5644E - 03	2.4022E + 00	3.4897E - 03
F13	Best	1.2635E + 08	3.8282E + 09	9.7005E + 00	8.5148E + 07	4.4192E + 00	7.2908E - 01	1.3182E - 04	4.6462E - 02
	Worst	1.2829E + 09	5.7099E + 09	1.7308E + 01	1.4560E + 09	7.1520E + 00	3.8612E + 00	1.8758E - 01	1.0221E + 00
	Mean	4.9198E + 08	5.0396E + 09	1.2587E + 01	4.8385E + 08	6.0128E + 00	1.4894E + 00	8.9687E - 03	3.1307E - 01
	STD	2.4760E + 08	4.4456E + 08	2.1371E + 00	3.1888E + 08	6.7643E - 01	5.1544E - 02	3.3856E - 02	2.0309E - 01

Bold Values indicate best solution obtained

Table 6 Results of benchmark functions (F1–F13), with 500 dimensions

No		SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO	PO
F1	Best	1.2672E + 05	1.4597E + 06	1.1832E - 03	1.0873E + 06	7.3923E - 05	5.7570E - 92	2.4565E - 47	1.8686E - 03	2.7232E - 285
	Worst	2.9483E + 05	1.5772E + 06	2.8523E - 02	1.1928E + 06	4.0018E - 04	5.8554E - 81	2.9863E - 37	7.5519E - 01	1.6034E - 273
	Mean	2.0687E + 05	1.5188E + 06	6.8359E - 03	1.1407E + 06	1.8714E - 04	2.1590E - 82	1.0904E - 38	1.2863E - 01	5.9823E - 275
	STD	4.8810E + 04	3.6286E + 04	6.5659E - 03	2.7703E + 04	7.3563E - 05	1.0676E - 81	5.4581E - 38	1.6050E - 01	0.0000E + 00
F2	Best	2.2400E + 01	3.3284E + 241	1.1884E - 03	7.1049E + 64	2.4045E - 03	5.3019E - 57	3.8031E - 11	9.0672E - 01	4.2328E - 139
	Worst	1.7969E + 02	5.5897E + 264	1.0486E - 02	1.6175E + 117	4.2522E - 03	3.7254E - 50	3.3857E - 09	1.2022E + 01	2.0964E - 132
	Mean	9.0912E + 01	1.8633E + 263	4.5423E - 03	5.3919E + 115	3.0205E - 03	1.3184E - 51	5.1511E - 10	4.1912E + 00	1.9446E - 133
	STD	4.4858E + 01	Inf	2.4856E - 03	2.9532E + 116	4.4593E - 04	6.7888E - 51	6.9158E - 10	2.6656E + 00	5.1594E - 133
F3	Best	3.8801E + 06	1.2225E + 07	8.0431E + 05	3.1727E + 06	1.5623E + 05	1.1734E + 07	6.1061E - 41	6.5248E + 01	1.4261E - 262
	Worst	9.8721E + 06	3.3679E + 07	1.8656E + 06	6.4550E + 06	4.3134E + 05	4.5027E + 07	2.0182E - 29	1.8869E + 07	1.9133E - 241
	Mean	6.4146E + 06	2.4714E + 07	1.2377E + 06	4.4473E + 06	2.4331E + 05	2.6019E + 07	1.2704E - 30	7.6716E + 06	6.8706E - 243
	STD	1.3315E + 06	5.1390E + 06	2.2190E + 05	9.3785E + 05	7.8053E + 04	8.2222E + 06	4.7846E - 30	7.7841E + 06	0.0000E + 00
F4	Best	9.8011E + 01	9.8712E + 01	9.8641E + 01	9.7988E + 01	4.6216E + 01	3.2657E + 00	4.1683E - 21	2.2615E - 03	2.9703E - 144
	Worst	9.9440E + 01	9.9435E + 01	9.9564E + 01	9.9126E + 01	6.8747E + 01	9.9286E + 01	4.6681E - 17	6.1008E - 02	6.0541E - 140
	Mean	9.9016E + 01	9.9216E + 01	9.9114E + 01	9.8688E + 01	5.8442E + 01	8.1130E + 01	3.8559E - 18	1.6397E - 02	5.7762E - 141
	STD	2.9435E - 01	1.6690E - 01	2.2534E - 01	3.7241E - 01	4.9416E + 00	2.3825E + 01	1.0178E - 17	1.4058E - 02	1.2557E - 140
F5	Best	8.3079E + 08	6.3237E + 09	1.3121E + 03	4.4391E + 09	4.9638E + 02	4.9457E + 02	5.3142E - 01	8.9428E - 02	4.9422E + 02
	Worst	2.7997E + 09	7.3960E + 09	7.6868E + 04	5.4636E + 09	4.9773E + 02	4.9608E + 02	4.9896E + 02	4.9443E + 02	4.9615E + 02
	Mean	1.8547E + 09	6.9864E + 09	1.6503E + 04	4.8295E + 09	4.9741E + 02	4.9543E + 02	3.5949E + 02	2.3095E + 01	4.9505E + 02
	STD	4.6729E + 08	2.1372E + 08	1.6894E + 04	2.1603E + 08	2.4457E - 01	3.7191E - 01	2.1290E + 02	8.9459E + 01	4.2659E - 01
F6	Best	4.9222E + 04	1.3815E + 06	9.3069E + 01	1.0432E + 06	8.2081E + 01	8.4805E + 00	5.0846E + 00	7.6303E + 00	6.7452E - 03
	Worst	3.1821E + 05	1.5634E + 06	1.0217E + 02	1.1955E + 06	8.9846E + 01	2.2665E + 01	1.2409E + 02	4.9868E + 01	4.0464E + 02
	Mean	1.7451E + 05	1.5172E + 06	9.8013E + 01	1.1326E + 06	8.6451E + 01	1.6317E + 01	9.0430E + 01	3.1251E + 01	1.3624E + 01
	STD	6.4835E + 04	3.8990E + 04	2.3137E + 00	3.4303E + 04	1.8450E + 00	3.9258E + 00	4.6694E + 01	9.9675E + 00	7.3851E + 00
F7	Best	9.7978E + 03	4.8915E + 04	3.6800E - 01	3.3120E + 04	1.5624E - 02	2.3852E - 05	4.2038E - 04	4.4137E - 03	6.0289E - 06
	Worst	2.1893E + 04	6.1295E + 04	2.8449E + 00	4.0779E + 04	4.0951E - 02	8.6571E - 03	5.1124E - 03	8.3531E - 01	6.1850E - 04
	Mean	1.5655E + 04	5.7312E + 04	1.3585E + 00	3.6846E + 04	2.8154E - 02	2.4323E - 03	2.2506E - 03	9.3854E - 02	1.4793E - 04
	STD	3.1164E + 03	2.5402E + 03	5.1362E - 01	2.1354E + 03	6.9844E - 03	2.4406E - 03	1.1199E - 03	1.6164E - 01	1.3555E - 04
F8	Best	-1.8490E + 04	-2.6209E + 61	-3.6615E + 04	-7.4392E + 04	-8.0508E + 04	-2.0949E + 05	-2.0949E + 05	-1.9834E + 05	-2.0714E + 05
	Worst	-1.4226E + 04	-7.8238E + 55	-2.7640E + 04	-5.8033E + 04	-5.2437E + 04	-1.3440E + 05	-2.0775E + 05	-8.8515E + 03	-9.1667E + 04
	Mean	-1.6018E + 04	-2.1186E + 60	-3.2388E + 04	-6.4758E + 04	-6.1590E + 04	-1.7973E + 05	-2.0931E + 05	-8.2537E + 04	-1.5364E + 05
	STD	1.0384E + 03	5.2843E + 60	2.2959E + 03	5.1581E + 03	5.5829E + 03	2.8166E + 04	3.3993E + 02	5.8222E + 04	3.3584E + 04
F9	Best	4.3855E + 02	8.4781E + 03	4.3140E + 03	6.4721E + 03	2.5973E + 01	0.0000E + 00	0.0000E + 00	2.4827E - 01	0.0000E + 00
	Worst	1.8803E + 03	8.9375E + 03	6.8259E + 03	7.1254E + 03	1.1790E + 02	1.8190E - 12	0.0000E + 00	5.8979E + 02	0.0000E + 00
	Mean	1.1733E + 03	8.7632E + 03	5.8712E + 03	6.8059E + 03	5.7701E + 01	6.0633E - 14	0.0000E + 00	1.4842E + 02	0.0000E + 00
	STD	3.7647E + 02	1.2391E + 02	5.1063E + 02	1.5626E + 02	2.3534E + 01	3.3210E - 13	0.0000E + 00	2.2697E + 02	0.0000E + 00

Table 6 (continued)

No		SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO	PO
F10	Best	7.0212E + 00	2.1035E + 01	2.2130E - 03	1.9975E + 01	4.1617E - 04	4.4409E - 16	4.4409E - 02	1.7936E - 02	4.4409E - 16
	Worst	2.0822E + 01	2.1142E + 01	8.6538E - 02	2.0459E + 01	8.3874E - 04	7.5495E - 15	1.0272E - 01	3.3553E - 01	4.4409E - 16
	Mean	1.8964E + 01	2.1103E + 01	7.6237E - 03	2.0258E + 01	6.1186E - 04	3.6415E - 15	6.6880E - 02	7.3072E - 02	4.4409E - 16
	STD	3.9075E + 00	2.3226E - 02	1.5035E - 02	1.4151E - 01	1.0687E - 04	2.1580E - 15	1.9028E - 02	7.3091E - 02	0.0000E + 00
F11	Best	4.3383E + 02	1.3277E + 04	1.8085E - 04	9.5181E + 03	8.5775E - 06	0.0000E + 00	0.0000E + 00	6.8972E - 05	0.0000E + 00
	Worst	3.7568E + 03	1.4195E + 04	1.3505E - 01	1.0983E + 04	7.4911E - 02	0.0000E + 00	0.0000E + 00	2.2367E + 02	0.0000E + 00
	Mean	1.6670E + 03	1.3670E + 04	1.4976E - 02	1.0204E + 04	9.7777E - 03	0.0000E + 00	0.0000E + 00	2.3723E + 01	0.0000E + 00
	STD	7.1355E + 02	2.3825E + 02	3.7855E - 02	3.0178E + 02	2.3217E - 02	0.0000E + 00	0.0000E + 00	5.8212E + 01	0.0000E + 00
F12	Best	4.1245E + 09	1.6216E + 10	9.9498E + 03	9.7269E + 09	6.1979E - 01	1.5622E - 02	8.8603E - 04	1.3158E - 06	2.9472E - 02
	Worst	9.0847E + 09	1.8704E + 10	1.6333E + 06	1.2411E + 10	7.8391E - 01	7.9800E - 02	9.4067E - 03	3.1101E + 00	2.5384E - 01
	Mean	5.9512E + 09	1.7423E + 10	4.4530E + 05	1.1170E + 10	6.9549E - 01	3.5221E - 02	4.2967E - 03	3.1120E - 01	1.0946E - 01
	STD	1.1390E + 09	5.8454E + 08	4.1841E + 05	7.1059E + 08	3.4597E - 02	1.5916E - 02	1.9890E - 03	9.4885E - 01	6.1355E - 02
F13	Best	6.2126E + 09	2.9877E + 10	1.1067E + 04	1.8485E + 10	4.5923E + 01	5.2129E + 00	3.2945E - 03	2.5888E - 04	6.9953E + 00
	Worst	1.3698E + 10	3.3254E + 10	7.4933E + 05	2.5134E + 10	5.0397E + 01	1.5178E + 01	4.7055E - 01	7.2581E - 02	2.8342E + 01
	Mean	9.4487E + 09	3.1851E + 10	1.6204E + 05	2.1432E + 10	4.7953E + 01	9.3914E + 00	2.2056E - 01	1.3349E - 02	1.6722E + 01
	STD	1.7570E + 09	9.6654E + 08	1.6522E + 05	1.4395E + 09	1.1528E + 00	2.9642E + 00	1.1826E - 01	1.8694E - 02	5.0807E + 00

Bold Values indicate best solution obtained

Table 7 Results of benchmark functions (F1–F13), with 1000 dimensions

No	SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO	PO	
F1	Best	1.5322E + 05	3.0221E + 06	2.9974E – 01	2.6205E + 06	3.1057E – 02	6.2075E – 94	4.3645E – 47	7.6393E – 03	2.2428E – 288
	Worst	7.3980E + 05	3.1990E + 06	2.8846E + 00	2.7728E + 06	8.4583E – 02	3.8921E – 79	1.0298E – 36	1.7085E + 00	2.5021E – 275
	Mean	5.0613E + 05	3.1193E + 06	1.2351E + 00	2.7030E + 06	5.1072E – 02	2.6692E – 80	4.2689E – 38	3.2407E – 01	9.5315E – 277
	STD	1.5735E + 05	4.3802E + 04	6.3176E – 01	4.0908E + 04	1.4082E – 02	9.1013E – 80	1.9015E – 37	3.6123E – 01	0.0000E + 00
F2	Best	6.5535E + 04	6.5535E + 04	5.5224E – 03	6.5535E + 04	9.6451E – 02	2.1313E – 58	1.5496E – 11	1.5543E + 00	5.2424E – 151
	Worst	6.5535E + 04	6.5535E + 04	3.8831E – 02	6.5535E + 04	7.7025E – 01	6.7348E – 49	9.1249E – 09	1.8019E + 01	2.5176E – 147
	Mean	6.5535E + 04	6.5535E + 04	1.8548E – 02	6.5535E + 04	2.9219E – 01	2.2723E – 50	7.2868E – 10	8.4754E + 00	1.5388E – 148
	STD	0.0000E + 00	0.0000E + 00	8.7509E – 03	0.0000E + 00	1.7257E – 01	1.2291E – 49	1.6293E – 09	4.6687E + 00	4.6360E – 148
F3	Best	1.6183E + 07	5.8022E + 07	4.0196E + 06	1.2464E + 07	7.8994E + 05	5.6927E + 07	6.0379E – 43	2.4400E + 03	1.6336E – 257
	Worst	3.1095E + 07	1.5567E + 08	7.0880E + 06	2.5160E + 07	1.6762E + 06	1.7589E + 08	1.8239E – 26	9.5763E + 07	2.4751E – 240
	Mean	2.5054E + 07	1.0755E + 08	5.3497E + 06	1.6814E + 07	1.1523E + 06	1.1091E + 08	7.8985E – 28	3.9785E + 07	1.3962E – 241
	STD	3.6953E + 06	2.3231E + 07	7.7191E + 05	2.9645E + 06	2.4918E + 05	3.4206E + 07	3.4194E – 27	3.5213E + 07	0.0000E + 00
F4	Best	9.9228E + 01	9.8956E + 01	9.8827E + 01	9.9064E + 01	7.1453E + 01	6.4030E + 00	1.8237E – 21	3.5001E – 03	1.5045E – 144
	Worst	9.9732E + 01	9.9704E + 01	9.9767E + 01	9.9713E + 01	8.5973E + 01	9.9236E + 01	2.4666E – 17	5.9358E – 02	2.5355E – 139
	Mean	9.9535E + 01	9.9531E + 01	9.9541E + 01	9.9434E + 01	7.8024E + 01	8.2410E + 01	3.2556E – 18	1.9798E – 02	1.5957E – 140
	STD	1.3448E – 01	1.6923E – 01	1.8058E – 01	1.3692E – 01	3.5761E + 00	2.0809E + 01	7.1596E – 18	1.2686E – 02	4.8484E – 140
F5	Best	2.9235E + 09	1.4341E + 10	1.4087E + 06	1.1566E + 10	1.0015E + 03	9.9058E + 02	2.7461E + 00	1.2256E + 00	9.9024E + 02
	Worst	6.0823E + 09	1.5065E + 10	2.5348E + 07	1.3000E + 10	1.0175E + 03	9.9312E + 02	9.9892E + 02	9.8949E + 02	9.9559E + 02
	Mean	4.3785E + 09	1.4710E + 10	9.1266E + 06	1.2348E + 10	1.0065E + 03	9.9192E + 02	8.8033E + 02	4.8192E + 01	9.9249E + 02
	STD	8.8119E + 08	1.9474E + 08	5.9392E + 06	3.3737E + 08	3.9645E + 00	5.9990E – 01	3.0227E + 02	1.7851E + 02	1.3218E + 00
F6	Best	1.8915E + 05	3.0593E + 06	2.1431E + 02	2.5681E + 06	1.8736E + 02	2.2102E + 01	1.2201E + 02	2.4866E + 01	4.0763E – 04
	Worst	8.1870E + 05	3.2008E + 06	2.3070E + 02	2.7992E + 06	1.9884E + 02	5.3095E + 01	2.4868E + 02	1.4647E + 02	1.6208E + 00
	Mean	4.8951E + 05	3.1290E + 06	2.2422E + 02	2.7063E + 06	1.9342E + 02	3.5649E + 01	2.3339E + 02	9.3104E + 01	2.8912E – 01
	STD	1.4883E + 05	3.7736E + 04	3.2911E + 00	5.1755E + 04	3.0116E + 00	8.8533E + 00	2.8790E + 01	2.8433E + 01	4.1562E – 01
F7	Best	4.5756E + 04	2.2914E + 05	8.1579E + 00	1.7951E + 05	5.5180E – 02	1.7703E – 05	3.4495E – 04	1.3767E – 02	1.4996E – 05
	Worst	9.1585E + 04	2.4845E + 05	4.8770E + 02	2.0455E + 05	1.1500E – 01	1.0187E – 02	5.5841E – 03	5.6996E – 01	3.6869E – 04
	Mean	6.6139E + 04	2.4002E + 05	9.9974E + 01	1.9359E + 05	8.4075E – 02	1.8358E – 03	2.7702E – 03	1.2161E – 01	1.1846E – 04
	STD	1.2711E + 04	5.7091E + 03	8.5520E + 01	5.0808E + 03	1.5675E – 02	2.1579E – 03	1.4184E – 03	1.3042E – 01	8.6279E – 05
F8	Best	– 2.6782E + 04	– 7.9133E + 62	– 5.5299E + 04	– 1.1162E + 05	– 1.1103E + 05	– 4.1898E + 05	– 3.9310E + 05	– 4.1535E + 05	– 4.1535E + 05
	Worst	– 1.9374E + 04	– 3.3902E + 55	– 3.9063E + 04	– 8.4419E + 04	– 2.0585E + 04	– 2.6786E + 05	– 4.1428E + 05	– 1.5336E + 04	– 1.3561E + 05
	Mean	– 2.2408E + 04	– 3.2004E + 61	– 4.6025E + 04	– 9.4028E + 04	– 9.5823E + 04	– 3.6657E + 05	– 4.1854E + 05	– 1.9021E + 05	– 2.6935E + 05
	STD	1.5891E + 03	1.4482E + 62	3.7872E + 03	6.4251E + 03	1.5359E + 04	5.7969E + 04	9.2584E + 02	1.1995E + 05	6.5499E + 04
F9	Best	8.1724E + 02	1.7572E + 04	6.6977E + 03	1.4631E + 04	9.9943E + 01	0.0000E + 00	0.0000E + 00	1.1336E + 00	0.0000E + 00
	Worst	3.2843E + 03	1.8087E + 04	1.2717E + 04	1.5628E + 04	2.3158E + 02	0.0000E + 00	0.0000E + 00	3.9997E + 03	0.0000E + 00
	Mean	2.0242E + 03	1.7806E + 04	1.0561E + 04	1.5198E + 04	1.5990E + 02	0.0000E + 00	0.0000E + 00	2.6047E + 02	0.0000E + 00
	STD	7.3579E + 02	1.3663E + 02	1.7166E + 03	2.3864E + 02	3.4232E + 01	0.0000E + 00	0.0000E + 00	7.6780E + 02	0.0000E + 00

Table 7 (continued)

No		SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO	PO
F10	Best	9.4090E + 00	2.1107E + 01	2.0329E - 02	2.0067E + 01	6.2040E - 03	4.4409E - 16	4.4409E - 16	8.0331E - 03	4.4409E - 16
	Worst	2.0846E + 01	2.1169E + 01	1.0171E - 01	2.0592E + 01	1.0180E - 02	7.5495E - 15	7.4296E - 13	1.8754E - 01	4.4409E - 16
	Mean	1.9522E + 01	2.1144E + 01	4.7176E - 02	2.0306E + 01	7.6237E - 03	3.9968E - 15	5.5748E - 14	5.0151E - 02	4.4409E - 16
	STD	3.3675E + 00	1.7690E - 02	1.8243E - 02	1.8036E - 01	1.0328E - 03	2.7990E - 15	1.3514E - 13	4.3416E - 02	0.0000E + 00
F11	Best	1.1728E + 03	2.7208E + 04	1.7431E - 02	2.3330E + 04	2.1377E - 03	0.0000E + 00	0.0000E + 00	2.3133E - 05	0.0000E + 00
	Worst	6.4529E + 03	2.8820E + 04	4.8672E - 01	2.5003E + 04	1.6422E - 01	0.0000E + 00	0.0000E + 00	1.0847E + 02	0.0000E + 00
	Mean	3.8815E + 03	2.8054E + 04	1.1703E - 01	2.4282E + 04	1.7693E - 02	0.0000E + 00	0.0000E + 00	7.0415E + 00	0.0000E + 00
	STD	1.1535E + 03	4.5695E + 02	1.2686E - 01	4.7171E + 02	4.4140E - 02	0.0000E + 00	0.0000E + 00	2.1774E + 01	0.0000E + 00
F12	Best	7.6397E + 09	3.4500E + 10	1.1542E + 07	2.7468E + 10	8.0716E - 01	1.6828E - 02	9.1386E - 04	1.3415E - 06	1.0342E - 01
	Worst	1.6869E + 10	3.8492E + 10	6.7496E + 08	3.0925E + 10	1.4121E + 00	8.9318E - 02	7.5694E - 03	1.2562E + 01	5.5354E - 01
	Mean	1.3380E + 10	3.6785E + 10	1.8585E + 08	2.9624E + 10	9.4541E - 01	3.8058E - 02	4.2593E - 03	1.1454E + 00	2.8556E - 01
	STD	2.3048E + 09	8.9173E + 08	1.4213E + 08	8.6823E + 08	1.2120E - 01	1.6278E - 02	1.7463E - 03	2.5348E + 00	1.2729E - 01
F13	Best	1.4150E + 10	5.9951E + 10	1.7940E + 07	5.0826E + 10	1.0507E + 02	8.0596E + 00	1.5520E - 03	3.3474E - 04	3.3195E + 01
	Worst	2.9930E + 10	6.8916E + 10	3.7308E + 08	5.8641E + 10	1.2263E + 02	3.1883E + 01	6.2478E - 01	5.1527E - 01	9.9090E + 01
	Mean	2.1753E + 10	6.5631E + 10	1.2334E + 08	5.4533E + 10	1.0976E + 02	1.8380E + 01	2.5618E - 01	4.7417E - 02	7.4450E + 01
	STD	3.7290E + 09	1.9759E + 09	8.3568E + 07	1.9185E + 09	3.3040E + 00	5.2527E + 00	1.5670E - 01	1.0632E - 01	2.3785E + 01

Bold Values indicate best solution obtained

Table 8 Results of benchmark functions (F14–F23)

No	SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO	PO
F14	Best	9.9800E + 01	9.9800E + 01	9.9800E + 01	9.9800E + 01	9.9800E + 01	9.9800E + 01	9.9800E + 01	9.9800E + 01
	Worst	2.9821E + 00	9.9800E + 01	5.9288E + 00	1.0763E + 01	1.0763E + 01	1.9920E + 00	8.2107E + 00	9.9800E + 01
	Mean	1.4622E + 00	9.9800E + 01	1.3940E + 00	4.3905E + 00	2.4376E + 00	1.0357E + 00	3.1044E + 00	9.9800E + 01
	STD	8.5284E + 01	0.0000E + 00	1.0246E + 00	4.3172E + 00	2.9332E + 00	1.8102E + 01	1.9181E + 00	0.0000E + 00
F15	Best	4.2737E + 04	8.6767E + 04	5.1520E + 04	3.0749E + 04	3.0775E + 04	5.0659E + 04	3.2734E + 04	3.0749E + 04
	Worst	1.5646E + 03	1.0652E + 03	7.1570E + 02	2.0363E + 02	1.3516E + 03	7.4293E + 03	2.2696E + 02	1.2232E + 03
	Mean	8.6829E + 04	9.9960E + 04	5.6019E + 03	4.3835E + 03	6.1845E + 04	1.2779E + 03	5.3088E + 03	4.2684E + 04
	STD	3.5051E + 04	5.4300E + 05	1.4234E + 02	8.1297E + 03	3.3045E + 04	1.4392E + 03	6.6988E + 03	2.4027E + 04
F16	Best	− 1.0316E + 00	− 1.0316E + 00	− 1.0316E + 00	− 1.0316E + 00	− 1.0316E + 00	− 1.0316E + 00	− 1.0316E + 00	− 1.0316E + 00
	Worst	− 1.0315E + 00	− 1.0316E + 00	− 1.0000E + 00	− 1.0316E + 00	− 1.0316E + 00	− 1.0316E + 00	− 1.0307E + 00	− 1.0316E + 00
	Mean	− 1.0316E + 00	− 1.0316E + 00	− 1.0274E + 00	− 1.0316E + 00	− 1.0316E + 00	− 1.0316E + 00	− 1.0315E + 00	− 1.0316E + 00
	STD	4.1996E + 05	6.7752E + 16	1.0936E + 02	6.7752E + 16	8.5576E + 09	9.6131E + 11	2.0053E + 05	6.7752E + 16
F17	Best	3.9790E + 01	3.9789E + 01	3.9789E + 01	3.9789E + 01	3.9789E + 01	3.9789E + 01	3.9790E + 01	3.9789E + 01
	Worst	4.0314E + 01	3.9789E + 01	3.9807E + 01	3.9789E + 01	3.9791E + 01	3.9959E + 01	4.2382E + 01	3.9789E + 01
	Mean	3.9888E + 01	3.9789E + 01	3.9791E + 01	3.9789E + 01	3.9789E + 01	3.9827E + 01	4.0229E + 01	3.9789E + 01
	STD	1.1941E + 03	0.0000E + 00	3.6212E + 05	0.0000E + 00	6.4529E + 07	4.0552E + 04	7.3926E + 03	0.0000E + 00
F18	Best	3.0000E + 00	3.0000E + 00	3.0000E + 00	3.0000E + 00	3.0000E + 00	3.0001E + 00	3.0003E + 00	3.0000E + 00
	Worst	3.0004E + 00	3.0000E + 00	9.2036E + 01	3.0000E + 00	3.0007E + 00	3.0065E + 00	4.2456E + 00	3.0000E + 00
	Mean	3.0001E + 00	3.0000E + 00	2.0636E + 01	3.0000E + 00	3.0000E + 00	3.0020E + 00	3.2220E + 00	3.0000E + 00
	STD	8.6898E + 05	1.2232E + 15	3.3986E + 01	1.4821E + 15	1.2467E + 04	1.9971E + 03	3.7004E + 01	1.1979E + 15
F19	Best	− 3.8619E + 00	− 3.8628E + 00	− 3.8628E + 00	− 3.8628E + 00	− 3.8628E + 00	− 3.8626E + 00	− 3.8627E + 00	− 3.8628E + 00
	Worst	− 3.8513E + 00	− 3.8628E + 00	− 3.8625E + 00	− 3.8628E + 00	− 3.8549E + 00	− 3.7925E + 00	− 3.7539E + 00	− 3.8628E + 00
	Mean	− 3.8548E + 00	− 3.8628E + 00	− 3.8627E + 00	− 3.8628E + 00	− 3.8619E + 00	− 3.8593E + 00	− 3.8492E + 00	− 3.8628E + 00
	STD	2.2346E + 03	2.7101E + 15	6.1928E + 05	2.7101E + 15	1.7281E + 03	1.2648E + 02	2.5990E + 02	2.5391E + 15
F20	Best	− 3.1472E + 00	− 3.3220E + 00	− 3.3216E + 00	− 3.3220E + 00	− 3.3220E + 00	− 3.2882E + 00	− 3.1909E + 00	− 3.3220E + 00
	Worst	− 1.9168E + 00	− 3.3220E + 00	− 3.1391E + 00	− 3.1376E + 00	− 3.0242E + 00	− 2.9994E + 00	− 1.7080E + 00	− 3.2031E + 00
	Mean	− 2.9571E + 00	− 3.3220E + 00	− 3.2509E + 00	− 3.2243E + 00	− 3.2619E + 00	− 3.2014E + 00	− 2.7423E + 00	− 3.2705E + 00
	STD	2.6187E + 01	1.8011E + 12	6.2897E + 02	5.1097E + 02	8.4967E + 02	6.6390E + 02	3.1229E + 01	5.9923E + 02
F21	Best	− 5.0084E + 00	− 1.0153E + 01	− 1.0095E + 01	− 1.0153E + 01	− 1.0153E + 01	− 9.8136E + 00	− 1.0152E + 01	− 1.0153E + 01
	Worst	− 4.9726E + 01	− 1.0152E + 01	− 2.6065E + 00	− 2.6305E + 00	− 2.6299E + 00	− 4.8786E + 00	− 2.6469E + 00	− 1.0153E + 01
	Mean	− 2.9222E + 00	− 1.0153E + 01	− 5.7167E + 00	− 8.4857E + 00	− 9.8149E + 00	− 8.6295E + 00	− 7.9576E + 00	− 1.0153E + 01
	STD	1.7384E + 00	2.6642E + 04	3.2076E + 00	3.1016E + 00	1.2817E + 00	1.5372E + 00	2.6880E + 00	3.8324E + 09

Table 8 (continued)

No	SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO	PO
F22	Best	- 8.7263E + 00	- 1.0403E + 01	- 1.0339E + 01	- 1.0403E + 01	- 1.0403E + 01	- 1.0287E + 01	- 1.0400E + 01	- 1.0403E + 01
	Worst	- 9.0520E - 01	- 1.0403E + 01	- 1.8334E + 00	- 2.7519E + 00	- 2.7648E + 00	- 5.0616E + 00	- 2.7511E + 00	- 3.7243E + 00
	Mean	- 4.1383E + 00	- 1.0403E + 01	- 8.2684E + 00	- 8.4421E + 00	- 8.5462E + 00	- 9.2271E + 00	- 8.4240E + 00	- 1.0180E + 01
	STD	2.4362E + 00	4.1756E - 06	3.3125E + 00	3.0936E + 00	2.9147E + 00	9.8023E - 01	2.7097E + 00	1.2193E + 00
F23	Best	- 7.2342E + 00	- 1.0536E + 01	- 1.0491E + 01	- 1.0536E + 01	- 1.0536E + 01	- 1.0368E + 01	- 1.0535E + 01	- 1.0536E + 01
	Worst	- 2.5657E + 00	- 1.0536E + 01	- 1.6741E + 00	- 2.8711E + 00	- 1.6763E + 00	- 8.0907E + 00	- 2.8682E + 00	- 1.0536E + 01
	Mean	- 4.7476E + 00	- 1.0536E + 01	- 6.9696E + 00	- 8.6280E + 00	- 8.1968E + 01	- 9.3670E + 00	- 8.5351E + 00	- 1.0536E + 01
	STD	9.5602E - 01	1.0166E - 08	3.9683E + 00	2.9955E + 00	9.8705E - 01	5.2027E - 01	2.7263E + 00	3.7101E - 09

Bold Values indicate best solution obtained

a statistical comparison, and the P-values obtained from this statistical test are shown in Table 10, 11, 12, 13, 14, 15. Values greater than 0.05 depicts that no considerable differences have been discovered between the results of the two compared optimizers in terms of this statistical test, and if the P-Value is less than 0.05, there are statistically huge differences between the performances of the two optimizers. It has been discovered. In Table 10, 11, 12, 13, P-Value values obtained from functions 1 to 13 with dimensions of 30, 100, 500, and 1000 are shown and benchmarks 14 to 23 in Table 14 and finally P-Values related to CEC2019 functions. It is shown in Table 15.

Looking at the P-Values shown in Table 10, 11, 12, 13, which are related to scalability tests, significant differences have been discovered in most cases, and the PO has a statistically superior performance than other optimization algorithms. But similar functions have been observed in benchmark 11 between PO, FHO, and WOA, which is equal in almost all dimensions in this function. In addition, the absence of significant differences between these three algorithms in function F1 in dimensions 100, 500, and 1000 has also been seen. In general, in benchmarks F1 to F13 in all dimensions, the PO has performed greater than other compared algorithms and achieved good and significant results.

In Table 14, the P-Values obtained from benchmarks 14 to 23 are shown that based on this statistical test, PO has once again performed well and significantly and has been able to generate and discover far better solutions than other compared algorithms, which signifies better search capabilities of the PO. But in some benchmarks, the statistical test has not been able to discover significant differences, which is highlighted in function F16 between PO, WOA, MFO, and DMOA algorithms. Also, the absence of significant differences in benchmarks 17 to 19 between PO, MFO, and DMOA has been observed.

Finally, in Table 15, the results obtained from the T-test using CEC 2019 functions are shown. By carefully inspecting at this table, it can be detected that PO only in the C1 benchmark between FHO and TSA did not have a statistically significant difference, but there are significant differences in other functions, and this indicates that the more complex and difficult the problem It is possible that the good performance of PO becomes better and more significant and can better explore and investigate the problem space. In general, according to a statistical comparison, PO has a better ability and efficiency than other compared optimizers and has been able to obtain quality solutions against a variety of problems, and this efficiency becomes more prominent with the increase in the difficulty of the problem.

Table 9 Results of CEC 2019

No		SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO	PO
C1	Best	6.8313E + 02	8.1509E + 05	1.0000E + 00	2.5536E + 04	1.0000E + 00	3.8781E + 03	1.0000E + 00	4.2079E + 01	1.0000E + 00
	Worst	1.7873E + 07	1.9301E + 07	6.9806E + 05	9.6733E + 07	2.7645E + 05	7.6391E + 07	1.0000E + 00	3.3121E + 07	1.0000E + 00
	Mean	3.3978E + 06	6.7567E + 06	5.7725E + 04	1.0864E + 07	3.6003E + 04	1.6940E + 07	1.0000E + 00	5.0922E + 06	1.0000E + 00
	STD	4.1637E + 06	4.5709E + 06	1.6999E + 05	2.3901E + 07	7.8005E + 04	1.8470E + 07	0.0000E + 00	8.1695E + 06	0.0000E + 00
C2	Best	1.2384E + 03	3.7966E + 03	1.3771E + 02	2.3659E + 02	7.9900E + 00	1.9471E + 03	4.3032E + 00	1.9245E + 03	4.2270E + 00
	Worst	9.7724E + 03	1.1769E + 04	2.0469E + 03	7.3383E + 03	1.1066E + 03	1.5775E + 04	5.0000E + 00	1.2451E + 04	4.9962E + 00
	Mean	4.5680E + 03	8.4761E + 03	8.9478E + 02	1.2636E + 03	4.4785E + 02	7.7559E + 03	4.9043E + 00	4.9503E + 03	4.6420E + 00
	STD	1.9037E + 03	1.7901E + 03	4.8222E + 02	1.9889E + 03	2.7017E + 02	3.5579E + 03	2.4899E + 01	2.1388E + 03	2.1392E + 01
C3	Best	7.0745E + 00	7.7513E + 00	2.3139E + 00	1.4118E + 00	1.4152E + 00	1.4837E + 00	5.7143E + 00	6.6573E + 00	1.7251E + 00
	Worst	1.1147E + 01	1.1432E + 01	1.1711E + 01	1.0712E + 01	1.0696E + 01	9.7114E + 00	8.5751E + 00	1.1250E + 01	8.6229E + 00
	Mean	9.6341E + 00	1.0611E + 01	8.2701E + 00	7.4658E + 00	3.4138E + 00	5.2692E + 00	7.5507E + 00	9.6371E + 00	5.0103E + 00
	STD	1.1798E + 00	7.0452E + 01	2.7904E + 00	2.3018E + 00	2.6355E + 00	2.2536E + 00	5.5476E + 01	1.3284E + 00	1.9438E + 00
C4	Best	3.5656E + 01	1.5727E + 01	2.0398E + 01	1.1945E + 01	4.1276E + 00	1.3760E + 01	2.5539E + 01	2.7234E + 01	8.9597E + 00
	Worst	6.9489E + 01	4.3886E + 01	9.1374E + 01	5.0273E + 01	4.5734E + 01	9.0653E + 01	4.5287E + 01	7.0654E + 01	3.5823E + 01
	Mean	5.2788E + 01	3.1074E + 01	6.6221E + 01	2.6793E + 01	1.7075E + 01	5.7554E + 01	3.7347E + 01	5.1676E + 01	1.9313E + 01
	STD	6.5251E + 00	6.5805E + 00	1.7058E + 01	9.7720E + 00	9.2497E + 00	1.9887E + 01	5.0623E + 00	9.6184E + 00	7.0406E + 00
C5	Best	4.9911E + 00	1.0512E + 00	2.1739E + 00	1.0467E + 00	1.2075E + 00	1.8285E + 00	2.7233E + 00	5.9275E + 00	1.0394E + 00
	Worst	2.4045E + 01	1.5727E + 00	8.8466E + 01	1.6753E + 01	4.5972E + 00	4.9428E + 00	2.1366E + 01	1.9675E + 01	1.3004E + 00
	Mean	1.1749E + 01	1.3376E + 00	3.5914E + 01	2.5883E + 00	2.0836E + 00	2.6373E + 00	5.7728E + 00	1.0187E + 01	1.1227E + 00
	STD	4.8076E + 00	1.3022E + 01	2.3844E + 01	3.8376E + 00	1.0444E + 00	6.8734E + 01	4.0177E + 00	3.2080E + 00	7.1008E + 02
C6	Best	5.8217E + 00	4.8596E + 00	4.8083E + 00	1.2745E + 00	1.3403E + 00	5.6045E + 00	4.5187E + 00	6.2974E + 00	1.0070E + 00
	Worst	1.0817E + 01	1.0451E + 01	1.1597E + 01	8.4829E + 00	5.3522E + 00	1.2203E + 01	8.6410E + 00	1.0152E + 01	4.6042E + 00
	Mean	7.8417E + 00	7.8534E + 00	8.2225E + 00	4.6383E + 00	2.7559E + 00	9.2206E + 00	6.2957E + 00	8.1315E + 00	1.7572E + 00
	STD	1.3924E + 00	1.6415E + 00	1.7485E + 00	1.8802E + 00	1.1629E + 00	1.4512E + 00	8.5519E + 01	1.0689E + 00	1.0504E + 00
C7	Best	1.0338E + 03	1.1433E + 03	8.5495E + 02	5.6642E + 02	1.8344E + 02	5.3959E + 02	1.4105E + 03	8.9391E + 02	2.4220E + 02
	Worst	1.9177E + 03	2.0840E + 03	1.8203E + 03	1.9013E + 03	1.5606E + 03	2.4549E + 03	2.4351E + 03	1.9784E + 03	1.1081E + 03
	Mean	1.6138E + 03	1.7190E + 03	1.2862E + 03	1.1159E + 03	8.0862E + 02	1.4889E + 03	1.9914E + 03	1.5957E + 03	7.0129E + 02
	STD	2.2451E + 02	1.7486E + 02	2.7316E + 02	3.1508E + 02	3.2303E + 02	4.3283E + 02	2.8125E + 02	2.1737E + 02	2.1921E + 02
C8	Best	4.1813E + 00	3.7403E + 00	3.6858E + 00	3.6887E + 00	2.7729E + 00	4.1521E + 00	4.0790E + 00	4.0842E + 00	2.7598E + 00
	Worst	5.0350E + 00	5.0649E + 00	5.2547E + 00	5.0475E + 00	4.8213E + 00	5.2489E + 00	5.3749E + 00	4.9658E + 00	4.7208E + 00
	Mean	4.6168E + 00	4.7422E + 00	4.6463E + 00	4.5064E + 00	3.8795E + 00	4.7255E + 00	4.7876E + 00	4.5270E + 00	3.8641E + 00
	STD	1.8196E + 01	2.3925E + 01	4.1742E + 01	3.2751E + 01	4.9874E + 01	2.9290E + 01	2.6038E + 01	2.1704E + 01	4.8769E + 01

Table 9 (continued)

No		SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO	PO
C9	Best	1.3945E + 00	1.1662E + 00	1.0956E + 00	1.0886E + 00	1.1060E + 00	1.1453E + 00	1.2412E + 00	1.2860E + 00	1.0788E + 00
	Worst	1.9792E + 00	1.3351E + 00	3.8076E + 00	1.7728E + 00	1.3760E + 00	1.8016E + 00	3.2748E + 00	1.8492E + 00	1.4747E + 00
	Mean	1.6642E + 00	1.2541E + 00	1.9511E + 00	1.3703E + 00	1.2280E + 00	1.4427E + 00	1.5763E + 00	1.5975E + 00	1.2105E + 00
	STD	1.4225E - 01	4.1415E - 02	9.5193E - 01	1.7792E - 01	7.2285E - 02	1.6051E - 01	4.2083E - 01	1.3567E - 01	1.2015E - 01
C10	Best	2.1370E + 01	2.1278E + 01	2.1367E + 01	2.1010E + 01	2.1173E + 01	2.1038E + 01	1.5781E + 01	2.1273E + 01	1.0015E + 00
	Worst	2.1717E + 01	2.1604E + 01	2.1679E + 01	2.1659E + 01	2.1645E + 01	2.1560E + 01	2.1610E + 01	2.1648E + 01	2.1399E + 01
	Mean	2.1523E + 01	2.1440E + 01	2.1506E + 01	2.1229E + 01	2.1471E + 01	2.1272E + 01	2.1142E + 01	2.1486E + 01	1.9787E + 01
	STD	8.6764E - 02	9.1180E - 02	7.5674E - 02	1.7517E - 01	1.2134E - 01	1.2923E - 01	1.1900E + 00	9.7257E - 02	5.0978E + 00

Bold Values indicate best solution obtained

4.6 Running time comparison

Comparison using the output results of optimizer algorithms alone cannot prove the efficiency of an algorithm because various factors reveal the ability of an algorithm, one of which is the duration of an algorithm's execution in solving a problem. Execution time is one of the important factors in distinguishing between optimizers, because the less time an algorithm spends on solving a problem, it can be the more suitable for dealing with a wide range of problems. To make a comparison based on the execution time, we have calculated the entry and exit time of each compared algorithm in benchmark F1 to F13 with dimensions of 30 as shown in Table 16.

According to the comparison based on the duration of execution illustrated in Table 16, it is clear that the PO, although it did not have the best performance, managed to achieve an average performance, but the noteworthy point is that the results obtained from This algorithm has been significantly better than other optimizer algorithms, and the average execution time can be recognized as a proper option for solving optimization problems. But the average performance of PO can have different reasons, one of these reasons can be the complexity of the implementation and multiple processes, including the selection of the smart phase, which must be precisely optimized, but in general, the average time required to solve the problems is acceptable that good and significant results can lead to neglecting the average execution time.

Considering that every optimization algorithm has its strengths and weaknesses because the optimization process and the equations used in the algorithms can be very different from each other, thus the Puma algorithm also has some strengths and weaknesses and is not an exception to this rule. In the Puma algorithm, a new type of phase change mechanism is presented, which leads to the selection of a better phase in different stages of optimization, which can be considered as an intelligent mechanism.

However, this smart selection can result in the complexity of the implementation process because smart processes usually lead to an increase in complexity. On the other hand, the Puma algorithm has many variables, for this reason, the memory must be well managed and the increase of the memory space should be avoided, which can be considered as the weak points of this algorithm. But with all these interpretations, the Puma algorithm can be a good option for solving problems in continuous and discrete optimization spaces due to its perfect innovation that for the first time the phases are selected automatically and intelligently, therefore it can provide many benefits to solve these problems. Moreover, in the future, it can be considered a very ideal hybrid method for combining

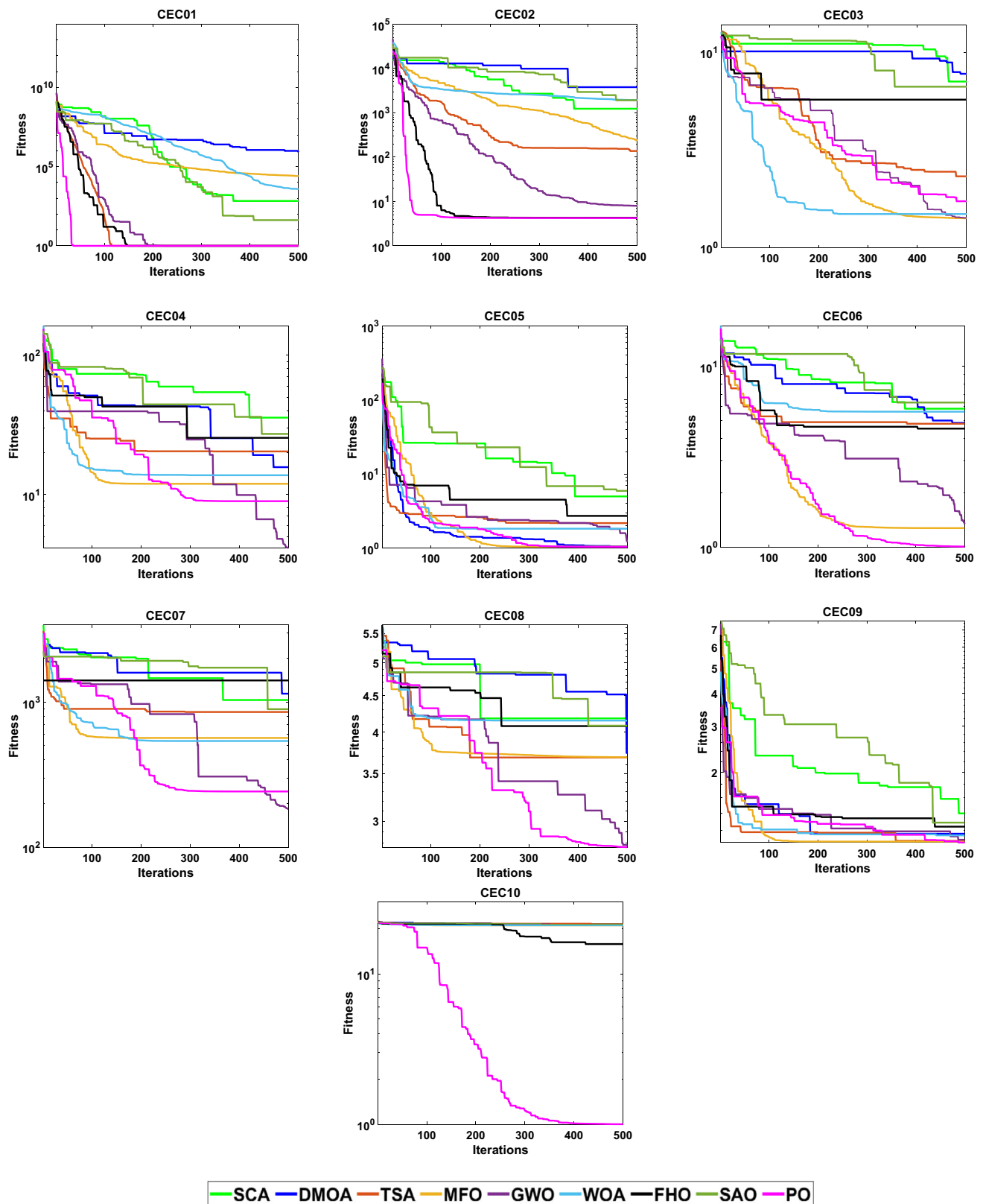


Fig. 14 Convergence curve of CEC2019 benchmarks

Table 10 The P-value obtained by *T*-test in F1–F13 benchmark with 30 dimension

Function	SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO
F1	2.2224E – 03	2.2224E – 03	1.7790E – 04	1.7790E – 04	1.7790E – 04	1.7790E – 04	2.2433E – 03	1.5701E – 04
F2	5.0000E – 04	2.2224E – 03	1.4000E – 03	1.5701E – 04	1.7790E – 04	3.7800E – 02	2.0000E – 04	1.5701E – 04
F3	2.2224E – 03	2.2224E – 03	1.0550E – 01	1.5701E – 04	2.8000E – 03	1.5701E – 04	3.0190E – 01	1.7700E – 02
F4	2.2224E – 03	2.2224E – 03	2.0000E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04	9.8000E – 03	1.8230E – 01
F5	7.0300E – 02	2.2224E – 03	1.7790E – 04	8.3000E – 03	1.5701E – 04	1.5701E – 04	2.2433E – 03	1.5701E – 04
F6	2.2224E – 03	2.2224E – 03	1.7790E – 04	8.2700E – 02	1.5701E – 04	1.5701E – 04	1.3800E – 02	4.0000E – 04
F7	2.2224E – 03	2.2224E – 03	1.7790E – 04	1.4700E – 02	1.5701E – 04	1.5701E – 04	2.2433E – 03	1.5701E – 04
F8	2.2224E – 03	5.3900E – 02	1.7790E – 04	1.7790E – 04	1.5701E – 04	1.5701E – 04	2.2433E – 03	1.5701E – 04
F9	2.2224E – 03	2.2224E – 03	1.7790E – 04	1.5701E – 04	1.7400E – 02	1.0000E + 00	3.2560E – 01	3.1000E – 03
F10	2.2224E – 03	2.2224E – 03	1.7790E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04	2.1190E – 01	5.7900E – 02
F11	2.9262E – 03	1.7790E – 04	1.7790E – 04	1.5701E – 04	7.0500E – 02	3.2560E – 01	1.0000E + 00	6.5000E – 03
F12	8.7300E – 02	1.7790E – 04	1.7790E – 04	3.2560E – 01	1.5701E – 04	1.5701E – 04	2.2433E – 03	1.5701E – 04
F13	1.3280E – 01	1.7790E – 04	1.7790E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04	2.2433E – 03	2.5840E – 01

Table 11 The P-value obtained by *T*-test in F1–F13 benchmark with 100 dimension

Function	SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO
F1	2.5446E – 03	2.5446E – 03	2.0370E – 04	2.0370E – 04	2.0370E – 04	2.7090E – 01	2.8390E – 01	1.7978E – 04
F2	2.5446E – 03	1.0500E – 02	2.0370E – 04	1.7978E – 04	2.0370E – 04	1.1470E – 01	1.6000E – 02	1.7978E – 04
F3	2.5446E – 03	2.5446E – 03	2.0370E – 04	1.7978E – 04	1.7978E – 04	1.7978E – 04	5.4300E – 02	1.7978E – 04
F4	2.5446E – 03	2.5446E – 03	2.0370E – 04	1.7978E – 04	1.7978E – 04	1.7978E – 04	7.7000E – 03	1.9410E – 01
F5	2.5446E – 03	2.5446E – 03	2.0370E – 04	1.7978E – 04	1.7978E – 04	1.7978E – 04	1.5300E – 02	1.6140E – 01
F6	2.5446E – 03	2.5446E – 03	2.0370E – 04	1.7978E – 04	1.7978E – 04	1.7978E – 04	2.5685E – 03	1.7978E – 04
F7	2.5446E – 03	2.5446E – 03	2.0370E – 04	1.7978E – 04	1.7978E – 04	1.7978E – 04	2.5685E – 03	1.7978E – 04
F8	2.5446E – 03	2.8820E – 01	2.0370E – 04	2.0370E – 04	1.7978E – 04	1.7978E – 04	2.5685E – 03	1.7978E – 04
F9	2.5446E – 03	2.5446E – 03	2.0370E – 04	1.7978E – 04	1.7978E – 04	3.2560E – 01	1.0000E + 00	1.7978E – 04
F10	2.5446E – 03	2.5446E – 03	1.8520E – 01	1.7978E – 04	1.7978E – 04	1.7978E – 04	4.1000E – 03	1.7978E – 04
F11	3.3505E – 03	2.0370E – 04	9.0000E – 04	1.7978E – 04	1.6240E – 01	1.0000E + 00	1.0000E + 00	7.6600E – 02
F12	3.3505E – 03	2.0370E – 04	2.0370E – 04	1.7978E – 04	1.7978E – 04	1.7978E – 04	2.5685E – 03	1.6150E – 01
F13	3.3505E – 03	2.0370E – 04	2.0370E – 04	1.7978E – 04	1.7978E – 04	1.7978E – 04	2.5685E – 03	1.5750E – 01

different optimization algorithms and enhancing operators. According to all the explanations given, the PUMA algorithm, due to its impressive performance in solving various problems and its innovations, the weaknesses can be ignored because the advantages of this algorithm outweigh its weaknesses.

5 Machine learning application

Meta-heuristic algorithms are utilized in an extensive range of fields, which leads to increased efficiency or improved performance in the output results of models and systems, which has been proven in many types of research. Nowadays, meta-heuristic algorithms are applied for optimization operations in improving various machine learning and

data mining models [72], which increase the performance of these models. In this study, we have evaluated the PO in 4 different fields of machine learning and data mining for a more complete and in-depth review, so that the effect of PO in solving such problems can be discussed and evaluated including data clustering, Multi-Layer Perceptron (MLP) training, community detection, and feature selection. Next, each issue is briefly explained and discussed in the relevant section.

5.1 Data clustering

The first problem to be investigated is the data clustering, which is used to divide a data set include D features N samples and D features, and into K clusters, which are segmented in a D -dimensional space and using criteria

Table 12 The P-value obtained by *T*-test in F1–F13 benchmark with 500 dimension

Function	SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO
F1	2.4868E – 03	2.4868E – 03	1.9907E – 04	1.9907E – 04	1.9907E – 04	2.7710E – 01	2.8280E – 01	1.7570E – 04
F2	2.4868E – 03	3.2560E – 01	1.9907E – 04	3.2560E – 01	1.9907E – 04	2.9630E – 01	3.0000E – 04	1.7570E – 04
F3	2.4868E – 03	2.4868E – 03	1.9907E – 04	1.7570E – 04	1.7570E – 04	1.7570E – 04	1.5660E – 01	1.7570E – 04
F4	2.4868E – 03	2.4868E – 03	1.9907E – 04	1.7570E – 04	1.7570E – 04	1.7570E – 04	4.7000E – 02	1.7570E – 04
F5	2.4868E – 03	2.4868E – 03	1.9907E – 04	1.7570E – 04	1.7570E – 04	1.7570E – 04	2.5102E – 03	1.6800E – 01
F6	2.4868E – 03	2.4868E – 03	1.9907E – 04	1.7570E – 04	1.7570E – 04	1.7570E – 04	2.5102E – 03	3.2060E – 01
F7	2.4868E – 03	2.4868E – 03	1.9907E – 04	1.7570E – 04	1.7570E – 04	1.7570E – 04	2.5102E – 03	3.5000E – 03
F8	2.4868E – 03	3.6200E – 02	1.9907E – 04	1.9907E – 04	1.7570E – 04	1.7570E – 04	2.5102E – 03	1.7570E – 04
F9	2.4868E – 03	2.4868E – 03	1.9907E – 04	1.7570E – 04	1.7570E – 04	3.2560E – 01	1.0000E + 00	1.2000E – 03
F10	2.4868E – 03	2.4868E – 03	9.5000E – 03	1.7570E – 04	1.7570E – 04	1.7570E – 04	6.4100E – 02	1.7570E – 04
F11	3.2745E – 03	1.9907E – 04	3.8600E – 02	1.7570E – 04	2.8400E – 02	1.0000E + 00	1.0000E + 00	3.3500E – 02
F12	3.2745E – 03	1.9907E – 04	1.9907E – 04	1.7570E – 04	1.7570E – 04	1.7570E – 04	2.5102E – 03	8.2900E – 02
F13	3.2745E – 03	1.9907E – 04	1.9907E – 04	1.7570E – 04	1.7570E – 04	1.7570E – 04	2.5102E – 03	5.0000E – 04

Table 13 The P-value obtained by *T*-test in F1–F13 benchmark with 1000 dimension

Function	SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO
F1	7.9117E – 03	7.9117E – 03	6.3333E – 04	6.3333E – 04	6.3333E – 04	1.1900E – 01	2.2870E – 01	5.5897E – 04
F2	1.0000E + 00	1.0000E + 00	6.3333E – 04	1.0000E + 00	6.3333E – 04	3.1960E – 01	7.9860E – 03	5.5897E – 04
F3	7.9117E – 03	7.9117E – 03	6.3333E – 04	5.5897E – 04	5.5897E – 04	5.5897E – 04	2.1590E – 01	5.5897E – 04
F4	7.9117E – 03	7.9117E – 03	6.3333E – 04	5.5897E – 04	5.5897E – 04	5.5897E – 04	1.8700E – 02	5.5897E – 04
F5	7.9117E – 03	7.9117E – 03	6.3333E – 04	5.5897E – 04	5.5897E – 04	5.5897E – 04	7.9860E – 03	1.5000E – 01
F6	7.9117E – 03	7.9117E – 03	6.3333E – 04	5.5897E – 04	5.5897E – 04	5.5897E – 04	7.9860E – 03	7.0000E – 04
F7	7.9117E – 03	7.9117E – 03	6.3333E – 04	5.5897E – 04	5.5897E – 04	5.5897E – 04	7.9860E – 03	5.5897E – 04
F8	7.9117E – 03	2.3590E – 01	6.3333E – 04	6.3333E – 04	5.5897E – 04	5.5897E – 04	7.9860E – 03	5.5897E – 04
F9	7.9117E – 03	7.9117E – 03	6.3333E – 04	5.5897E – 04	5.5897E – 04	1.0000E + 00	1.0000E + 00	5.5897E – 04
F10	7.9117E – 03	7.9117E – 03	6.3333E – 04	5.5897E – 04	5.5897E – 04	5.5897E – 04	3.1500E – 02	5.5897E – 04
F11	1.0417E – 02	6.3333E – 04	6.3333E – 04	5.5897E – 04	3.6300E – 02	1.0000E + 00	1.0000E + 00	8.7000E – 02
F12	1.0417E – 02	6.3333E – 04	6.3333E – 04	5.5897E – 04	5.5897E – 04	5.5897E – 04	7.9860E – 03	1.9400E – 02
F13	1.0417E – 02	6.3333E – 04	6.3333E – 04	5.5897E – 04	5.5897E – 04	5.5897E – 04	7.9860E – 03	2.0900E – 02

[73, 74]. There are different distances, such as cosine, Euclidean, and Manhattan. In this problem, a dataset $S = (x_1, x_2, x_3, \dots, x_n)$ is considered with N samples, and each of the samples in the dataset has a D dimension $(x_{i1}, x_{i2}, x_{i3}, \dots, x_{id})$. If the above dataset is divided into i clusters, then it can be express the problem by Eqs. 39–40.

$$C = (C_1, C_2, C_3, \dots, C_i) \quad (39)$$

$$C_j \neq \emptyset, \forall j \in \{1, 2, 3, \dots, k\} \quad (40)$$

$$\bigcup_{j=1}^k C_j = S$$

$$C_j \cap C_i = \emptyset, \forall j \neq i \text{ and } j, i \in \{1, 2, 3, \dots, k\}$$

Regarding Eqs. (39 and 40), a dataset is subdivided into K clusters, which none of the clusters are null, and the total amount of the clusters is equivalent to the entire dataset, and there is no similarity between the instances in the clusters. The objective function is expressed using Eq. (41):

$$f(S, C) = \sum_{j=1}^n \min\{\|x_j - C_i\|^2\} \quad i = \{1, 2, 3, \dots, k\} \quad (41)$$

Regarding to Eq. 41, every data is defined to the closest cluster head. The Euclidean method is utilized to measure the distance between the cluster head and the sample in this paper, shown in Eq. (42).

Table 14 The P-value obtained by *T*-test in F14–F23 benchmarks

Function	SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO
F14	2.9262E – 03	1.0000E + 00	1.5701E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04	2.2433E – 03	1.7790E – 04
F15	2.9262E – 03	1.7790E – 04	3.9600E – 02	1.5701E – 04	1.5701E – 04	1.5701E – 04	2.2433E – 03	1.5701E – 04
F16	2.9262E – 03	1.0000E + 00	1.5701E – 04	1.0000E + 00	1.5701E – 04	1.0000E + 00	2.2433E – 03	1.5701E – 04
F17	2.9262E – 03	1.0000E + 00	1.5701E – 04	1.0000E + 00	1.5701E – 04	1.5701E – 04	1.7790E – 04	1.5701E – 04
F18	2.9262E – 03	1.0000E + 00	2.4000E – 03	1.0000E + 00	1.5701E – 04	1.5701E – 04	1.7790E – 04	1.5701E – 04
F19	2.9262E – 03	1.0000E + 00	1.5701E – 04	1.0000E + 00	1.5467E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04
F20	2.9262E – 03	1.0000E + 00	1.5467E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04
F21	2.9262E – 03	1.7790E – 04	1.5467E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04
F22	2.9262E – 03	1.7790E – 04	1.5467E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04
F23	2.9262E – 03	1.7790E – 04	1.7790E – 04	1.7790E – 04	1.7790E – 04	1.5701E – 04	1.5701E – 04	1.5701E – 04

Table 15 The P-value obtained by *T*-test in CEC2019 benchmarks

Function	SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO
C1	5.2004E – 03	5.2004E – 03	7.3100E – 02	1.8800E – 02	1.7200E – 02	4.1629E – 04	1.0000E + 00	3.6741E – 04
C2	5.2004E – 03	5.2004E – 03	4.1629E – 04	1.6000E – 03	4.1629E – 04	3.6741E – 04	5.2492E – 03	3.6741E – 04
C3	5.2004E – 03	5.2004E – 03	4.1629E – 04	3.6741E – 04	3.6741E – 04	3.6741E – 04	5.2004E – 03	3.6741E – 04
C4	5.2004E – 03	5.2004E – 03	4.1629E – 04	3.6741E – 04	3.6741E – 04	3.6741E – 04	5.2492E – 03	3.6741E – 04
C5	5.2004E – 03	5.2004E – 03	4.1629E – 04	9.0000E – 04	3.6741E – 04	3.6741E – 04	5.2492E – 03	3.6741E – 04
C6	5.2004E – 03	5.2004E – 03	4.1629E – 04	3.6741E – 04	3.6741E – 04	3.6741E – 04	5.2492E – 03	3.6741E – 04
C7	5.2004E – 03	5.2004E – 03	4.1629E – 04	3.6741E – 04	3.6741E – 04	3.6741E – 04	5.2492E – 03	3.6741E – 04
C8	5.2004E – 03	5.2004E – 03	4.1629E – 04	4.1629E – 04	3.6741E – 04	3.6741E – 04	5.2492E – 03	3.6741E – 04
C9	5.2004E – 03	5.2004E – 03	4.1629E – 04	3.6741E – 04	3.6741E – 04	3.6741E – 04	5.2492E – 03	3.6741E – 04
C10	5.2004E – 03	5.2004E – 03	4.1629E – 04	3.6741E – 04	3.6741E – 04	3.6741E – 04	5.2492E – 03	3.6741E – 04

$$d(x_i, x_j) = \sqrt{\sum_{m=1}^d |x_{im} - x_{jm}|^2} \quad (42)$$

Various standard datasets have been applied to evaluate PO, which is shown in Table 17 for the description of these data sets.

In this sub-section, the parameters of population and iteration of PO and other optimizer are set equal to 30 and 100, respectively, and have been tested and evaluated, which are demonstrated in Table 18 of the obtained results.

The results of solving the clustering problem using PO and other optimization algorithms are depicted in Table 18. In terms of the results displayed, PO has achieved superior quality results compared to competing algorithms in seven out of the ten datasets. However, the GWO algorithm exhibits the second best performance, demonstrating the best results in three datasets. The convergence diagrams obtained from solving the clustering problem are displayed in Fig. 15.

The convergence diagrams presented in Fig. 15 illustrate the progress made in solving the clustering problem. The diagrams clearly demonstrate the favorable convergence behavior of PO in addressing this problem, with a consistent downward trend observed in almost all of them. This pattern indicates the algorithm's ability to overcome local optima and generate improved and optimal solutions throughout the optimization process.

5.2 Multi-layer perceptron (MLP)

Multilayer perceptron is a type of neural feedforward network that includes three general layers such as an output layer, a hidden layer, and an input layer. In this form of neural network, each node is considered a neuron (except for the input nodes), which uses a non-linear activation function. Multilayer perceptron is one of the supervised learning techniques used for training. Moreover, the non-linear activation and multiple layers lead to the differentiation of an MLP with a linear perceptron as it is capable

Table 16 Time Results of benchmark functions (F1–F13), with 30 dimensions

No		SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO	PO
F1	Best	0.0582	0.2708	0.0695	0.0531	0.0846	0.0309	0.1626	0.0934	0.06495
	Worst	0.0839	0.3777	0.0883	0.0868	0.0960	0.0672	0.3120	0.1526	0.68225
	Mean	0.0609	0.2913	0.0722	0.0576	0.0884	0.0359	0.1877	0.1035	0.0872
	STD	0.0046	0.0239	0.0042	0.0060	0.0028	0.0066	0.0359	0.0123	0.1124
F2	Best	0.0657	0.3135	0.0793	0.0597	0.0958	0.0359	0.1865	0.1095	0.06495
	Worst	0.1408	0.3879	0.0972	0.0914	0.1262	0.0617	0.2681	0.1332	0.49605
	Mean	0.0790	0.3431	0.0825	0.0652	0.1009	0.0416	0.2004	0.1195	0.081
	STD	0.0152	0.0195	0.0035	0.0056	0.0056	0.0045	0.0148	0.0053	0.0784
F3	Best	0.1732	0.5447	0.1812	0.1657	0.1985	0.1475	0.3671	0.5205	0.1299
	Worst	0.2233	0.6209	0.2291	0.1962	0.2476	0.2066	0.4431	0.6083	1.3353
	Mean	0.1818	0.5698	0.1974	0.1777	0.2132	0.1580	0.3960	0.5562	0.1734
	STD	0.0105	0.0202	0.0114	0.0066	0.0107	0.0118	0.0184	0.0236	0.2195
F4	Best	0.0638	0.2931	0.0755	0.0582	0.0929	0.0340	0.1880	0.1115	0.06495
	Worst	0.1148	0.3808	0.0978	0.0798	0.1128	0.0591	0.2598	0.1352	0.5644
	Mean	0.0701	0.3072	0.0798	0.0628	0.0993	0.0396	0.2055	0.1162	0.08325
	STD	0.0102	0.0163	0.0055	0.0046	0.0049	0.0044	0.0174	0.0051	0.0909
F5	Best	0.0766	0.3241	0.0878	0.0702	0.1090	0.0475	0.1999	0.1512	0.06495
	Worst	0.1335	0.3834	0.1072	0.0897	0.1276	0.0700	0.2648	0.1832	0.6043
	Mean	0.0837	0.3406	0.0924	0.0741	0.1138	0.0519	0.2153	0.1614	0.0846
	STD	0.0112	0.0137	0.0042	0.0037	0.0039	0.0048	0.0155	0.0067	0.09815
F6	Best	0.0650	0.3122	0.0764	0.0594	0.0922	0.0331	0.1966	0.1060	0.06495
	Worst	0.1040	0.4310	0.0931	0.0827	0.1198	0.0620	0.2578	0.1549	0.6392
	Mean	0.0723	0.3368	0.0796	0.0648	0.0981	0.0398	0.2107	0.1140	0.08575
	STD	0.0088	0.0247	0.0033	0.0048	0.0053	0.0047	0.0147	0.0093	0.10455
F7	Best	0.1433	0.4495	0.1570	0.1365	0.1684	0.1203	0.3250	0.4236	0.06495
	Worst	0.1735	0.5507	0.1862	0.1737	0.1870	0.1433	0.4067	0.4637	0.53725
	Mean	0.1496	0.4818	0.1668	0.1456	0.1744	0.1281	0.3504	0.4354	0.08235
	STD	0.0066	0.0266	0.0062	0.0081	0.0048	0.0048	0.0177	0.0091	0.08595
F8	Best	0.0822	0.4405	0.0892	0.0692	0.1098	0.0520	0.2097	0.1645	0.06495
	Worst	0.1420	0.5301	0.1175	0.0962	0.1336	0.0834	0.2745	0.2125	0.5405
	Mean	0.0917	0.4677	0.0984	0.0755	0.1157	0.0582	0.2255	0.1751	0.0825
	STD	0.0118	0.0206	0.0048	0.0062	0.0055	0.0068	0.0131	0.0095	0.08655
F9	Best	0.0720	0.3254	0.0864	0.0646	0.0917	0.0350	0.1843	0.1560	0.06495
	Worst	0.1295	0.4075	0.1243	0.0876	0.1206	0.0628	0.2692	0.1830	0.5671
	Mean	0.0785	0.3491	0.0939	0.0724	0.1003	0.0406	0.2011	0.1630	0.08335
	STD	0.0101	0.0201	0.0081	0.0045	0.0069	0.0057	0.0154	0.0078	0.0914
F10	Best	0.0791	0.3349	0.0834	0.0701	0.0970	0.0383	0.2025	0.1725	0.06495
	Worst	0.1383	0.4163	0.1171	0.0963	0.1112	0.0756	0.2999	0.2302	0.53605
	Mean	0.0912	0.3541	0.0936	0.0770	0.1015	0.0459	0.2372	0.1909	0.08235
	STD	0.0117	0.0196	0.0067	0.0051	0.0035	0.0072	0.0269	0.0136	0.0857
F11	Best	0.0857	0.3606	0.0928	0.0833	0.1047	0.0488	0.2083	0.2035	0.06495
	Worst	0.2832	0.4324	0.1205	0.1102	0.1315	0.0758	0.2993	0.2462	0.6172
	Mean	0.1009	0.3829	0.0976	0.0914	0.1122	0.0555	0.2316	0.2167	0.08505
	STD	0.0353	0.0207	0.0055	0.0068	0.0057	0.0059	0.0204	0.0103	0.10055
F12	Best	0.2672	0.7119	0.2836	0.2657	0.2969	0.2458	0.5643	0.9432	0.06495
	Worst	0.4694	0.8953	0.3117	0.3305	0.3365	0.2962	0.6453	1.0965	0.53155
	Mean	0.2917	0.7665	0.2951	0.2854	0.3110	0.2637	0.5995	0.9795	0.0822

Table 16 (continued)

No		SCA	DMOA	TSA	MFO	GWO	WOA	FHO	SAO	PO
	STD	0.0353	0.0377	0.0070	0.0146	0.0085	0.0144	0.0199	0.0355	0.0849
F13	Best	0.2798	0.7350	0.2877	0.2821	0.3068	0.2547	0.5958	0.9521	0.06495
	Worst	0.3333	0.9208	0.3585	0.3203	0.3300	0.2874	0.6395	1.0452	0.7457
	Mean	0.2921	0.7809	0.3141	0.2983	0.3179	0.2719	0.6159	0.9969	0.0893
	STD	0.0130	0.0423	0.0171	0.0091	0.0058	0.0085	0.0135	0.0240	0.124

Table 17 Clustering dataset

Data set	Instance number	Number of classes	Number of features
Iris	150	3	4
Wine	178	3	13
CMC	1473	3	9
Glass	214	6	9
Vowel	871	6	3
Cancer	683	2	9
Zoo	101	2	16
Spect	267	2	22
Ionosphere	351	2	34
Blood	748	2	4

of discriminating data that are not linearly separable. Today, to improve MLP performance, optimizers and meta-heuristics are extensively utilized to calculate weights and biases, which ultimately leads to an increase in accuracy. The general scheme of an MLP is shown in Fig. 16.

For evaluation in this section, two standard data sets are used, which are shown in Table 19.

In this sub-section, 30 populations and 500 iterations are used for the optimization operation for all the compared optimizer, and the results are illustrated in Table 20. Also, each dataset is separated into two parts, testing and training, 70 and 30%, respectively.

Upon analyzing the evaluation results, it is evident that PO outperforms the other algorithms under comparison, displaying significantly higher accuracy throughout the operation process. The outcomes indicate that the proposed algorithm exhibits superior performance across all criteria, with particularly notable advancements observed in the diabetes dataset. Subsequently, Fig. 17 presents the convergence diagrams derived from the conducted tests for MLP.

Carefully as shown in Fig. 17, the MLP trained using PO in both data sets has the same continuous reduction pattern, which indicates the improvement operation in all iterations. It avoids falling into the local optimum trap and improves MLP. However, the decreasing pattern until the last iteration depicts that if the iteration increases, it can still increase the accuracy of the MLP performance.

5.3 Community detection

In the community detection problem, the main goal is to search for nodes belonging to a network and graph, which assumes that the nodes of a community have more similarities and common characteristics. To solve this problem, we have used two different objective functions, which are NMI, and Q, which is the majority cost function (Q) is one of the most favored objective functions in the CD, and the reason for this popularity is the ease of implementation, which for more information You can refer to [75]. This cost function is shown in Eq. 43.

$$Q = \frac{1}{2M} \sum \left(a_{ij} - \frac{K_i K_j}{2M} \delta(i, j) \right) \quad (43)$$

The second objective function is called Normalized Mutual Information (NMI), which is utilized as a measure to determine the efficiency of community detection and is generally provided to measure the performance of optimizers. One can refer to [76] for more information. This cost function is shown using Eq. (44)

$$NMI = \frac{-2 \sum_{i=1}^{C_A} \sum_{j=1}^{C_B} C_{ij} \log(N_{ij}N/N_i N_j)}{\sum_{i=1}^{C_A} N_i \log(N_i/N) + \sum_{j=1}^{C_B} N_j \log(N_j/N)} \quad (44)$$

PO and other optimizer algorithms have been evaluated using the first objective function in this sub-section. Furthermore, we tested using the second objective function, and the results gained from these tests are demonstrated in Tables 22, 23.

Table 18 Data clustering results

Dataset	Metric	Puma	PSO	GWO	WOA	BAT	ABC
Iris	Min	96.7	96.68	97.26	107.72	97.14	111.98
	Max	97.76	131.86	126.26	159.3	132.6	128.63
	Mean	97.05	111.76	104.35	134.14	114.52	118.56
	STD	0.32	11.13	10.13	13.66	11.7	3.94
Wine	Min	16,297.05	16,350.18	16,328.12	16,425.02	17,000.97	16,331.84
	Max	16,307.93	17,130.32	16,446.56	17,224.56	21,684.73	16,523.07
	Mean	16,302.9	16,506.68	16,384.73	16,671.04	18,826.89	16,396.58
	STD	2.85	178.82	32.06	196.03	1096.48	44.29
Cmc	Min	5547.05	5695.65	5692.78	6058.82	5685.47	5713.5
	Max	5649.42	6662.63	6084.3	7789.88	8217.47	6071.52
	Mean	5579.92	6042.97	5882.68	6609.32	6877.2	5928.49
	STD	24	219.4	111.36	456.03	672.73	96.98
Cancer	Min	2968.02	3086.35	2965.13	3175.64	3724.98	3092.11
	Max	2993.14	4278.38	2974.53	4690	6331.68	3334.59
	Mean	2973.56	3621.2	2966.71	3688.46	5210.4	3209.7
	STD	6.77	338.55	2.21	313.07	668.38	66.53
Glass	Min	246.79	269.32	341.41	367.79	332.58	292.28
	Max	286.89	415.55	490.49	567.45	506.15	343.51
	Mean	267.35	340.16	401.5	457.97	410.95	320.94
	STD	8.81	36.61	33.95	58.27	52	14.3
Vowel	Min	149,631.19	150,132.67	152,795.84	166,767.87	193,723.62	182,973.26
	Max	165,687.91	187,659.45	171,370.49	231,849.73	273,141.71	198,474.88
	Mean	154,364.33	170,295.81	160,830.22	196,908.92	240,118.12	191,063.9
	STD	4222.33	10,213.35	5076.01	17,999.83	18,427.38	4382.45
Zoo	Min	182.23	184.17	182.81	206.53	181.69	185.16
	Max	185.19	204.32	197.01	233.29	210.64	192.93
	Mean	183.41	190.46	187.05	218.23	191.05	189.07
	STD	0.76	4.83	4.33	6.59	7.54	1.74
Spect	Min	498.23	525.21	496.4	524.52	502.8	514.51
	Max	506.51	561.41	517.51	564.18	544.84	530.19
	Mean	501.96	541.97	501.94	547.15	522.46	522.54
	STD	2.05	9.74	5.12	10.54	9.61	4.09
Ionosphere	Min	875.1	1007.62	811.41	977.85	988.25	1004.63
	Max	982.09	1140.34	933.91	1065.62	1142.86	1064.07
	Mean	914.36	1079.11	864.7	1035.92	1074.92	1035.91
	STD	26.05	38.47	35.61	22.54	42.03	16.2
Blood	Min	407,714.36	407,719.67	407,837.69	408,290.95	417,703.13	408,918.47
	Max	407,720.16	419,988.16	409,436	414,162.35	605,423.89	411,881.95
	Mean	407,715.44	409,533.21	408,664.87	410,024.01	461,177.14	410,159.86
	STD	1.27	2998.85	425.54	1378.23	52,994.86	857.62

Bold Values indicate best solution obtained

In addition, we have used 4 standard data sets to analyze PO and competitors, which are based on four real complex networks with different dimensions and characteristics. The specifications and explanations of each dataset are illustrated in Table 21.

Firstly, the optimizer algorithms have been analyzed using the modularity cost function, and the results obtained are depicted in Table 22.

Regarding the outcomes presented in Table 22, PO has demonstrated superior performance compared to other optimization algorithms in three out of four datasets, exhibiting enhanced accuracy in detecting consensus. However, in the Karate dataset, ABC algorithm has shown a comparable performance, while in the Dolphine dataset, it has managed to find the optimal solution. Nonetheless, PO has also achieved the second-best performance in the

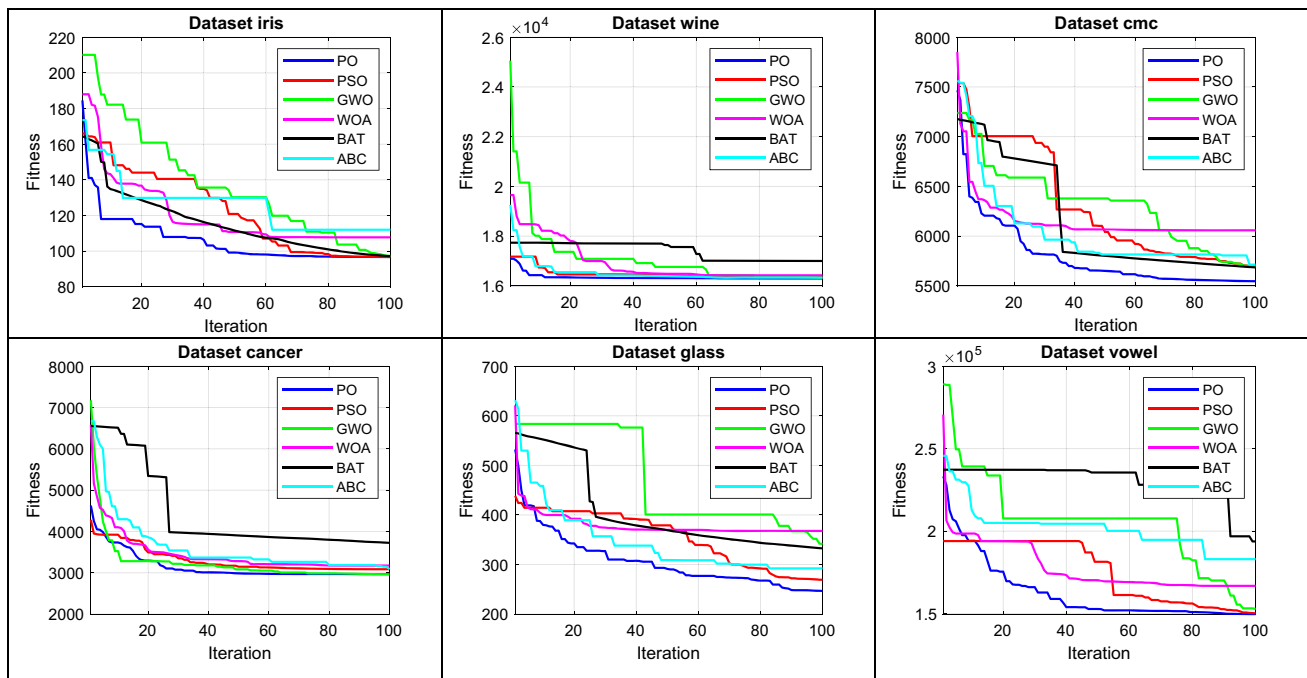


Fig. 15 Convergences curve of the data clustering problem

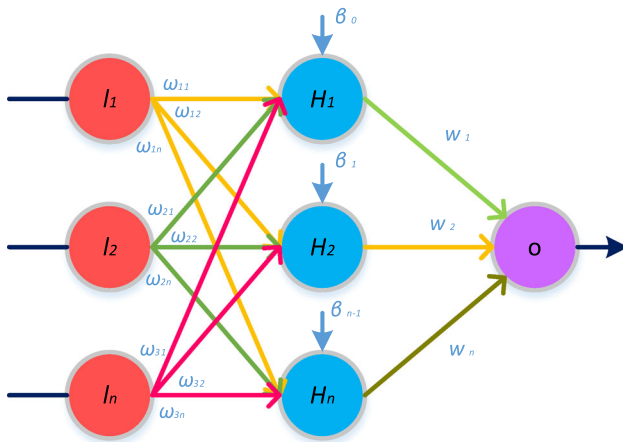


Fig. 16 MLP overview

Table 19 MLP dataset

Data set	Instance number	Number of classes	Number of features
diabetes	768	2	8
wdbc	569	2	30

Dolphine dataset. Figure 18 displays the convergence curve and the community detected by the PO diagram.

By looking carefully at the convergence diagram and community diagram detected in Fig. 18, it can be concluded that PO has a lot of convergence and is rapidly

Table 20 Multi-layer perceptron training results

Dataset	Algorithms	AUC	TP	FP	FN	TN
Diabetes	PO	0.8603	146	41	20	55
	GA	0.8518	147	44	19	52
	PSO	0.8282	150	46	16	50
	BBO	0.8567	145	43	21	53
	TSA	0.8386	143	42	23	54
	FFA	0.8530	150	46	16	50
wdbc	PO	0.9944	68	2	6	118
	GA	0.9937	66	3	8	117
	PSO	0.9940	67	3	7	117
	BBO	0.9935	67	3	7	117
	TSA	0.9925	68	2	6	118
	FFA	0.9777	65	8	9	112

Bold Values indicate best solution obtained

improving the solutions and continuously improving the solutions in each iteration. On the other hand, the convergence graph demonstrates the escape from the local optimal trap repeatedly by the PO. But regarding the diagram of the detected community, it can be perceiving that in general 2 communities have been detected, that almost half of the nodes are correctly detected, which can improve the accuracy by increasing the iteration. Table 23 demonstrates the results of the PO test and other optimizers applying the NMI function.

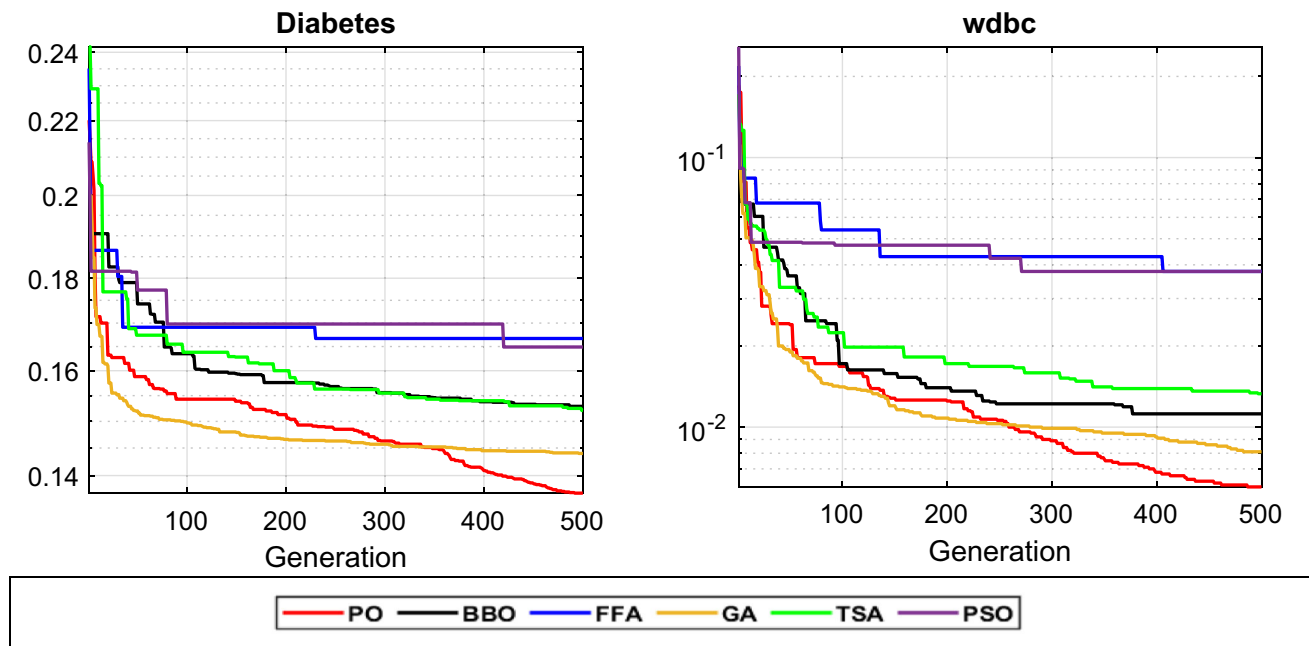


Fig. 17 Convergence curve of MLP

Table 21 Community detection dataset description

Dataset	Node	Edge	Community
Book [77, 78]	105	441	3
Football [77, 78]	115	613	12
Dolphine [79]	62	159	5
Karate [80]	34	78	2

Table 22 Community detection result with Q fitness function

Data set	GA	GWO	ABC	CSO	PO
Karate	0.1695	0.2636	0.2945	0.1774	0.2945
Dolphine	0.1334	0.1298	0.1536	0.1155	0.1478
Football	0.0340	0.0329	0.0387	0.0328	0.0541
Book	0.1118	0.1367	0.0946	0.0569	0.1473

Bold Values indicate best solution obtained

Focusing on the results shown in Table 23, PO has once again demonstrated very good performance and was able to obtain significant good results in three data sets out of four datasets compared to other optimizers, which shows the capability of exploratory and intensification components. But in the Karate dataset, considering that the superior performance belongs to ABC, PO has also been able to

show the second-best performance. Figure 19 shows the convergence diagram and the detected community diagram.

According to the graphs shown in Fig. 19, PO once again has a good performance in convergence, and even in the low number of iterations, it has been able to achieve optimization operations much better than the compared optimizer. The observed convergence pattern indicates the solutions' improvement in all optimization steps. On the other hand, according to the community diagram detected by PO, this algorithm has been able to recognize almost two communities acceptably even with a deficient number of iterations, which indicates the huge power of the PO in exploring the problem even in low iteration.

5.4 Feature selection

The feature selection problem is the critical processes in the pre-processing data phase, through which an optimal subset of the main and essential features is selected. The main goal of this issue is to remove redundant or unnecessary features to reduce the execution time and data scale. This problem has a dataset with n samples $i = 1, 2, \dots, n$. Each sample contains D number of features $j = 1, 2, \dots, D$ where $i = 1, 2, \dots, n$ also refers to the i^{th} sample and the j^{th} feature. Also, each sample belongs to a class, as shown in Fig. 20.

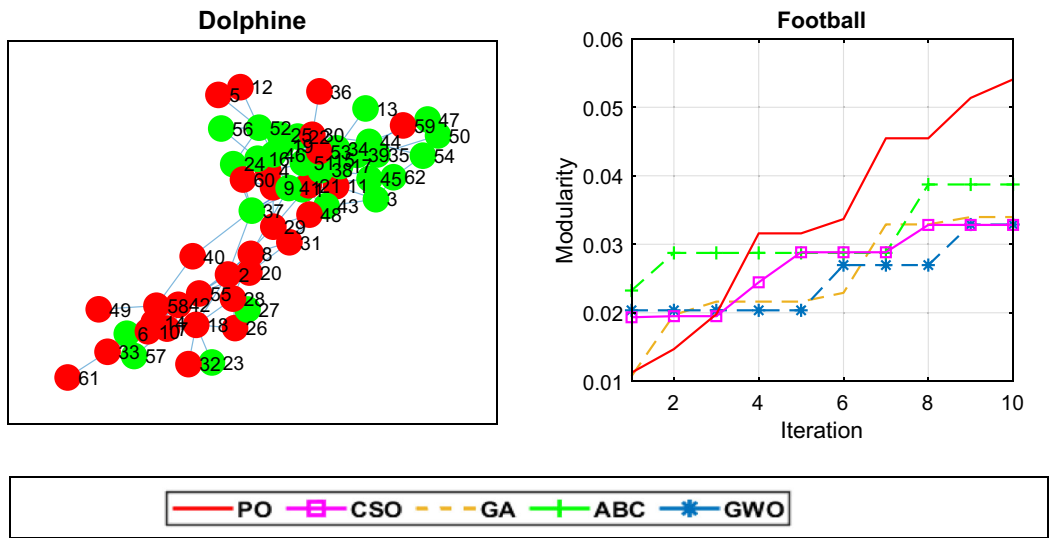


Fig. 18 Convergence curve and recognized community diagram by PO with Q fitness function

Table 23 Community detection result with NMI fitness function

Data set	GA	GWO	ABC	ABC	PO
Karate	0.2616	0.5106	0.7329	0.7329	0.5738
Dolphine	0.1395	0.3471	0.4593	0.4593	0.5445
Football	0.3182	0.2933	0.3070	0.3070	0.3514
Book	0.1090	0.1162	0.1473	0.1473	0.2050

Bold Values indicate best solution obtained

For testing and evaluation, the datasets available in the data cluster section and Table 17 have been used, and the population and repetition parameters in this section have been set with values of 5 and 50, respectively, and the KNN classifier has been used with the value of 5n. The results obtained from the feature selection test applying PO and other compared optimizers in this section are shown in Table 24.

The results depicted in Table 24 regarding the accuracy criterion in the Iris data set show that all optimizers have achieved a common result. Still, in the Wine, Glass, and Ionosphere data sets, the PO algorithm has obtained the best solution and achieved better quality solutions compared to other optimizers used for comparison in this part. On the other hand, according to the time criterion, PO has an average performance among the optimizers for solving problems. Still, due to obtaining good accuracy, this case can be ignored. Figure 21 shows the convergence diagram obtained from solving the feature selection problem.

By paying close attention to the convergence graphs shown in Fig. 21, PO has fast convergence and has been able to discover perfect solutions in the initial stages of optimization procedure compared to other optimizers. In general, according to all the tests conducted by PO, the excellent performance of this algorithm can be seen because it has been able to produce reasonable solutions in almost all sectors, and as a result, it is superior to other algorithms. But good performance requires the ability in exploration and exploitation phases to produce diverse solutions in parallel in all optimization stages and then quickly move to promising areas to find better quality solutions. The phase change can play a crucial role in the performance of the PO because the phase change at the appropriate stage of the optimization operation can help in the capability of an algorithm.

6 Conclusion and future work

Nowadays, the use of meta-heuristic algorithms to optimize all kinds of real-world problems has become necessary because they can be implemented simply and easily escape the local optimality trap. However, each meta-heuristic algorithm has a different approach to each other because each of these algorithms is inspired by a different natural phenomenon, implemented and formulated. Meta-heuristic algorithms have different performances from each other because they use different operators from each other. The Puma algorithm has different operators in both the

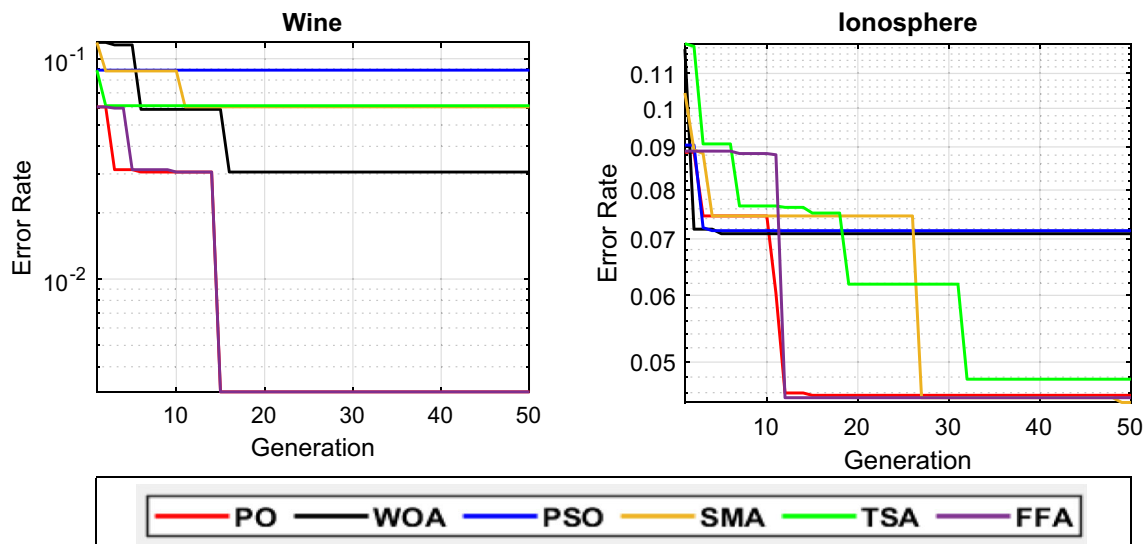


Fig. 21 Convergence cure of feature selection problem

Table 25 Details of unimodal benchmark functions

No	Type	Function	Dimensions	Range	F_{min}
F1	SU	$f(x) = \sum_{i=1}^d x_i^2$	30,100,500,1000	$[-100, 100]^d$	0
F2	SU	$f(x) = \sum_{i=1}^d x_i + \prod_{i=1}^d x_i $	30,100,500,1000	$[-10, 10]^d$	0
F3	SU	$f(x) = \sum_{i=1}^d \left(\sum_{j=1}^i x_j \right)^2$	30,100,500,1000	$[-100, 100]^d$	0
F4	SU	$f(x) = \max_i \{ x_i , 1 \leq i \leq d\}$	30,100,500,1000	$[-100, 100]^d$	0
F5	SU	$f(x) = \sum_{i=1}^{d-1} \left[100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2 \right]$	30,100,500,1000	$[-30, 30]^d$	0
F6	SU	$f(x) = \sum_{i=1}^d ([x_i + 0.5])^2$	30,100,500,1000	$[-100, 100]^d$	0
F7	SU	$f(x) = \sum_{i=1}^d ix_i^4 + \text{random}[0, 1)$	30,100,500,1000	$[-128, 128]^d$	0

problem Puma reached higher quality results in 7 data sets out of 10 data sets. Moreover, Puma algorithm had far the best performance in the more than about 90 percent of datasets compared to the other algorithm in the Multi-layer perceptron, feature selection and community detection problems.

In the future, researchers can use PO to evaluate a broad range of problems and to solve different problems. They can provide a discrete version of this algorithm due to its nature. This algorithm can even be used using a series of special operators to introduce a robust discrete algorithm for solving optimization problems. Furthermore, this algorithm can be applied to solve real-world engineering

problems, and the performance of this algorithm in solving such problems can be discussed and investigated because algorithms in solving constructed problems can show different reactions. On the other hand, due to parameters in the PO, it can provide a version of the PO in which the parameters are set automatically. Finally, a multi-objective version of PO can be presented, which can be utilized to solve multi-objective problems.

Appendix 1

See Tables 25, 26, 27, 28.

Table 26 Details of multimodal benchmark functions

No	Type	Function	Dimensions	Range	F_{min}
F8	SM	$f(x) = -\sum_{i=1}^d (x_i \sin(\sqrt{ x_i }))$	30,100,500,1000	$[-500, 500]^d$	$-418.9829 \times n$
F9	SM	$f(x) = 10d + \sum_{i=1}^d [x_i^d - 10 \cos(2\pi x_i)]$	30,100,500,1000	$[-5, 12, 5, 12]^d$	0
F10	SM	$f(x) = -20 \exp\left(-0.2 \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2}\right) - \exp\left(\frac{1}{d} \sum_{i=1}^d \cos 2\pi x_i\right) + 20 + e$	30,100,500,1000	$[-32, 32]^d$	0
F11	SM	$f(x) = \frac{1}{4000} \sum_{i=1}^d x_i^2 - \prod_{i=1}^d \cos\left(\frac{x_i}{\sqrt{i}}\right) + 1$	30,100,500,1000	$[-600, 600]^d$	0
F12	SM	$f(x) = \frac{\pi}{m} \left\{ 10 \sin(\pi y_1) + \sum_{i=1}^m (y_i - 1)^2 [1 + 10 \sin^2(\pi y_{i+1})] + (y_n - 1)^2 \right\} + \sum_{i=1}^m u(x_i, 10, 100, 4)$, where $y_i = 1 + \frac{x_i + 1}{4}$, $u(x_i, a, i, n) = \begin{cases} k(x_i - a)^n, & x_i > a; \\ 0, & -a \leq x_i \leq a; \\ k(-x_i - a)^n, & x_i < -a, \end{cases}$	30,100,500,1000	$[-50, 50]^d$	0
F13	SM	$f(x) = 0.1 \left\{ \sin^2(3\pi x_1) + \sum_{i=1}^d (x_i - 1)^2 [1 + \sin^2(3\pi x_i + 1)] \right\} + (x_d - 1)^2 [1 + \sin^2(2\pi x_d)] + \sum_{i=1}^d U(x_i, 5, 100, 4)$	30,100,500,1000	$[-50, 50]^d$	0

Table 27 Details of fixed-dimension multimodal benchmark functions

No	Type	Function	Dimensions	Range	F_{min}
F14	FM	$f(x) = \left[\frac{1}{500} + \sum_{i=1}^{25} \frac{1}{i + \sum_{j=1}^{25} (x_j - a_{ij})^6} \right]^{-1}$	2	$[-65, 65]^d$	1
F15	FM	$f(x) = \sum_{i=1}^d \left[a_i - \frac{x_i(b_i^2 + b_i x_i)}{b_i^2 + b_i x_i + x_i} \right]^2$	4	$[-5, 5]^d$	0.00030
F16	FM	$f(x) = 4x_1^2 - 2.1x_1^4 + \frac{1}{3}x_1^6 + x_1x_2 - 4x_2^2 + 4x_2^4$	2	$[-5, 5]^d$	− 1.0316
F17	FM	$f(x) = \left(x_2 - \frac{5.1}{4\pi^2}x_1^2 + \frac{5}{\pi}x_1 - 6 \right)^2 + 10 \left(1 - \frac{1}{8\pi} \right) \cos x_1 + 10$	2	$[-5, 5]^d$	0.398
F18	FM	$f(x) = \left[1 + (x_1 + x_2 + 1)^2 (19 - 14x_1 + 3x_1^2 - 14x_2 + 6x_1x_2 + 3x_2^2) \right] \times$ $\left[30 + (2x_1 - 3x_2)^2 \times (18 - 32x_1 + 12x_1^2 + 48x_2 - 36x_1x_2 + 27x_2^2) \right]$	2	$[-2, 2]^d$	3
F19	FM	$f(x) = -\sum_{i=1}^4 a_i \exp \left(-\sum_{j=1}^3 b_{ij} (x_j - p_{ij})^2 \right)$	3	$[1, 3]^d$	− 3.86
F20	FM	$f(x) = -\sum_{i=1}^4 a_i \exp \left(-\sum_{j=1}^6 b_{ij} (x_j - p_{ij})^2 \right)$	6	$[0, 1]^d$	− 3.32
F21	FM	$f(x) = -\sum_{i=1}^5 \left[(X - a_i)(X - a_i)^T + c_i \right]^{-1}$	4	$[0, 10]^d$	− 10.1532
F22	FM	$f(x) = -\sum_{i=1}^7 \left[(X - a_i)(X - a_i)^T + c_i \right]^{-1}$	4	$[0, 10]^d$	− 10.4028
F23	FM	$f(x) = -\sum_{i=1}^{10} \left[(X - a_i)(X - a_i)^T + c_i \right]^{-1}$	4	$[0, 10]^d$	− 10.5363

Table 28 The 100-digit challenge basic test functions

Number	Functions	$F_i^* = F_i(x^*)$	Dim	Search range
1	Storn's Chebyshev polynomial fitting problem	1	9	[− 8192, 8192]
2	Inverse Hilbert matrix problem	1	16	[− 16384, 16384]
3	Lennard–Jones minimum energy cluster	1	18	[− 4, 4]
4	Rastrigin's function	1	10	[− 100, 100]
5	Griewangk's function	1	10	[− 100, 100]
6	Weierstrass function	1	10	[− 100, 100]
7	Modified Schwefel's function	1	10	[− 100, 100]
8	Expanded Schaffer's F6 function	1	10	[− 100, 100]
9	Happy cat function	1	10	[− 100, 100]
10	Happy cat function	1	10	[− 100, 100]

Author contributions Methodology, BA, NK; Software, BA, NK; Investigation, SB and SK and LA; Writing–original draft, BA and NA; Writing–review & editing, SK, PT and FS; Supervision, SM; Project administration, SM All authors have read and agreed to the published version of the manuscript.

Funding The authors have not disclosed any funding.

Data availability Enquiries about data availability should be directed to the authors.

Declarations

Competing interests The authors declare no competing interests.

References

1. Floudas, C.A., Gounaris, C.E.: A review of recent advances in global optimization. *J. Global Optim.* **45**, 3–38 (2009)
2. Törn, A., Zilinskas, A.: *Global optimization*. Springer, Berlin (1989)
3. Parsopoulos, K.E., Vrahatis, M.N.: Recent approaches to global optimization problems through particle swarm optimization. *Nat. Comput.* **1**, 235–306 (2002)
4. Beyer, H.-G., Sendhoff, B.: Robust optimization—a comprehensive survey. *Comput. Methods Appl. Mech. Eng.* **196**(33–34), 3190–3218 (2007)
5. Gharehchopogh, F.S., Gholizadeh, H.: A comprehensive survey: whale optimization algorithm and its applications. *Swarm Evol. Comput.* **48**, 1–24 (2019)
6. Talbi, E.-G.: *Metaheuristics: from design to implementation*. John Wiley & Sons, Hoboken (2009)
7. Khodadadi, N., et al.: Chaotic stochastic paint optimizer (CSPO). In: *Proceedings of 7th International Conference on Harmony Search Soft Computing and Applications: ICHSA 2022*. Springer, Singapore (2022)
8. Deb, K., Deb, K.: Multi-objective optimization. In: *Search methodologies: introductory tutorials in optimization and decision support techniques*, pp. 403–449. Springer, Boston (2013)
9. Storn, R., Price, K.: Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces. *J. Global Optim.* **11**(4), 341 (1997)
10. Kennedy, J., Eberhart, R.: Particle swarm optimization. In: *Proceedings of ICNN'95-international conference on neural networks*. IEEE (1995)
11. Trojovský, P., Dehghani, M.: Pelican optimization algorithm: a novel nature-inspired algorithm for engineering applications. *Sensors* **22**(3), 855 (2022)
12. Rashedi, E., Nezamabadi-Pour, H., Saryazdi, S.: GSA: a gravitational search algorithm. *Inf. Sci.* **179**(13), 2232–2248 (2009)
13. El-kenawy, E.-S.M., et al.: Al-Biruni Earth Radius (BER) metaheuristic search optimization algorithm. *Comput. Syst. Sci. Eng.* **45**, 1917–1934 (2023)
14. Rao, R.V., Savsani, V.J., Vakharia, D.: Teaching–learning-based optimization: a novel method for constrained mechanical design optimization problems. *Comput. Aided Des.* **43**(3), 303–315 (2011)
15. Shayanfar, H., Gharehchopogh, F.S.: Farmland fertility: a new metaheuristic algorithm for solving continuous optimization problems. *Appl. Soft Comput.* **71**, 728–746 (2018)
16. Rardin, R.L., Rardin, R.L.: *Optimization in operations research*. Prentice Hall Upper Saddle River, NJ (1998)
17. Rao, S.S.: *Engineering optimization: theory and practice*. John Wiley & Sons, Hoboken (2019)
18. Khodadadi, N., Talatahari, S., Gandomi, A.H.: ANNA advanced neural network algorithm for optimisation of structures. *Proc. Inst. Civil Eng. Struct. Build.* (2023). <https://doi.org/10.1680/jstbu.22.00083>
19. Wolpert, D.H., Macready, W.G.: No free lunch theorems for optimization. *IEEE Trans. Evol. Comput.* **1**(1), 67–82 (1997)
20. Mirjalili, S.: The ant lion optimizer. *Adv. Eng. Softw.* **83**, 80–98 (2015)
21. Yapici, H., Cetinkaya, N.: A new meta-heuristic optimizer: Pathfinder algorithm. *Appl. Soft Comput.* **78**, 545–568 (2019)
22. Faramarzi, A., et al.: Equilibrium optimizer: a novel optimization algorithm. *Knowl. Based Syst.* **191**, 105190 (2020)
23. Trojovský, P., Dehghani, M.: A new optimization algorithm based on mimicking the voting process for leader selection. *PeerJ Comput. Sci.* **8**, e976 (2022)
24. Yang, X.-S.: Firefly algorithms for multimodal optimization. In: *Stochastic Algorithms: Foundations and Applications: 5th International Symposium, SAGA 2009, Sapporo, Japan*, pp. 26–28. Springer, Berlin (2009)
25. Dorigo, M., Birattari, M., Stutzle, T.: Ant colony optimization. *IEEE Comput. Intell. Mag.* **1**(4), 28–39 (2006)
26. Mirjalili, S., Lewis, A.: The whale optimization algorithm. *Adv. Eng. Softw.* **95**, 51–67 (2016)

27. Karaboga, D., Basturk, B.: A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. *J. Global Optim.* **39**, 459–471 (2007)
28. Mirjalili, S., et al.: Salp swarm algorithm: a bio-inspired optimizer for engineering design problems. *Adv. Eng. Softw.* **114**, 163–191 (2017)
29. Abdollahzadeh, B., et al.: Mountain gazelle optimizer: a new nature-inspired metaheuristic algorithm for global optimization problems. *Adv. Eng. Softw.* **174**, 103282 (2022)
30. Yazdani, M., Jolai, F.: Lion optimization algorithm (LOA): a nature-inspired metaheuristic algorithm. *J. Comput. Design Eng.* **3**(1), 24–36 (2016)
31. Abdollahzadeh, B., Gharehchopogh, F.S., Mirjalili, S.: Artificial gorilla troops optimizer: a new nature-inspired metaheuristic algorithm for global optimization problems. *Int. J. Intell. Syst.* **36**(10), 5887–5958 (2021)
32. Kaveh, A., Talatahari, S., & Khodadadi, N. (2020). Stochastic paint optimizer: theory and application in civil engineering. *Engineering with Computers*, 1–32
33. Kumar, N., Singh, N., Vidyarthi, D.P.: Artificial lizard search optimization (ALSO): a novel nature-inspired meta-heuristic algorithm. *Soft. Comput.* **25**(8), 6179–6201 (2021)
34. Abdollahzadeh, B., Gharehchopogh, F.S., Mirjalili, S.: African vultures optimization algorithm: a new nature-inspired meta-heuristic algorithm for global optimization problems. *Comput. Ind. Eng.* **158**, 107408 (2021)
35. Faramarzi, A., et al.: Marine predators algorithm: a nature-inspired metaheuristic. *Expert Syst. Appl.* **152**, 113377 (2020)
36. Holland, J.H.: Genetic algorithms. *Sci. Am.* **267**(1), 66–73 (1992)
37. Cheraghalipour, A., Hajiaghaei-Keshteli, M., Paydar, M.M.: Tree growth algorithm (TGA): a novel approach for solving optimization problems. *Eng. Appl. Artif. Intell.* **72**, 393–414 (2018)
38. Tang, D., et al.: ITGO: invasive tumor growth optimization algorithm. *Appl. Soft Comput.* **36**, 670–698 (2015)
39. Dehghani, M., et al.: Coati optimization algorithm: a new bio-inspired metaheuristic algorithm for solving optimization problems. *Knowl. Based Syst.* **259**, 110011 (2023)
40. Simon, D.: Biogeography-based optimization. *IEEE Trans. Evol. Comput.* **12**(6), 702–713 (2008)
41. Kirkpatrick, S., Gelatt, C.D., Jr., Vecchi, M.P.: Optimization by simulated annealing. *Science* **220**(4598), 671–680 (1983)
42. Erol, O.K., Eksin, I.: A new optimization method: big bang–big crunch. *Adv. Eng. Softw.* **37**(2), 106–111 (2006)
43. Formato, R.A.: Central force optimization. *Prog. Electromagn. Res.* **77**(1), 425–491 (2007)
44. Abedinpourshotorban, H., et al.: Electromagnetic field optimization: a physics-inspired metaheuristic optimization algorithm. *Swarm Evol. Comput.* **26**, 8–22 (2016)
45. Dehghani, M., Trojovská, E., Trojovský, P.: A new human-based metaheuristic algorithm for solving optimization problems on the base of simulation of driving training process. *Sci. Rep.* **12**(1), 9924 (2022)
46. Hashim, F.A., et al.: Henry gas solubility optimization: a novel physics-based algorithm. *Futur. Gener. Comput. Syst.* **101**, 646–667 (2019)
47. Mirjalili, S., Mirjalili, S.M., Hatamlou, A.: Multi-verse optimizer: a nature-inspired algorithm for global optimization. *Neural Comput. Appl.* **27**, 495–513 (2016)
48. Kumar, M., Kulkarni, A.J., Satapathy, S.C.: Socio evolution & learning optimization algorithm: a socio-inspired optimization methodology. *Futur. Gener. Comput. Syst.* **81**, 252–272 (2018)
49. Zhang, Q., et al.: Collective decision optimization algorithm: a new heuristic optimization method. *Neurocomputing* **221**, 123–137 (2017)
50. Ackerman, B.B., Lindzey, F.G., Hemker, T.P.: Cougar food habits in southern Utah. *J. Wildl. Manag.* **48**, 147–155 (1984)
51. Robinette, W.L., Gashwiler, J.S., Morris, O.W.: Food habits of the cougar in Utah and Nevada. *J. Wildl. Manag.* **23**(3), 261–273 (1959)
52. Knopff, K.H., et al.: Cougar kill rate and prey composition in a multiprey system. *J. Wildl. Manag.* **74**(7), 1435–1447 (2010)
53. Bartnick, T.D., et al.: Variation in cougar (*Puma concolor*) predation habits during wolf (*Canis lupus*) recovery in the southern greater yellowstone ecosystem. *Can. J. Zool.* **91**(2), 82–93 (2013)
54. Kunkel, K.E., et al.: Winter prey selection by wolves and cougars in and near Glacier National Park Montana. *J. Wildl. Manag.* **63**, 901–910 (1999)
55. Murphy, K.M., et al.: Encounter competition between bears and cougars: some ecological implications. *Ursus* **10**, 55–60 (1998)
56. Monroy-Vilchis, O., et al.: Cougar and jaguar habitat use and activity patterns in central Mexico. *Anim. Biol.* **59**(2), 145–157 (2009)
57. Lambert, C.M., et al.: Cougar population dynamics and viability in the Pacific Northwest. *J. Wildl. Manag.* **70**(1), 246–254 (2006)
58. LaRue, M.A., et al.: Cougars are recolonizing the midwest: analysis of cougar confirmations during 1990–2008. *J. Wildl. Manag.* **76**(7), 1364–1369 (2012)
59. Demers, A., et al.: The cougar project: a work-in-progress report. *ACM SIGMOD Rec.* **32**(4), 53–59 (2003)
60. Anderson, C.R., Jr., Lindzey, F.G.: Estimating cougar predation rates from GPS location clusters. *J. Wildl. Manag.* **67**, 307–316 (2003)
61. Drake, J.H., Özcan, E., Burke, E.K.: An improved choice function heuristic selection for cross domain heuristic search. In: *Parallel Problem Solving from Nature-PPSN XII: 12th International Conference, Taormina, Italy. Springer, Berlin* (2012)
62. Yao, X., Liu, Y., Lin, G.: Evolutionary programming made faster. *IEEE Trans. Evol. Comput.* **3**(2), 82–102 (1999)
63. Digalakis, J.G., Margaritis, K.G.: On benchmarking functions for genetic algorithms. *Int. J. Comput. Math.* **77**(4), 481–506 (2001)
64. Mirjalili, S., Mirjalili, S.M., Lewis, A.: Grey wolf optimizer. *Adv. Eng. Softw.* **69**, 46–61 (2014)
65. Mirjalili, S.: SCA: a sine cosine algorithm for solving optimization problems. *Knowl.-Based Syst.* **96**, 120–133 (2016)
66. Agushaka, J.O., Ezugwu, A.E., Abualigah, L.: Dwarf mongoose optimization algorithm. *Comput. Methods Appl. Mech. Eng.* **391**, 114570 (2022)
67. Kaur, S., et al.: Tunicate swarm algorithm: a new bio-inspired based metaheuristic paradigm for global optimization. *Eng. Appl. Artif. Intell.* **90**, 103541 (2020)
68. Mirjalili, S.: Moth-flame optimization algorithm: a novel nature-inspired heuristic paradigm. *Knowl. Based Syst.* **89**, 228–249 (2015)
69. Azizi, M., Talatahari, S., Gandomi, A.H.: Fire hawk optimizer: a novel metaheuristic algorithm. *Artif. Intell. Rev.* **56**(1), 287–363 (2023)
70. Salawudeen, A.T., et al.: A novel smell agent optimization (SAO): an extensive CEC study and engineering application. *Knowl. Based Syst.* **232**, 107486 (2021)
71. Balachandran, M., et al.: Optimizing properties of nanoclay–nitrile rubber (NBR) composites using face centred central composite design. *Mater. Des.* **35**, 854–862 (2012)
72. Gambella, C., Ghaddar, B., Naoum-Sawaya, J.: Optimization problems for machine learning: a survey. *Eur. J. Oper. Res.* **290**(3), 807–828 (2021)
73. Nanda, S.J., Panda, G.: A survey on nature inspired metaheuristic algorithms for partitional clustering. *Swarm Evol. Comput.* **16**, 1–18 (2014)
74. José-García, A., Gómez-Flores, W.: Automatic clustering using nature-inspired metaheuristics: a survey. *Appl. Soft Comput.* **41**, 192–213 (2016)

75. Zhou, H., Zhang, Y., Li, J.: An overlapping community detection algorithm in complex networks based on information theory. *Data Knowl. Eng.* **117**, 183–194 (2018)
76. Jiang, J.Q., McQuay, L.J.: Modularity functions maximization with nonnegative relaxation facilitates community detection in networks. *Phys. A* **391**(3), 854–865 (2012)
77. Kim, P., Kim, S.: Detecting community structure in complex networks using an interaction optimization process. *Phys. A* **465**, 525–542 (2017)
78. Lancichinetti, A., Fortunato, S., Radicchi, F.: Benchmark graphs for testing community detection algorithms. *Phys. Rev. E* **78**(4), 046110 (2008)
79. Lusseau, D.: Evidence for social role in a dolphin social network. *Evol. Ecol.* **21**, 357–366 (2007)
80. Porter, M.A., Onnela, J.-P., Mucha, P.J.: Communities in networks. *Notices of the AMS* **56**(9), 1082–1097 (2009)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Benyamin Abdollahzadeh has a master's degree in computer engineering from Azad University of Urmia, Urmia, Iran. He is a very active and motivated researcher, and he has started his research since early 2019 and now he is the senior researcher in advanced AI and machine learning methods in the Department of Mathematics, Faculty of Science University of Hradec Králové Czech Republic. His primary interest is metaheuristic algorithms, and he is

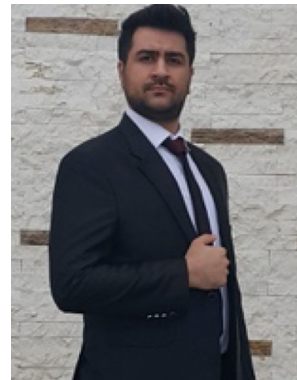
currently researching various projects.



Nima Khodadadi is currently a research assistant and studying for his 2nd Ph.D. at the University of Miami (UM), Miami, USA. He has published over 60 publications with over 1400 citations. Nima graduated with a bachelor's degree in civil engineering. He obtained a master's degree in structural and civil engineering from the University of Tabriz (One of the top ten universities in Iran). His main research interests lie in Artificial Intelligence, Machine

Learning, FRP, Concrete, and Metaheuristic algorithms. In addition, he has been actively involved in engineering optimization, especially evolutionary algorithms, he has been engaged in research in

single and multi-objective engineering optimization, especially in solving large-scale and practical structural design problems. Recently, he has been working on artificial Intelligence techniques and applications for FRP-Integrated concrete structures.



Saeid Barshandeh received the B.Sc. degree in Computer Engineering from Payame Noor University, Urmia, Iran, in 2014, the M.Sc. degree in Computer Engineering, with focus on metaheuristic algorithms and their applications on computer networks from Islamic Azad University, Urmia, Iran, in 2017. His current research interests include Artificial Intelligence, Machine Learning, Deep Learning, Data mining, Optimization, Internet

of Things, Mobile Edge/Fog/Cloud Computing.



Pavel Trojovský received his M.Sc. degree at the University of Hradec Kralove, Czech Republic, in 1989, his Ph.D. at Faculty of Mathematics and Physics, Charles University, Prague, Czech Republic in 2001, and became an associate professor in 2011 in System Engineering and Informatics at University of Pardubice, Pardubice, Czech Republic. His research interests include applied mathematics, computer simulation and optimization and

applications of machine learning.



Farhad Soleimanian Gharehchogh was born in 1979 in Iran. He received his B.S. in Computer Engineering from Shabestar Branch, Islamic Azad University, Iran, in 2002, his M.S. in Computer Engineering from Cukurova University, Adana/Turkey, in 2011 and his Ph.D. degree in Computer Engineering from Hacettepe University, Ankara/Turkey in 2015. He has been an assistant professor in Computer Engineering at Urmia Branch, Islamic Azad University, Urmia, IRAN, since 2015. He is researching metaheuristic algorithms and their application issues. And he published more than 150 papers in international journals and conferences.



El-Sayed M. El-Kenawy is an assistant professor at Delta Higher Institute for Engineering & Technology (DHJET), Mansoura, Egypt. He published more than 35 papers with more than 1200 citations and an H-index of 22. He has launched and pioneered independent research programs. He motivates and inspires his students in different ways by providing a thorough understanding of various computer concepts. He explains complex concepts in an easy-to-

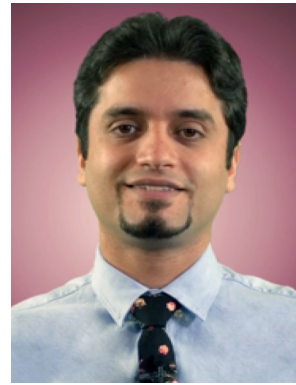
understand manner. He is a reviewer for Computers, Materials & Continua Journal, IEEE Access, and other journals. His research interests include artificial intelligence, machine learning, optimization, deep learning, digital marketing, and data science.



Laith Abualigah received the degree in computer information system from Al-Albait University, Jordan, in 2011, the master's degree in computer science from Al-Albait University, in 2014, and the Ph.D. degree from the School of Computer Science, Universiti Sains Malaysia (USM), Malaysia, in 2018. He is an Associate Professor with the Prince Hussein Bin Abdullah College for Information Technology, Al-Albait University. He is also a Distinguished

Researcher with the School of Computer Science, USM. According to the report published by Clarivate, he was one of the highly cited researchers in 2021 and 2022, and the 1% influential researchers, which depicts the 6,938 top scientists in the world, and a first researcher in the domain of computer science in Jordan, in 2021. According to the report published by Stanford University, in 2020, he is one of the 2% influential scholars, which depicts the 100,000 top scientists in the world. He has published more than 350 journal articles and books, which collectively have been cited more than

12500 times (H-index is 53). His main research interests include arithmetic optimization algorithm (AOA), bioinspired computing, nature-inspired computing, swarm intelligence, artificial intelligence, meta-heuristic modeling, optimization algorithms, evolutionary computations, information retrieval, text clustering, feature selection, combinatorial problems, optimization, advanced machine learning, big data, and natural language processing. He currently serves as an Associate Editor for the Journal of Cluster Computing (Springer), the Journal of Soft Computing (Springer), and the Journal of Engineering Applications of Artificial Intelligence (Elsevier).



Seyedali Mirjalili is currently a Professor and the Founding Director of the Centre for Artificial Intelligence Research and Optimization, Torrens University Australia. He is internationally recognized for his advances in optimization and swarm intelligence, including the first set of algorithms from a synthetic intelligence standpoint—a radical departure from how natural systems are typically understood—and a systematic design framework to

reliably benchmark, evaluate, and propose computationally cheap robust optimization algorithms. He has published over 300 publications with over 94,000 citations and an H-index of 104. As the most cited researcher in robust optimization, he has been on the list of 1% highly-cited researchers and named as one of the most influential researchers in the world by Web of Science for three consecutive years, since 2019. In 2020, he was ranked 21st across all disciplines and 4th in artificial intelligence and image processing in the Stanford University's list of World's Top Scientists. In 2021, The Australian newspaper named him as the top researcher in Australia in three fields of artificial intelligence, evolutionary computation, and fuzzy systems. His research interests include optimization, swarm intelligence, evolutionary algorithms, and machine learning. He is an Associate Editor of several AI journals including Neurocomputing, Applied Soft Computing, Advances in Engineering Software, Computers in Biology and Medicine, Healthcare Analytics, Applied Intelligence, and IEEE ACCESS.

Authors and Affiliations

Benyamin Abdollahzadeh¹ · Nima Khodadadi² · Saeid Barshandeh³ · Pavel Trojovský¹ · Farhad Soleimanian Gharehchopogh⁴ · El-Sayed M. El-kenawy⁵ · Laith Abualigah^{6,7,8} · Seyedali Mirjalili^{9,10}

✉ Nima Khodadadi
nima.khodadadi@miami.edu

¹ Department of Mathematics, Faculty of Science, University of Hradec Králové, 500 03 Hradec Králové, Czech Republic

² Department of Civil and Architectural Engineering, University of Miami, Coral Gables, FL, USA

³ Department of Computer Science, School of Engineering, Afagh Higher Education Institute, Urmia, Iran

⁴ Department of Computer Engineering, Urmia Branch, Islamic Azad University, Urmia, Iran

⁵ Department of Communications and Electronics, Delta Higher Institute of Engineering and Technology, Mansoura 35111, Egypt

- ⁶ Hourani Center for Applied Scientific Research, Al-Ahliyya Amman University, Amman 19328, Jordan
- ⁷ Computer Sciences Department, University of Tabuk, Tabuk 47913, Saudi Arabia
- ⁸ Computer Science Department, Al al-Bayt University, Mafraq 25113, Jordan
- ⁹ Centre for Artificial Intelligence Research and Optimization, Torrens University Australia, Adelaide, Australia
- ¹⁰ Research and Innovation Center, Obuda University, Budapest 1034, Hungary

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH (“Springer Nature”).

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users (“Users”), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use (“Terms”). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval, sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com